

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Aleksandar Babić

**INFORMATIVNA PLATFORMA S
INTELIGENTNIM ASISTENTOM O
ERASMUS+ ISKUSTVU U ŁÓDŻU**

DIPLOMSKI RAD

Varaždin, 2025.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Aleksandar Babić

Matični broj: 0016147642

Studij: Baze podataka i baze znanja

**INFORMATIVNA PLATFORMA S INTELIGENTNIM ASISTENTOM O
ERASMUS+ ISKUSTVU U ŁÓDŽU**

DIPLOMSKI RAD

Mentor:

doc. dr. sc. Bogdan Okreša Đurić

Varaždin, rujan 2025.

Zahvala

Na samom početku želim izraziti iskrenu zahvalnost svom mentoru, doc. dr. sc. Bogdanu Okreši Đuriću, na izdvojenom vremenu, nesebičnoj podršci i strpljenju koje mi je pružio tijekom izrade ovog diplomskog rada.

Posebnu zahvalnost dugujem svojoj obitelji – ocu, majci i sestri – za njihovu podršku i ohrabrenje tijekom cijelog mog studija. Njihova prisutnost i vjera u mene davali su mi snagu u trenucima kada je to bilo najpotrebnije.

Na kraju, veliko hvala svim prijateljima koje sam upoznao tijekom studiranja. Zajedno smo prošli kroz brojne izazove, ali i podijelili mnogo smijeha i lijepih trenutaka koje ću zauvijek nositi sa sobom.

Izjava o izvornosti

Izjavljujem da je ovaj diplomski rad izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Teoretski dio rada istražuje studiranje kao značajan segment u životnom razdoblju pojedinca, s posebnim naglaskom na Erasmus+ program studentske razmjene. Nakon toga, obrađuje se pojam inteligentnih asistenata kao jednog od najpopularnijih suvremenih alata za snalaženje i informiranje. Opisani su različiti tipovi rješenja te kriteriji prema kojima se međusobno razlikuju ovisno o svojoj namjeni. Posebna pažnja posvećena je primjeni inteligentnih asistenata u obrazovnom sustavu, njihovim mogućnostima i izazovima. U nastavku je prikazana Retrieval-Augmented Generation (RAG) metoda, njezina arhitektura te usporedba s fine-tuningom. Istaknute su prednosti RAG pristupa, područja njegove primjene, ali i izazovi poput pojave halucinacija i upravljanja višestrukim izvorima podataka. Teoretski dio završava pregledom dostupnih alata i biblioteka koji se mogu koristiti za implementaciju RAG sustava. Praktični dio rada obuhvaća izradu web-stranice posvećene Erasmus+ studijskom boravku te razvoj inteligentnog asistenta temeljenog na postojećem jezičnom modelu s Groq platforme. U okviru praktičnog dijela provedeno je prikupljanje podataka, dizajn korisničkog sučelja, izrada poslužiteljskog dijela aplikacije te priprema popratne dokumentacije. Na kraju je prikazan rad aplikacije, nakon čega je dan završni osvrt s naglaskom na postignute rezultate te prijedlozima za daljnji razvoj i poboljšanja.

Ključne riječi: Inteligentni asistent, RAG, web-stranica, studiranje, Erasmus+, umjetna inteligencija, LLM, model, informacija.

Sadržaj

1. Uvod	1
2. Studiranje kao dio života pojedinca	2
2.1. Odluka i benefiti	2
2.2. Erasmus+ studentska mobilnost	4
2.2.1. Izazovi i strahovi	5
3. Aktualne tehnologije i alati za uspješno snalaženje i informiranje	7
3.1. Uvod u inteligentne sustave	7
3.2. Tipovi rješenja i kriteriji razlikovanja	10
3.2.1. Asistenti - Siri	11
3.2.2. Kopiloti - Microsoft Copilot Studio	12
3.2.3. Autonomni agenti - ChatGPT Operator	14
3.2.4. Chatbot platforme - Intercom	15
3.3. Inteligentni asistenti u sustavu obrazovanja	16
3.3.1. Primjena inteligentnih asistenata u obrazovanju	16
3.3.2. Izazovi inteligentnih asistenata u obrazovanju	17
4. Retrieval Augmented Generation metoda	18
4.1. Uvod u RAG	18
4.1.1. Povijest i razvoj	19
4.2. Arhitektura RAG modela	22
4.2.1. Faza dohvaćanja (Retriever)	23
4.2.1.1. BM25	24
4.2.1.2. Dense Passage Retrieval	24
4.2.1.3. REALM	24
4.2.2. Faza generiranja (Generator)	25
4.2.2.1. T5	25
4.2.2.2. BART	25
4.2.2.3. GPT-4	25
4.3. Evaluacija i primjena RAG metode	26
4.3.1. Usporedba: RAG vs. Fine-tuning	26
4.3.2. Prednosti RAG metode	27
4.3.3. Najčešći slučajevi primjene	28
4.3.4. Izazovi RAG metode	30
4.3.4.1. Halucinacije	32
4.3.4.2. Upravljanje višestrukim izvorima	34

4.4.	Dostupni alati i biblioteke	35
4.4.1.	Okviri (frameworkovi) za upravljanje RAG-om	35
4.4.1.1.	Haystack	35
4.4.1.2.	LangChain	36
4.4.1.3.	LlamaIndex	36
4.4.2.	Vektorske baze podataka	37
4.4.2.1.	FAISS	39
4.4.2.2.	Qdrant	39
4.4.2.3.	Pinecone	39
4.4.3.	Veliki jezični modeli	39
4.4.3.1.	OpenAI	40
4.4.3.2.	Hugging Face	40
4.4.4.	Ostale tehnologije	41
4.4.4.1.	React	41
4.4.4.2.	SMTP	42
4.4.4.3.	FastAPI	43
5.	Praktični dio rada	44
5.1.	Arhitektura i struktura web-aplikacije	45
5.2.	Prikupljanje podataka	46
5.3.	Korisničko sučelje (Frontend)	51
5.4.	Poslužiteljski dio (Backend)	56
5.4.1.	Poslužitelj	57
5.4.2.	Servisi	57
5.4.3.	Rute	61
5.5.	Prikaz rada aplikacije	66
5.6.	Praćenje performansi i ograničenja korištenih alata	70
6.	Zaključak	71
	Popis literature	77
	Popis slika	79
	Popis tablica	80
	Popis isječaka koda	82

1. Uvod

Na početku rada obrađuje se pojam studiranja kao i svi aspekti koji se usko vežu uz njega. Poseban naglasak je stavljen na važnost studiranja u kontekstu osobnog razvoja, izgradnje karaktera te stjecanja različitih znanja. U nastavku se detaljnije razmatra Erasmus+ program, s ciljem približavanja njegovih mogućnosti i koristi. Glavni cilj je ukazati na izazove poput nedostatka interesa za prijavu na Erasmus+ studijski boravak, prisutnog straha, nesigurnosti te neinformiranosti među studentima.

Kao rješenje na navedene izazove izrađena je web-stranica s inteligentnim asistentom koji je namijenjen da odgovori na specifične upite korisnika. Web-stranica predstavlja sredstvo informiranja prvotno za sve studente koji dolaze u Łódź iz svojih država. Osim toga web-stranica je naravno dostupna i namijenjena i za sve ostale zainteresirane posjetitelje. Web-stranica se sastoji od sljedećih dijelova: početne stranice, stranice s kratkim opisom grada Łódź, stranice koja prikazuje 15 najvećih gradova Poljske, stranice za pregled i upravljanje dokumentima korištenima u RAG sustavu, te stranice s kontaktnom formom. Forma je namijenjena za pitanja na koja AI (eng. Artificial Intelligence) asistent nije znao odgovoriti.

Za izradu AI asistenta odabrano je korištenje RAG metode (eng. Retrieval Augmenting Generation) gdje se uzima postojeći AI model i on se bazira na specifičnom skupu podataka. Za takve potrebe pronađeno je nekoliko lokalnih web-stranica koje opisuju život u Łódźu. Za potrebe rada razvijen je scraper koji dohvaća primarno tekstualni sadržaj s istih tih stranica i sprema ih u JSON (eng. JavaScript Object Notation) obliku. Naknadno su te JSON datoteke pročišćene i iskorištene za daljnje korištenje. Tijekom svoje Erasmus+ studentske mobilnosti imao sam prilike razgovarati s profesoricom dr. Marta Lipnicka na kolegiju Umjetna inteligencija, kojoj sam ukratko predstavio ideju svog diplomskog rada. Naknadno sam je zamolio da mi preporuči nekoliko korisnih stranica koje su vezane za Łódź. Budući da se profesorica rodila u Łódźu i tamo živi te radi cijeli život, smatrao sam da je upravo ona prava osoba kojoj se mogu obratiti s pitanjem o korisnim izvorima informacija vezanima za život u Łódźu.

Za izradu rješenja korištene su sljedeće tehnologije: React (JavaScript) za frontend, Python za razvoj scrapera te Python FastAPI za implementaciju backend logike. AI asistent izrađen je korištenjem tehnologije LangChain, uz model llama-3.1-8b-instant koji je integriran putem API ključa s platforme Groq. Za generiranje vektorskih reprezentacija korišten je HuggingFaceEndpointEmbeddings, također preko API ključa s platforme Hugging Face. Kao vektorska baza podataka korišten je FAISS (eng. Facebook AI Similarity Search), dok je dokumentacija pisana u LaTeX-u, koristeći distribuciju MiKTeX.

Prethodni ulomci uvode nas u temu s kojom se bavi ovaj rad, a to je „Informativna platforma s inteligentnim asistentom o Erasmus+ iskustvu u Łódźu“. Prvotna motivacija za proučavanje ove teme proizišla je iz Erasmus+ studijskog boravka u Łódźu (Poljska) u trajanju jednog semestra.

2. Studiranje kao dio života pojedinca

Važnost učenja se proteže kroz sve uzraste i faze života. Već od malih nogu školska djeca se potiču da marljivo uče jer to na neki način može odrediti njihov put u budućnosti. Unutar ovog poglavlja više je vremena posvećeno studiranju kao važnom segmentu u životnom razdoblju pojedinca. U tom kontekstu visoko obrazovanje može napraviti razliku između uspješne osobe i neuspjeha. Ulaganje u znanje je beskrajno i nikada ne prestaje i može se razmotriti kao investicija u budućnost. Život bez obrazovanja je kao tijelo bez duše, jer obrazovanje pojedincu donosi smisao, znanje i prilike koje oblikuju njegov osobni i profesionalni razvoj. Pomaže u razvoju kritičkog mišljenja, ima plemenit karakter i razvija vještine koje su potrebne za suočavanje sa svim životnim izazovima. Prema riječima stručnjaka iz škola u Bengaluru (Indija), studij je važan jer se koristi za ublažavanje kompleksnosti većine izazova s kojim se susretnu pojedinac [1].

2.1. Odluka i benefiti

Kod donošenja velikih odluka kao što je početak studiranja vrlo je bitno uzeti u obzir nekoliko čimbenika koji mogu oblikovati bolji pogled na razmišljanje o nečemu. Kada se razmišlja o tome da li krenuti studirati ili ne, postoje ljudi koji govore da upisivanje fakulteta više nije presudno ni važno za uspjeh ili da se može dobiti dobar posao bez odlaska na fakultet. Primarni cilj ove sekcije nije zastupanje određene strane, već pružanje uvida u razloge zbog kojih bi upis na fakultet mogao biti dobra odluka. Za početak fakultet priprema radnu snagu za budući posao i može dati značajnu prednost prilikom prijave za posao. Fakultetsko obrazovanje pokazuje poslodavcima da je osoba sposobna dovršiti dugoročni projekt, da je sposobna kritički razmišljati, rješavati probleme i da je sposobna učiti nove stvari. Većina poslova danas zahtijeva barem neko fakultetsko iskustvo i vjerojatno će osobe bez diplome biti u nepovoljnijoj situaciji u usporedbi s konkurencijom. Čak i ako se ne odluči upisati fakultet postoje razni programi, certifikati i tečajevi koji mogu pomoći u razvoju vještina i znanja koje su potrebne za radno mjesto [2].

Umrežavanje je možda i najveći benefit koji se može stećnuti tijekom studiranja. Upoznavanje novih ljudi i gradnja važnih odnosa koji mogu potrajati dugo u budućnosti. Fakultet je mjesto koje nudi priliku upoznavanja drugih studenata s istim/različitim interesima a osim toga postoje i prilike umrežavanja s profesorima i drugim stručnjacima. S toliko studenata i članova fakulteta iz različitih geografskih sredina, sigurno će se pronaći netko tko dijeli iste stavove. Pronaći će se kolega za učenje, kavu ili osoba koja će postati najbolji prijatelj. Osim toga, unutar studentskih kampusa moguće je upoznavanje velikog broja ljudi kroz razna sportska događanja. Kroz upoznavanja, osoba može puno naučiti o sebi i onome što ju zanima. Moguće je upoznavanje s osobama koje dolaze iz svih krajeva svijeta čime se mogu okusiti nove kulture i perspektive. Možda će se pojaviti prilika za život sa studentom/studenticom iz druge zemlje ili će se možda pojaviti profesor koji predaje s međunarodnim iskustvom. Takva iskustva mogu razviti bolje razumijevanje svijeta oko sebe. Pohađanje sajмова karijera na fakultetima ili pridruživanje profesionalnim organizacijama su također neke od prilika koje mogu osnažiti i

izgraditi mrežu koja može obogatiti buduću karijeru [2].

Mnogi ljudi vjeruju da je primarna svrha fakulteta pripremiti osobu za određenu karijeru. Fakulteti općenito nude nekoliko programa koji se mogu upisati, ali fakultetsko obrazovanje je više od "samog dobivanja posla". Jedna od važnijih stvari je razvoj kritičkog razmišljanja i rješavanja problema. U stvarnom svijetu vjerojatno će biti situacija kada će osoba ostati sama i u tom trenutku mora biti sposobna shvatiti što je problem i kako ga riješiti [2].

Na današnjem "konkurentnom tržištu rada", fakultetska diploma može dati prednost koja je potrebna kako bi se osoba izdvojila u moru ostalih kandidata. Iako je za dosta poslova dovoljna srednjoškolska diploma, poslodavci često preferiraju kandidate s fakultetskim obrazovanjem. U zadnje vrijeme mnogo fakulteta nudi i zahtjeva stažiranje ili obavljanje nekog oblika prakse čime se zapravo pruža prilika osobi dati uvid u buduću profesiju [2].

Svaka industrija više-manje zahtjeva da osoba bude u toku s trendovima. Ako je prisutna svjesnost o najnovijima dostignućima i osim toga ako postoji šansa da se primjene u radu time je zapravo pokazana želja za povećanjem produktivnosti i učinkovitosti. Praćenje trendova obično radi razliku i privlači tržište. Na fakultetima su profesori većinom dobro povezani i informirani o najnovijim trendovima u industriji [2].

Fakultet pruža pristup vrijednim resursima tijekom studiranja pa i nakon diplomiranja. Jedan od primjera su sveučilišne knjižnice koje često nude širok izbor knjiga i drugih materijala. Također, nude pomoć u istraživanju od strane knjižničara koji su stručnjaci u pronalaženju potrebnih informacija. Centar karijera je još jedno vrijedno sredstvo koje može pomoći u domenu poput pisanja životopisa pa do vježbanja razgovora za posao [2].

Pojedini ljudi odlaze na fakultet iz razloga jer do tog trenutka još ne znaju što će biti njihov posao i kako će izgledati njihova karijera. U tom slučaju fakultet može biti korisna odskočna daska ako osoba nije sigurna što želi raditi sa svojim životom. Istraživanjem različitih područja studija može se više saznati o svojim interesima i samim time prepoznati koja područja su vrijedna daljnjeg istraživanja. I onda naknadno to može biti izvrsna prilika za istraživanje ili specijalizaciju područja koje je predmet interesa [2].

Osim toga, što fakultet nudi određeni oblik akademskog učenja, on također nudi priliku razvoja osobnih i profesionalnih vještina. Na primjer, učinkovite komunikacijske vještine i vještine upravljanja vremenom ključne su za uspjeh u svakoj karijeri. Izvannastavne aktivnosti također mogu pomoći u razvoju liderskih vještina ili vještina timskog rada [2].

Fakultet predstavlja veliku promjenu jer je to uglavnom za većinu studenata prvi put da žive daleko od doma i sami donose odluke. To podrazumijeva veliku prilagodbu, ali ujedno i uzbudljivo vrijeme za stjecanje neovisnosti. Kako teče proces rasta tako se i osoba mijenja. U svemu tome postoje trenutci kada se pojavi osjećaj izgubljenosti ili nesigurnosti u vezi budućnosti (pokazatelj da osoba brine o sebi) [2].

I za kraj, jedna od zanimljivijih prednosti je prilika za putovanje i upoznavanje drugih drugih kultura. Mnogi fakulteti nude programe studiranja u inozemstvu koji studentima omogućuju da provedu semestar (ili dulje) živeći u drugoj zemlji. To se može smatrati vrijednim iskustvom jer omogućuje izravan uvid u nove običaje i perspektive [2].

2.2. Erasmus+ studentska mobilnost

Erasmus+ je program Europske unije za potporu obrazovanju, osposobljavanju, mladima i sportu u Europi. Program za razdoblje 2021. do 2027. godine snažno se fokusira na socijalnu uključenost, zelenu i digitalnu tranziciju te promicanje sudjelovanja mladih u demokratskom životu. Erasmus+ program nudi mogućnosti i prilike u područjima kao što su: visoko obrazovanje, strukovno obrazovanje i osposobljavanje, školsko obrazovanje, obrazovanje odraslih, mladi i sport. Na stranicama erasmus-plus.ec.europa.eu postoje izvješća o državama s interaktivnim podacima o razmjenama i projektima suradnje za sljedeće države članice: Austrija, Belgija, Bugarska, Hrvatska, Cipar, Češka, Danska, Estonija, Finska, Francuska, Grčka, Mađarska, Island, Irska, Italija, Latvija, Lihtenštajn, Litva, Luksemburg, Malta, Sjeverna Makedonija, Nizozemska, Njemačka, Norveška, Poljska, Portugal, Rumunjska, Srbija, Slovačka, Slovenija, Španjolska, Švedska i Turska [3].

U nastavku su detaljnije razmotreni podaci o Erasmus+ programu za Hrvatsku u 2023. godini. Hrvatska je jedna od 27 zemalja EU-a, čime ima pristup svim Erasmus+ aktivnostima kao što su razmjene, osposobljavanja, financiranja i slično. U nastavku za 2023. godinu su doneseni sljedeći zaključci [4]:

- U 2023. godini najveći broj projekata mobilnosti bio je u području školskog obrazovanja a slijede ga strukovno učenje i obrazovanje,
- Najveći udio sredstava u projektima mobilnosti potrošen je u sektoru visokog obrazovanja,
- Broj sudionika koji putuju u Hrvatsku povećava se svake godine,
- Tijekom 2023. godine došlo je do smanjenja broja sudionika koji su odlazili na razmjene u inozemstvo,
- Digitalna transformacija istaknuta je kao glavni prioritet u ukupnim bespovratnim sredstvima u 2023. godini.

Boravak u inozemstvu predstavlja izazovnu stvar koju poslodavci cijene i uzimaju u obzir prilikom intervjuiranja. Svaka osoba se može prijaviti na Erasmus+ program tako da se za početak obrati međunarodnom ili Erasmus+ uredu matičnog sveučilišta ili visokoškolske ustanove. Visokoškolska ustanova pošiljateljica dužna je odabrati kandidate na pošten i transparentan način. Maksimalno ukupno trajanje boravka u inozemstvu je 12 mjeseci unutar jednog studijskog ciklusa i shodno tome moguće je obaviti više od jedne razmjene unutar tog ograničenja. U nastavku se spominju dvije vrste mobilnosti [5]:

- Dugoročna mobilnost - minimalno 2 mjeseca do maksimalno 12 mjeseci u inozemstvu,
- Kratkotrajna mobilnost - između 5 i 30 dana u inozemstvu.

2.2.1. Izazovi i strahovi

Erasmus+ predstavlja uzbudljivo iskustvo koje je istovremeno veliki skok u nepoznato. Takvo iskustvo zahtijeva da osoba ostavi sve prethodno iza sebe i da se suoči s potpuno novim stvarima s kojima se prethodno nije imala prilike susresti. Na „Blogu Erasmus generacije“ navodi se nekoliko strahova koje Erasmus+ studenti mogu imati [6]:

- **Odlazak iz doma** - ako je napravljen popis za i protiv prije nego što se je predala prijava za Erasmus+, onda je odlazak iz svog doma bio poprilično visoko na lošijoj strani popisa. Taj loš osjećaj traje neko vrijeme ali nakon par provedenih dana kreće upuštanje u avanturu „ograničenu vremenom“. U nekim trenucima zna se pojaviti osjećaj kada nedostaje sve ono od čega se je otišlo (to je u situacijama kada je osoba sama) ili kada se razgovara s bližnjima (dobiva se osjećaj da se propušta nešto dok je osoba na drugom mjestu). Biti udaljen od osoba s kojima je nekada dijeljeno sve predstavlja značajan izazov, no po povratku kući postaje jasno tko je prošao test "udaljenosti" i tko su istinski prijatelji.
- **Financijska (ne)odgovornost** - pri početku Erasmus+ dobiva se veći dio stipendije koji je namijenjen za život tijekom cijelog Erasmus+. Bitno je da osoba ima vještinu upravljanja velikom količinom novca i da racionalno troši novce na potrebne stvari. Ovdje je posebno izazovno jer se radi o trošenju novca u stranoj zemlji pa se onda obično u početku potroše veće količine novca (npr. radi nepredvidivih troškova). Najbolja je stvar na početku donesti odluku što se želi raditi s novcem. Jesu li to putovanja, uživanje u hrani, tjedna kupovina, izlasci itd... Zanimljiva činjenica je da obično postoji želja za kombiniranjem/isprobavanjem svega.
- **Jezična barijera** - nedostatak povjerenja u znanje stranih jezika predstavlja jedan od glavnih razloga zašto studenti ne sudjeluju u programima kao što je Erasmus+. Pretjerana briga oko toga nema smisla jer Erasmus+ zahtijeva osnovno znanje stranog jezika i nije problem ako netko nije dobar u tome. Erasmus+ je upravo dobra prilika da se razbije ta barijera i nauči novi jezik.
- **Akademski život** - postoje studenti koji nikada nisu imali nastavu ili prezentiranje materijala na stranom jeziku. Bez obzira na to koliko osoba dobro zna jezik, učenje o svojoj budućoj profesiji na stranom jeziku može biti nezgodno. Upisuju se kolegiji o kojima se vrlo vjerojatno malo ili ništa ne zna, gdje predaju profesori koji nikada prije nisu viđeni. Bez obzira koliko god je to izazovno, iskušavanje drugačijeg načina podučavanja u kompletno drugačijem sustavnom obrazovanju je neprocjenjivo iskustvo.
- **Pakiranje** - je vrlo bitan faktor koji se općenito veže za svaku vrstu putovanja (trodnevni izlet, dvotjedni odmor) a u pravilu znači pakiranje cijelog života u jedan kofer. Pakiranjem se zapravo dobiva svijest o tome koliko stvari osoba posjeduje i koristi na dnevnoj bazi.

U nastavku su prikazani primjeri prijašnjih Erasmus+ studenta koji pričaju konkretno o prvom izazovu koji je naveden na prošloj stranici (Odlazak iz doma) [7]:

Ixan Curtamet – Rumunjska

Tijekom Erasmus+ imao je osjećaj da mu nedostaje dom. Osjećao je da propušta puno toga u svojoj državi, falili su mu prijatelji. Imao je periode kada baš nije uživao na Erasmusu+ jer se osjećao loše zbog toga što je bio tužan.

Katarina Bižanović – Hrvatska

U početku joj nije nedostajao dom. Pogodilo ju je nakon 3 ili 4 mjeseca kada su joj prijatelji počeli nedostajati, a to je obično bilo za vrijeme praznika, jer je tada bila sama u stranoj državi.

Agnese Smirolfo – Italija

Nedostajao joj je dom jer su joj nedostajale određene kulturološke stvari poput hrane. Falio joj je i njezin tata.

Marc Dolcet Sadurni – Estonija

U početku mu je falio dom, ali uglavnom se puno toga događalo u isto vrijeme i nije imao vremena razmišljati o tome. Uživao je i upoznao puno ljudi.

Vitor Bizarro – Portugal

Misli da je nemoguće da ti fali dom kad si na Erasmusu+. Kuhao je kao kod kuće, družio se kao kod kuće i stalno je zvao prijatelje iz njegove države i pritom uživao.

Minerva Martinez – Italija

Nije joj nedostajao dom jer su Italija i Španjolska jako slične. Ipak, nakon tri mjeseca znala je ponekad osjetiti neki loš osjećaj.

Agnes Mfourmou – Belgija

Ona naglašava da joj dom nije nedostajao jer je stalno bila u kontaktu s prijateljima – dopisivala se, razgovarala s majkom te im slala slike i videozapise. Ističe važnost potpunog uživanja u Erasmus+ iskustvu, jer kasnije može nastupiti žaljenje zbog odlaska i javiti se nostalgija za Erasmus+ domom. Takav osjećaj, prema njezinu iskustvu, proizlazi iz činjenice da se osoba u određenom trenutku ne nalazi na nekom mjestu.

Kroz sljedeće poglavlje predstavljeno je koje su trenutno aktualne tehnologije i alati koji pomažu u današnjem informiranju i snalaženju među velikim brojem informacija.

3. Aktualne tehnologije i alati za uspješno snalaženje i informiranje

Unutar ovog poglavlja napravljen je uvod u inteligentne pomoćne sustave, a potom su opisane njihove ključne značajke. Slijedi podjela na asistente, kopilote, autonomne agente i chatbot platforme gdje je svaki tip ukratko opisan i potkrijepljen reprezentativnim primjerom. Za kraj, ovog poglavlja opisano je korištenje inteligentnih asistenata u sustavu obrazovanja, njihova primjena i koji su izazovi takvog korištenja.

3.1. Uvod u inteligentne sustave

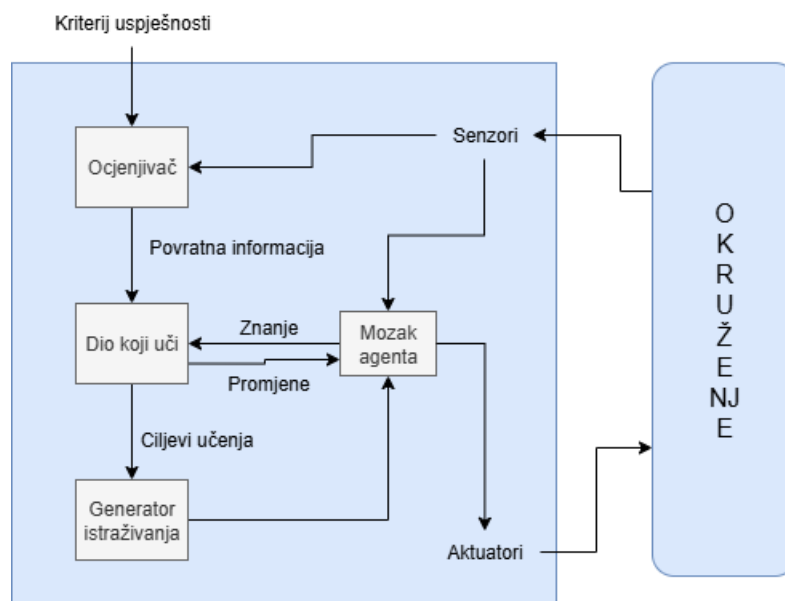
"Davne 1961. godine IBM je predstavio prvi sustav koji je mogao prepoznati govor (izgovorene znamenke)" [8]. Do 1990-ih kreirani su prvi komercijalni osobni asistenti koji su se aktivirali na postojanje glasa. Godine 2010. svijet je prvi put čuo za Siri [8], Appleovog osobnog asistenta koji je pokrenut na temelju umjetne inteligencije. U to vrijeme Siri je predstavljao revolucionarno rješenje. Petnaest godina kasnije, korisnici su zatrpani ogromnom količinom inteligentnih asistenata. Korisničko iskustvo (eng. User experience) je sada više nego ikada važno. Održavanje zadovoljstva korisnika/kupca i upravljanje njihovim očekivanjima zahtijeva stalne inovacije i poboljšanja. Korisnici žele da web-stranice budu intuitivne i učinkovite te da na neki način zadovolje njihove potrebe. AI asistenti su postali popularni alati koji pomažu u poboljšanju korisničkog iskustva. Točnije rečeno, dodaju novi način interakcije između korisnika i pružatelja usluge i samim time povećavaju angažman, zadovoljstvo ili profit [8].

Umjetna inteligencija je grana koja se u zadnje vrijeme usredotočuje na sve vrste inteligentnih agenata koji predstavljaju nešto što percipira svoju okolinu putem senzora i djeluje na nju putem aktuatora. Postoji razlika između apstraktne agentske funkcije (mapiranje iz potpunog perceptivnog slijeda u radnju) i konkretnog agentskog programa koji se izvodi na nekoj fizičkoj arhitekturi (npr. robot) [[9], pp. 55–56]. Ponašanje se temelji na racionalnosti, gdje u svakom trenutku racionalan agent bira akciju za koju se očekuje da maksimizira njegovu mjeru performansi, s obzirom na: prethodno znanje, raspoložive akcije i dosadašnji niz percepata [[9], pp. 58–59]. Racionalnost nije sveznanje jer agenti djeluju pod nesigurnošću i ograničenim informacijama. To se ilustrira "svijetom usisavača" (eng. vacuum world) i pokazuje kako različite mjere performansi mijenjaju što se smatra racionalnim. Time se nadalje odvaja autonomija od perfekcije, odnosno što se agent više oslanja na vlastite perceptive (sinonim za opažanje) i učenje, a manje na prethodno znanje dizajnera, to je autonomniji. Potpuna autonomija od početka nije nužna, ali bi racionalan agent s vremenom trebao učiti kako bi kompenzirao netočne pretpostavke [[9], p. 60]. Također, autor predstavlja specifikaciju okruženja zadatka prema PEAS shemi (Performance measure, Environment, Actuators, Sensors). Klasičan je primjer autonomnog taksija gdje performanse uključuju sigurnost, zakonitost, udobnost, učinkovitost i profit. Okolina sadrži ceste, promet, pješake, vrijeme i korisnike. Aktuatori uključuju upravljanje, gas i kočnice. Senzori uključuju kamere, GPS i dr. Postoji više tipova okolina koje usmjeravaju dizajn agenta: potpuno / djelomično saglediva, jednoagentna / višeagentna, deterministička /

nedeterministička, epizodična / sekvencijalna, statična / dinamična, diskretna / kontinuirana, poznata / nepoznata. Vožnja je, primjerice, djelomično saglediva, višeagentna, nedeterministička, sekvencijalna, dinamična i uglavnom kontinuirana [[9], pp. 60–65].

U literaturi [9] navode se četiri kanonska “kostura” programa agenta, uz opći omotač za učenje:

- **Jednostavni refleksni agenti** — biraju akcije isključivo iz trenutnog percepta pomoću pravila (npr. ako auto ispred koči → koči). Brzi su, ali krhki u djelomično sagledivim okolinama [pp. 68–69].
- **Refleksni agenti utemeljeni na modelu** — održavaju interno stanje ažurirano modelom prijelaza (“kako se svijet razvija” i učinci vlastitih akcija) i senzorskim modelom (“kako stanja generiraju percepte”), što omogućuje razumno ponašanje pod djelomičnom sagledivošću [pp. 70–71].
- **Ciljno vođeni agenti** — dodaju eksplicitne ciljeve i koriste pretraživanje i planiranje za izbor akcija koje ih postižu. Manje su učinkoviti, ali znatno fleksibilniji [pp. 71–72].
- **Agenti temeljeni na korisnosti** — obrađuju ciljeve usporedbom koliko su ishodi dobri te, pod nesigurnošću, djeluju tako da maksimiziraju očekivanu korisnost. To daje opći okvir odlučivanja, ali traži snažno modeliranje i optimizaciju [pp. 72–74].
- **Agenti koji uče** — omataju bilo koji od navedenih agenata s izvedbenim elementom (djeluje), kritičarem (evaluira), učećim elementom (poboljšava izvedbeni element) i generatorom problema (potječe istraživanje). Time raste autonomija, a prilagodljivost se poboljšava kroz vrijeme [pp. 74–76].



Slika 1: Opći učećći agent - objašnjenje [[9], p. 74]

Prethodna Slika 1. prikazuje općeg učećeg agenta: okolina je desno, a agent preko senzora prima opažaje i preko aktuatora šalje radnje; mozak agenta (eng. performance element) trenutačno bira akcije. Ocjenjivač uspoređuje ponašanje s kriterijem uspješnosti i vraća povratnu informaciju. Učeći dio koristi povratnu informaciju i znanje o trenutačnom radu te upisuje promjene u mozak agenta kako bi ga poboljšao. Generator istraživanja prema ciljevima učenja potiče istraživačke radnje kako bi agent prikupio korisna nova iskustva.

Na kraju krajeva, i savršeno “racionalno” ponašanje može promašiti bit ako je cilj pogrešno specificiran. Naivne mjere performansi (npr. “maksimiziraj skupljanje prljavštine”) mogu se iskoristiti, a dizajneri mogu biti nesigurni oko stvarnih preferencija korisnika (sustavi bi ponekad trebali modelirati nesigurnost u samu mjeru performansi i učiti je kroz vrijeme). Paralelno, racionalni agenti trebaju postajati sve autonomniji učenjem iz iskustva, smanjujući oslanjanje na ugrađeno znanje [[9], pp. 57–58].

Ne postoji jedna jedinstvena, općeprihvaćena definicija za inteligentnog agenta, ali većina istraživača slaže se oko jedne stvari: autonomije. Agent je računalni sustav smješten u nekom okruženju koji može samostalno djelovati kako bi postigao zadane ciljeve. U Wooldridgeovoj formulaciji, okruženje daje perceptivne ulaze preko senzora, gdje agent odabire radnje kroz interni proces odlučivanja, dok efektori provode te radnje natrag u okruženje. Presudno je da ista radnja ne mora uvijek imati isti učinak, pa se agenti moraju pripremiti na neuspjeh i neizvjesnost [[10], p. 40].

Praktičan način za prepoznavanje agenata je kroz tri komplementarne sposobnosti: reaktivnost (osjećanje okoline i pravovremeni odgovor), proaktivnost (inicijativno djelovanje prema ciljevima) i socijalnu sposobnost (interakcija s drugim agentima i često ljudima). Agenti se suočavaju i s dugoročnim posljedicama, kao što je npr. bezazlen izbor raspoređivanja (sada odabrati p1, a p2 je obećan za kasnije), koji, ako se ponavlja, može dovesti do nepravde gdje p2 nikada ne dođe na red [[10], p. 45].

Autonomija je spektar. Na jednom su kraju ljudi, a na drugom jednostavne usluge koje djeluju samo kad ih se pozove. Agenti koji su zanimljivi nalaze se u sredini i njima se mogu delegirati ciljevi na visokoj razini jer oni odlučuju kako ih ostvariti. Autonomija se često podešava dinamički. Uz podešivu autonomiju agent prepušta kontrolu čovjeku kada je vjerojatno da će čovjek odlučiti bolje, kada je neizvjesnost visoka, potencijalna šteta značajna ili agent nema sposobnost odlučiti. Umjetnost je izbjeći krajnosti (agent koji stalno pita je beskoristan, a onaj koji nikad ne pita može biti rizičan) [[10], pp. 41–42].

Agenti se često uspoređuju s objektima u objektno-orijentiranom programiranju. Oba slučaja enkapsuliraju stanje i komuniciraju porukama, ali postoji nekoliko razlika [[10], pp. 47–48]:

- Žarište kontrole — Objekt izlaže metode koje drugi mogu pozvati kada požele. Agent može odbiti traženu radnju ako je u sukobu s njegovim ciljevima ili politikom. Objekti to čine kad im se kaže, dok agenti to čine jer oni to žele.
- Fleksibilna autonomija — Osnovni objektni model ne govori kako integrirati reaktivnost, proaktivnost i socijalno ponašanje, dok je to u središtu agentnog pogleda.

- Tokovi kontrole — Višeagentni sustavi su inherentno višedretveni; svaki agent kontrolira vlastitu dretvu, dok klasični objektni modeli obično pretpostavljaju jednu dretvu.

U srcu dizajna agenta je odabir radnje, gdje se odlučuje što učiniti s obzirom na ono što agent trenutačno vjeruje o okruženju i ciljevima. Takvim razmišljanjem zapravo se razmišlja o arhitekturi agenata, gdje se definiraju moduli za osjet, ažuriranje stanja, odlučivanje, jednostavni regulatori povratne sprege itd. Termostat je agent u minimalnom smislu jer osjeća temperaturu i prebacuje grijanje prema jednostavnoj politici. Kako rastu sposobnosti, agenti moraju podnositi djelomičnu vidljivost, promjenjive ciljeve, složene kompromisne odluke i neprekidni rad [[10], p. 39 i 45].

Prvi od načina zadavanja zadatka je na temelju cilja, gdje se nastoje ostvariti ili održati uvjeti. Drugi od načina je na temelju korisnosti, gdje postoje funkcije nad stanjima ili putanjama koje ocjenjuju poželjnost, a agent maksimizira očekivanu korisnost. U slučaju da je okruženje i djelomično opažljivo, načelan je okvir POMDP (eng. Partially Observable Markov Decision Process), kod kojeg rješenje daje politiku (što činiti s obzirom na trenutačne informacije, kako maksimizirati očekivani povrat pod neizvjesnim informacijama) [[10], pp. 56 i 57 i 64].

Kroz sljedeću sekciju prikazani su različiti tipovi inteligentnih asistenata s obzirom na njihovu primjenu i način korištenja. Za svaki tip naveden je tipičan scenarij i reprezentativan primjer.

3.2. Tipovi rješenja i kriteriji razlikovanja

U svijetu umjetne inteligencije često se pojavljuju pojmovi poput asistenta, kopilota, autonomnih agenata i chatbot platformi. Iako na prvi pogled djeluju kao sinonimi, svaki od njih ima svoju specifičnu svrhu, primjenu i skup prednosti.

Tip	Ključne značajke	Primjena	Primjer
Asistent	Personalizacija i multitasking	Dnevna produktivnost	Siri
Kopilot	Specijalizirana podrška	Razvoj, IT	GitHub Copilot
Autonomni agent	Autonomija i proaktivnost	Upravljanje prijavama, korisničko iskustvo	ChatGPT Operator
Chatbot platforma	Jednostavni ponavljajući zadaci	Osnovna korisnička podrška	Intercom

Tablica 1: Razlika između pojmova [11]

U okviru praktičnog rada napravljen je asistent koji omogućuje generiranje personaliziranih odgovora i pristupa specifičnim dokumentima čime pomaže korisnicima u pronalasku pravovaljanih i točnih informacija.

3.2.1. Asistenti - Siri

AI asistenti donose inteligenciju automatizaciji razumijevanjem konteksta i time se personalizira interakcija kako bi se mogao obaviti niz zadataka. Ovi sustavi nadilaze skriptirane odgovore koristeći tehnologije poput obrade prirodnog jezika (eng. Natural processing language) ili strojnog učenja (eng. Machine learning). Kontekstualno su svjesni i razumiju korisnički unos što omogućava vođenje višestrukih razgovora. Integriraju se s alatima poput kalendara, aplikacijama za razmjenu poruku i poslovnim sustavima. Još jedna od značajki AI asistenata jest njihova sposobnost prilagodbe ponašanju korisnika, čime se s vremenom poboljšava njihova učinkovitost. Npr. mogu se koristiti za zakazivanje sastanaka, upravljanje e-poštom ili prilagođavanje rasvjete i temperature u pametnim domovima [11].

U nastavku su navedene prednosti inteligentnih asistenata [8]:

- **Brzina i jednostavnost** - AI asistenti omogućuju korisnicima da brzo i jednostavno dobiju ono što im je potrebno, smanjujući gubitak vremena. Sličan princip koristi se u sustavima preporuke, poput onog na TikToku, gdje mehanizam na temelju prikupljenih podataka i naučenih obrazaca predviđa što bi korisnik želio gledati. Iako TikTok nije AI asistent u užem smislu, njegov primjer pokazuje kako inteligencija može optimizirati korisničko iskustvo.
- **Pružanje podrške 24/7** - AI asistenti su dostupni stalno, čime je omogućena podrška korisnicima kad god je to potrebno,
- **Prihvatanje od strane ljudi** - razgovor s AI asistentima se odvija kroz korisnička sučelja, gdje korisnici mogu komunicirati na ljudski razumljiv način s asistentima.

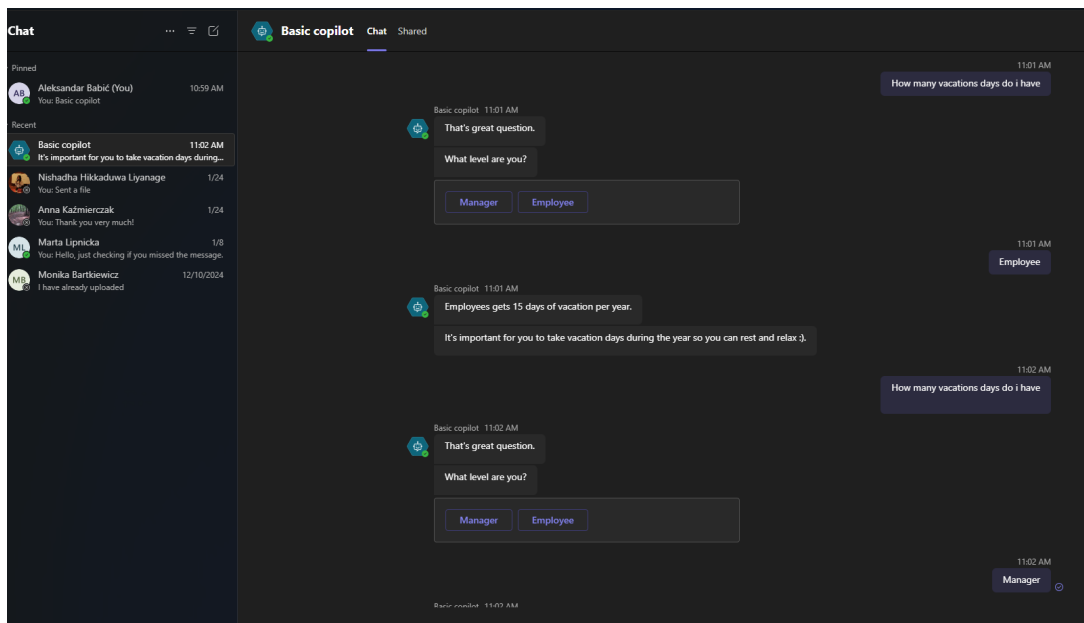
Siri je Appleov virtualni asistent za iOS, macOS, tvOS, i watchOS uređaje koji se koristi preko glasa i pokreće ga umjetna inteligencija [12]. Siri odgovara na izgovorena pitanja korisnika tako što im odgovara putem zvučnika uređaja i prikazuje paralelno relevantne informacije na početnom zaslonu iz određenih aplikacija kao što je Kalendar. Siri se počeo izrađivati u neprofitnom istraživačkom institutu sa sjedištem u Kaliforniji nakon čega su pojedini članovi odlučili osnovati startup tvrtku pod nazivom Siri. Godine 2010. Apple je preuzeo Siri što je dovelo do brze integracije u iOS uređaje. Tijekom 2011. i 2012. godine Siri je bio na Iphone 4S i na iPad-ovima treće generacije. U lipnju 2024. Apple je na konferenciji WWDC 2024 najavio značajnu reviziju Siriya gdje je predstavljen Apple Intelligence koji predstavlja novi osobni AI sustav koji poboljšava Sirijeve mogućnosti. U početku glas Siriya je bio od umjetnice Susan Bennet. Zanimljiv slučaj je bio da sve dok joj nije prijateljica javila, Bennet nije bila svjesna da je postala Sirin glas. Iako Apple nikada nije priznao da je Bennet originalni Sirin glas, audio stručnjaci na CNN-u su to potvrdili. Karen Jacobsen je žena koja je posudila svoj originalni ženski glas za GPS sustave, dok je Jon Biggs, posudio svoj glas za mušku verziju Siriya. U međuvremenu Apple je razvio novi ženski glas za iOS11 pomoću tehnologije dubokog učenja snimanjem i kombiniranjem govora stotine kandidatkinja. Siri može obavljati razne zadatke kao što su: navigacija uputama (npr. kakav je promet na putu do posla?), zakazivanje događaja i podsjetnika (npr. slanje poruke za rođendan), pretraživanje weba (npr. pronađi sliku psa) ili mijenjanje postavki (npr. povećanje svjetline zaslona) [12].

3.2.2. Kopiloti - Microsoft Copilot Studio

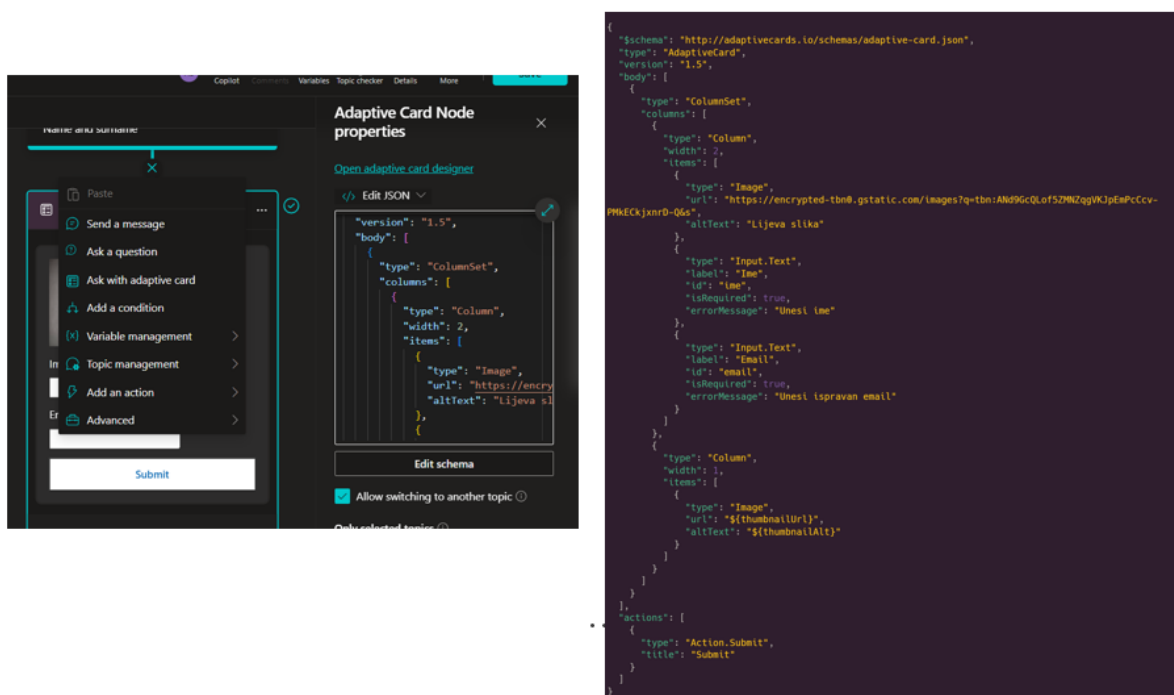
Kopilot je AI alat koji je u ulozi suradnika i primarno je osmišljen za poboljšanje ljudskih sposobnosti u složenijim zadacima. Za razliku od asistenata opće namjene, kopiloti se usredotočuju na specifične domene, nudeći stručnost i proaktivnu podršku. Duboko su ugrađeni u specifične platforme čime pružaju podršku. Predviđaju korisničke potrebe analiziranjem konteksta i nude relevantne prijedloge ili rješenja. Na primjer, Github Copilot se koristi u razvoju softvera gdje predlaže isječke koda, otkriva pogreške i ubrzava razvoj automatizacijom rutinskih zadataka [11].

Tijekom studentske prakse u jednoj varaždinskoj firmi imao sam priliku raditi s alatom Microsoft Copilot Studio. Rukovodstvo te tvrtke u tom trenutku je imalo mišljenje da korištenjem AI alata se znatno povećava produktivnost zaposlenika. Primarno, bio je zadatak napraviti razna istraživanja u AI domeni jer su svi popularniji AI sustavi primjeri nekog oblika općenitog znanja. Ni jedan od njih ne zna ništa o poslovanju i domeni kojim se firma bavi. Unutar Copilot Studija mogu se razvijati kopiloti u područjima kao što su: prodaja i podrška, informacije (podaci o otvaranju trgovina), zdravlje zaposlenika i godišnji odmori. Također, unutar sustava postoji nekolicina kreiranih predložaka s kopilotima koji odgovaraju na karakteristična pitanja poput IT Helpdeska ili Team Navigatora. Primarno, taj sustav je razvijen od strane Microsofta i koristi se za izradu kopilota koji mogu odgovarati na specifična pitanja koja su usko vezana za neku domenu. U alatu se isprobalo korištenje varijabli, entiteta, petlji, tijeka rada (eng. Workflow) ali ono što je bilo najzanimljivije je baza znanja (eng. Knowledge base) gdje se može spremati velika količina podataka koja je baza za odgovore. Osim toga, kada se kreira agent, on se može ugraditi u Microsoft Teams ili u web-stranicu. Microsoft Copilot studio se bazira na kreiranju tema (eng. Topic) i dok god ima definirane teme za određene slučajeve od tamo će crpiti inspiraciju za odgovore. Postoje prilagođene (eng. Custom) i sistemske (eng. System) teme. Prilagođene teme se koriste za komuniciranje s korisnikom (Dobar dan, doviđenja) dok se sistemske koriste za pozadinske operacije (inicijalna poruka pri početku konverzacije ili završetak razgovora). Kada kopilot nema definiranu temu za odgovor na neko pitanje onda odlazi u sistemske teme i tamo otvara conversational boosting temu gdje je to zapravo tijekom rada koji traži informacije iz baze znanja [13]. Kao i svaki sustav, Copilot Studio ima svoje limitacije a to su: max 8000 zahtjeva po minuti u Dataverse okruženju, max datoteka veličine 512 MB, max 500 datoteka, max 200 fraza po temi i max 50 kopilota po timu [14].

Osim toga, isproban je i Bing Custom Search koji je vanjski servis koji se može koristiti za navođenje više izvora koji se koriste za odgovaranje na upite. Cilj je da se ne koristi cijeli internet nego da se koristi taj skup podataka koji je naveden. Putem "custom configuration ID-a" može se povezati Custom Bing Search i Copilot Studio. Naknadno isprobana je funkcionalnost Adaptive Cards Designer koja omogućava kreiranje vlastitih UI (eng. User Interface) elemenata koji se mogu koristiti unutar razgovora s korisnikom. Slika 3. prikazuje da je to JSON koji je potrebno uvesti u Copilot Studio. Na stranici <https://adaptivecards.io/designer/> postoji veliki broj primjera koji su već kreirani i besplatni su za korištenje [13].



Slika 2: Primjer kopilota kreiranog u Microsoft Copilot Studiu



Slika 3: Adaptive cards - primjer

Od drugih stvari koje još postoje u Copilot Studiju:

- Dvije vrste kreiranja entiteta (Copilot sadrži veliki broj entiteta koji se mogu koristiti, ali se mogu kreirati i vlastiti (Closed List i Regular Expression)),
- Message variation koji omogućava postojanje nekoliko verzija incijalnih poruka koje se pojavljuju kada se kopilot pokrene.

3.2.3. Autonomni agenti - ChatGPT Operator

Autonomni agenti predstavljaju vrhunac automatizacije i autonomije jer su sposobni samostalno donositi odluke i poduzimati radnje bez stalnog ljudskog vodstva. Dizajnirani su za složena okruženja a također se dinamički prilagođavaju kako bi postigli definirane ciljeve. Djeluju samostalno kako bi analizirali podatke i odredili prioritetne zadatke. Prate sustave i okruženja, predviđaju i rješavaju potencijalne probleme prije nego što eskaliraju. Pri radu sa složenijim problemima, identificiraju optimalnija rješenja. Bankarski agenti zamrzavaju kompromitirane račune kada se otkrije neovlašteni pristup s visokorizične lokacije. Na primjer, prate stanje servera, otkrivaju anomalije, identificiraju kašnjenje pošiljka i proaktivno obavještavaju kupca nudeći naknadu ili alternative i poduzimaju korektivne mjere bez ljudske intervencije [11].

ChatGPT Operator je agent koji može ići na web kako bi obavljao zadatke umjesto korisnika. Može pregledavati web-stranicu i komunicirati s njom tipkanjem, klikanjem i pomicanjem. Pojavio se u siječnju 2025. godine i trenutno je još u fazi razvoja gdje se nastoji poboljšati rješenje na temelju povratnih informacija korisnika. Od Operatora se može zatražiti da obavi širok raspon repetitivnih zadataka u pregledniku, kao što je ispunjavanje formi, naručivanje proizvoda, pa čak i stvaranje memeova. Operator je dostupan unutar PRO verzije na području SAD-a i pokreće se na temelju modela CUA (eng. Computer-Using Agent). CUA je obučen za interakciju s grafičkim korisničkim sučeljima (eng. Graphical User Interface) - gumbima, izbornicima i tekstualnim poljima koje ljudi vide na zaslonu. Ako se dogodi da se Operator susretne s izazovima ili ako napravi pogreške, u tom slučaju vraća kontrolu korisniku, osiguravajući glatko i korisničko iskustvo [15].

U nastavku se navodi jedan primjer koji ChatGPT Operator obavlja [16]:

1. Upravlja preglednikom i obavlja radnje u ime korisnika.
2. Analizira podatke na temelju YouTube veze.
3. Generira PNG sliku analize.
4. Ide na X.com, lokalno odabire sliku i prenosi je.
5. Lajka i komentira sliku.
6. Često traži potvrdu prije izvođenja radnji.
7. Može se koristiti za pretraživanje weba, kupnju ulaznica ili rezervaciju restorana.

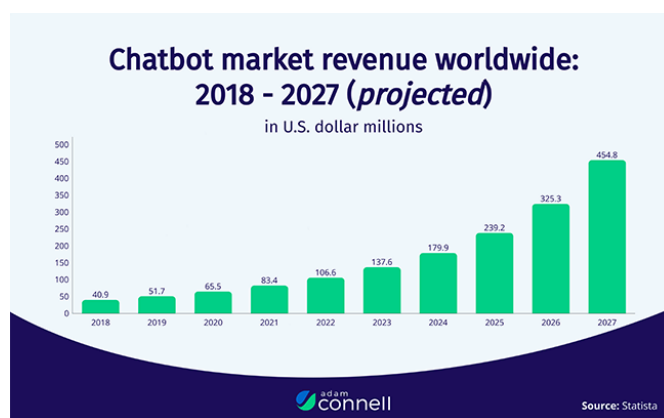
OpenAI spominje kako planiraju u budućnosti dati na korištenje model CUA preko API-ja čime će omogućiti izradu vlastiti agenata svakome. Osim toga, spominju kako planiraju proširiti Operator korisnicima Plusa, Teama i Enterprisea nakon što se uvjere u njegovu sigurnost i upotrebljivost u velikim razmjerima.

3.2.4. Chatbot platforme - Intercom

Chatbot platforme su sustavi temeljeni na kreiranim pravilima koje su osmišljene za izvršavanje određenih zadataka putem unaprijed definirane interakcije. Iako posjeduju dosta ograničenja jako se dobro i brzo snalaze u dosljednom rješavanju jednostavnih repetitivnih zadataka. Djeluju na temelju stabla odlučivanja ili prepoznavanja ključnih riječi ograničavajući svoju interakciju na unaprijed postavljene scenarije. Koriste se za automatizaciju zadataka gdje se obrađuje veliki broj ponavljajućih upita poput rezervacija ili narudžbi. Njihova mana je da se ne mogu prilagoditi izvan svog programiranja što ih čini neprikladnim za složenije zadatke. Uz ovakav pristup, sve češće se razvijaju i LLM-based chatbot platforme koje koriste modele dubokog učenja za generiranje fleksibilnijih i prirodnijih odgovora [11].

Intercom je platforma za korisničku podršku koja olakšava komunikaciju između kupaca i tvrtki. Poboljšava interakciju s kupcima putem brojnih komunikacijskih kanala koji uključuju chat uživo i automatizirano slanje poruka. Intercom je moguće integrirati s drugim alatima i platformama kao što su Salesforce, HubSpot, Zendesk i Slack.

Chatbot platforme su posebno zanimljive jer su to inteligentni sustavi koji možda najviše rade s kupcima i samim time mogu dosta utjecati na profit i vrijednost tvrtke. Na sljedećoj slici može se vidjeti kako se pretpostavlja da će globalno tržište chatbotova do kraja 2027. godine vrijediti 455 milijuna dolara. Ova procijenjena brojka za 2027. godinu (455 milijuna američkih dolara) izračunata je na temelju tržišne vrijednosti u 2018. godini [17].



Slika 4: Globalno tržište chatbotova; Izvor

Također, u sljedećem članku su doneseni sljedeći zaključci [17]:

- Tvrtke mogu smanjiti troškove korisničke podrške za 30 % korištenjem chatbot platformi,
- Trećinu korisnika chatbot platforme čine tzv. „napredni korisnici”,
- Najveći uočeni nedostatak kod chatbot platformi jest njihova nemogućnost odgovaranja na složena i personalizirana pitanja,
- Čak 55 % ljudi radije bi koristilo chatbot platformu nego čekalo dostupnost agenta uživo.

3.3. Inteligentni asistenti u sustavu obrazovanja

Današnje obrazovanje se sve više oslanja na tehnologiju, a inteligentni asistenti postaju alat koji može poboljšati iskustvo učenja i poučavanja. Na primjer, učenici mogu dobiti personaliziranu podršku, dok učitelji imaju više vremena za fokusiranje na ono što je zaista važno. Takvi asistenti mogu prilagođavati materijale, odgovarati na pitanja i ocjenjivati domaće zadatke čime se rasterećuje profesore. Umjesto univerzalnog pristupa, asistenti se mogu individualno posvetiti svakom učeniku/studentu.

3.3.1. Primjena inteligentnih asistenata u obrazovanju

Automatizacija procesa ocjenjivanja - ocjenjivanje pismenih zadataka, pružajući detaljne povratne informacije u roku od nekoliko minuta umjesto dana. Osim što skraćuje vrijeme ocjenjivanja profesorima, učenici dobivaju brže povratne informacije o svom radu. Jedan od primjera je Gradescope kojeg je razvio Turnitin. Kroz Gradescope moguće je ocjenjivati zadatke iz različitih predmeta, uključujući programiranje, fiziku, matematiku, kemiju, biologiju i ekonomiju. Također, moguće je ocjenjivati programske projekte (u digitalnom obliku) i zadatke napisane na papiru. Neki od komentara profesora koji su koristili Gradescope su [18]:

- "S Gradescopom je čast ocjenjivati. Prije što sam radio u rasponu od 2 do 3 sata sada odradim u 15 minuta" - Romulo Chumacero - University of Chile
- "Statistika mi stvarno pomaže da shvatim što sljedeći put mogu drugačije objasniti kako bih pomogla svojim učenicima da bolje uče" - Katie Johnson - Florida Gulf Coast University
- "Gradescope omogućuje da svaki dan svojoj grupi od 60 učenika dam kratki kviz i da ih sve ocijenim tijekom 30-minutne vožnje vlakom kući." - Jesse Tov - Northwestern University

Smanjenje administrativnog opterećenja - na primjer, sustavi pokretani umjetnom inteligencijom mogu pratiti prisutnost, upravljati zadacima pa čak i predviđati kojim studentima je potrebna dodatna pomoć. Takav pristup pomaže u otkrivanju problema prije nego što postane još veći problem. Jedan od zanimljivijih citata od Rose Luckin, profesorice dizajna usmjerenog na učenike na University College u Londonu, je: "Prava moć inteligentnih agenata u obrazovanju ne leži u zamjeni učitelja, već u pojačavanju njihovog utjecaja. Obavljanjem rutinskih zadataka, AI asistenti oslobađaju edukatore da se usredotoče na ljudske elemente poučavanja, inspiraciju, mentorstvo i njegovanje vještina kritičkog razmišljanja" [19].

Prilagođavanje nastavnih materijala - analiziranje snaga, slabosti i načina učenja svakog učenika kako bi se stvorili prilagođeni materijali. Na primjer, ako postoji učenik koji se muči s razlomcima, asistent može pružiti dodatne resurse kako bi se poboljšalo njegovo razumijevanje [19].

Question 2 (points total):
 Pipeline the circuit below. Optimize throughput (inputs processed per ns) and cost (\$). You may add any number of the blocks in the circuit (A-F), edge-triggered flipflops, edge-triggered interleavers, an edge-triggered de-interleavers (see figure below). No other modifications are permitted. The latency of flipflops, interleavers, and de-interleavers is 0ns. The cost of any item (whether already there or one you add), regardless of which item is \$1. You have a total budget of \$25 and the components already drawn below cost \$6. Draw your answer on top of the diagram below if at all possible.

Cost = \$6 + \$18 = \$24
 Latency = 7.5 x 8 = 60ns
 Throughput = 1/7.5ns
 (If we add a Slip at the output,)

Legend:
 Flipflop Interleaver Deinterleaver

Total Points
24.0 / 33.0 pts

1	+6.0	Correct interleaving concept
2	+6.0	Correct interleaving of multiple inputs
3	-3.0	Abused interleavers (too many outputs/inputs)
4	+6.0	Well-formed ("balanced" latch count)
5	+8.0	Correct extra stages for long-latency interleaved approach
6	-4.0	Not including latch for each input / extra latches if assuming interleavers latch

Slika 5: Ocjenjivanje zadatka - Gradescope [18]

3.3.2. Izazovi inteligentnih asistenata u obrazovanju

Jedan od najvećih izazova je pitanje privatnosti podataka. Obično takvi sustavi zahtijevaju ogromne količine podataka o studentima da bi učinkovito funkcionirali, što ukazuje na opasnost od potencijalne zlouporabe tih podataka. Na primjer, studija Sveučilišta u Michiganu iz 2020. godine otkrila je da 86 % obrazovnih aplikacija umjetne inteligencije prikuplja osobne podatke od studenta, a samo 42 % ima jasne politike privatnosti [19].

AI asistenti su dobri onoliko koliko su dobri podaci na kojima su obučeni. U obrazovnom sustavu, osiguranje kvalitete i relevantnosti tih podataka je od najveće važnosti. Kako bi rješenje bilo zadovoljavajuće škole i programeri moraju surađivati kako bi stvorili sustave koji su ne samo učinkoviti, već etični i sigurni [19].

Implementacija inteligentnih asistenata zahtjeva značajnu tehnološku infrastrukturu. Mnoge škole, posebno u ruralnim područjima, nemaju potreban hardver, brzi internet ili softver za podršku ovim sustavima. Izvješće za obrazovanje iz 2022. godine kaže da gotovo 17 milijuna učenika u SAD-u još uvijek nema pristup internetu kod kuće [19].

Unatoč tim izazovima, integracija inteligentnih asistenata u sustav obrazovanja donosi brojne prednosti. Sustavi pokretani umjetnom inteligencijom mogu automatizirati dugotrajne administrativne zadatke, oslobađajući učitelje da se usredotoče na ono što najbolje rade: inspiriranje i poučavanje učenika. Osim toga, AI asistenti mogu pružiti personalizirano učenje, povećanu produktivnost i učinkovitost, te poboljšano korisničko iskustvo [19].

Kroz sljedeće poglavlje je prikazano kako funkcioniraju inteligentni asistenti temeljeni na RAG metodi.

4. Retrieval Augmented Generation metoda

U ovom poglavlju obrađena je RAG (eng. Retrieval Augmented Generation) metoda, koja predstavlja suvremeni pristup u području obrade prirodnog jezika i kombinira sposobnosti generativnih modela s mehanizmima za dohvaćanje informacija iz vanjskih izvora. Poglavlje započinje pregledom povijesti a zatim slijedi teorijska osnova modela i njegova arhitektura kroz faze dohvaćanja i generiranja. Nadalje je napravljena kratka usporedba RAG-a i fine-tuninga, opisane su prednosti ove metode, predstavljeni najčešći slučajevi upotrebe, izazovi karakteristični za RAG pristup te je zaključeno pregledom dostupnih alata i biblioteka za implementaciju RAG sustava.

4.1. Uvod u RAG

RAG je proces optimizacije izlaza velikog jezičnog modela (eng. Large Language Models - LLM) pri čemu se LLM referencira na autoritativnu bazu znanja izvan izvora podataka za obuku prije generiranja odgovora. Veliki jezični modeli obučavaju se na ogromnim količinama podataka i koriste milijarde parametara za generiranje izlaza za zadatke poput odgovaranja pitanja, prevođenja jezika i dovršavanja rečenica. Ideja RAG-a je da proširi već moćne mogućnosti LLM-ova na određene specifične domene, a sve to bez potrebe za ponovnom obukom modela. Takav pristup poboljšava LLM-ove u smislu točnosti, korisnosti i relevantnosti [20].

Veliki jezični model može se zamisliti kao entuzijastičnog zaposlenika koji odbija biti informiran o aktualnim događajima, ali uvijek odgovara na pitanja s visokim samopouzdanjem. Nažalost, takav stav može negativno utjecati na povjerenje korisnika i nije nešto što bi trebalo postojati kod AI sustava. Iz takvih razloga RAG tehnologija preusmjerava LLM kako bi dohvatio relevantne informacije iz unaprijed definiranih izvora znanja [20].

Bez RAG-a, LLM uzima korisnički unos i stvara odgovor na temelju informacija na kojima je obučen. S RAG-om se uvodi komponenta za pronalaženje informacija koja koristi korisnički unos kako bi prvo privukla informacije iz novog izvora podataka. Nakon toga LLM kombinira novo znanje i svoje podatke za stvaranje boljih odgovora [20].

Primjerice, ako se postavi pitanje: "Koji planet u Sunčevom sustavu ima najviše satelita?", osoba može odgovoriti: "Odlično pitanje! Kao dijete sam volio astronomiju i pročitao sam članak o tome - mislim da je to Jupiter s 88 satelita." Međutim, takav odgovor ima nekoliko problema: osoba ne navodi konkretan izvor, iako nastupa s visokim samopouzdanjem ("ja to znam, pročitao sam") te informacija može bit zastarjela jer se temelji na sjećanju iz prošlosti [21].

Kada bi osoba rekla: "Idem prvo na NASA-inu stranicu i tamo ću potražiti tu informaciju", tada bi traženje odgovora bilo temeljeno na provjerenom i ažurnom izvoru. Nasuprot tome, LLM-ovi, poput GPT-a, često odgovaraju vrlo samouvjereno, primjerice: "Na temelju podataka na kojima sam treniran, odgovor je Jupiter." Iako zvuči uvjerljivo, takav odgovor može biti netočan, a korisnik često nije svjestan ograničenja modela ni datuma do kojeg je model tre-

niran. U ovom slučaju, točan odgovor zapravo može biti Saturn, koji trenutno ima 274 poznatih prirodnih satelita [22]. Rješenje za ovaj problem je korištenje spremišta znanja (eng. content store) koje može sadržavati izvore poput internetskih stranica, PDF dokumenta ili baza podataka. LLM tada prvo pristupa tom vanjskom spremištu informacija, pretražuje ga, identificira relevantne sadržaje i tek tada generira odgovor na temelju tih aktualnih podataka. U RAG sustavu prompt se najčešće sastoji od tri dijela: instrukcija (što model treba učiniti), korisnikovog upita (pitanje ili zahtjeva) i informacija koje su dohvaćene iz spremišta znanja [21].

Još jedan primjer je situacija u kojoj Alice želi saznati koliko dana porodiljnog dopusta može dobiti. AI sustav koji ne koristi RAG odgovara veselo i netočno "Uzmi koliko želiš". Politike o porodiljnom dopustu su složene i razlikuju se ovisno o državi u kojoj se boravi. U ovom slučaju, veliki jezični model (LLM), umjesto da prizna ograničenje u znanju odgovarajući "Žao mi je, ne znam", generirao je frazu iz svog skupa za treniranje koristeći korisnički orijentiran, ali neutemeljen jezik [23].

4.1.1. Povijest i razvoj

Korijeni RAG-a mogu se pratiti od 1950-ih i 1960-ih kada su se koristili sustavi za pronalazjenje informacija. Istraživači Hans Peter Luhn i Gerald Salton postavili su temelje za modele vektorskog prostora omogućujući računalima da pronađu dokumente koji su relevantni za korisničke upite. Obrada prirodnog jezika značajnije je napredovala u 1980-ima i 1990-ima putem statističkih jezičnih modela, poput n-gram modela [24].

Integracija pretraživanja i generiranja pojavila se u 2000-ima s razvojem sustava poput IBM Watsona [24], koji je koristio metodu pretraživanja za prikupljanje informacija i statističke modele za generiranje odgovora. Razvoj neuronskih mreža u 2010-ima dodatno je ubrzao ovu integraciju. Transformatori omogućuju značajan napredak u modeliranju jezika, dok tehnike gustog pretraživanja, poput pretraživanja temeljenog na ugrađivanju, dodatno poboljšavaju točnost pretraživanja [24].

RAG se pojavljuje u radu "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" 2020. godine [24], gdje su autori predložili sustav u kojem vanjski model dohvaća relevantne dokumente kao odgovor na upit, a generativni model uvjetovan tim dohvaćenim znanjem daje odgovor. U radu su autori koristili bi-encoder za pretraživanje, temeljen na pretraživanju gustih prolaza (eng. Dense Passage Retrieval) i T5 transformator kao generativni model, pokazujući izvanredne performanse na testovima koji zahtijevaju puno znanja. Ovaj pristup rješava ključne izazove s kojima se suočavaju modeli velikih jezika, poput halucinacija i zastarjelog znanja [24].

Objavljivanje RAG okvira potaknulo je nagli rast istraživanja, gdje je napravljeno nekoliko napredaka koja su se nadovezali na izvorni koncept [24]:

- **Poboljšani modeli pretraživanja** - modeli neuronskog pretraživanja poput ColBERTa postaju standard u RAG sustavima,
 - Kroz rad "ColBERT: Efficient and Effective Passage Search via Contextuali-

zed Late Interaction over BERT" predstavljen je inovativan pristup neuralnog retrievala koji koristi kontekstualnu interakciju između upita i dokumenta. Takav model omogućava detaljnije razumijevanje značenja riječi u kontekstu, što značajno poboljšava točnost pretrage. ColBERT kombinira efikasnost i preciznost, pružajući skalabilno rješenje za sofisticirane pretrage u velikim skupovima podataka [25].

- **Skalabilnost i učinkovitost** - kako bi se obradile ogromne količine podataka. Tehnike poput pretraživanja najbližeg susjeda ANN (eng. Approximate Nearest Neighbor) i destilacije modela pomogle su u optimizaciji performansi,

- u članku "Billion-scale similarity search with GPUs" prikazana je metoda ANN koja omogućuje brzu obradu i skaliranje pretraga na milijardama vektora. Korištenjem GPU ubrzanja, takva metoda značajno smanjuje vrijeme pretraživanja pri minimalnom gubitku točnosti. To je ključno za RAG sustave jer zahtijevaju efikasno dohvaćanje informacija u stvarnom vremenu [26].

- **Primjene specifične za domenu** - do 2023. godine RAG sustavi su se prilagođavali specifičnim slučajevima upotrebe, kao što su pravne i medicinske domene, gdje su točnost i objašnjivost ključne stavke. Dodatno se je istraživalo fino podešavanje kako bi se dobio što precizniji odgovor,

- u radu "Retrieval augmented generation for large language models in health-care" prikazana je primjena RAG modela prilagođenih specifičnoj domeni medicine. Fokus je na povećanju preciznosti i pouzdanosti odgovora korištenjem specijaliziranih medicinskih baza podataka i fine-tuning modela. Time je omogućeno generiranje točnijih i relevantnijih informacija, što je ključno u osjetljivoj domeni poput zdravstvene skrbi [27].

- **Multimodalni RAG** - proširivanje RAG-a kako bi uključio tekst, slike, zvuk i video. Proširivanje na područja poput obrazovanja i zabave,

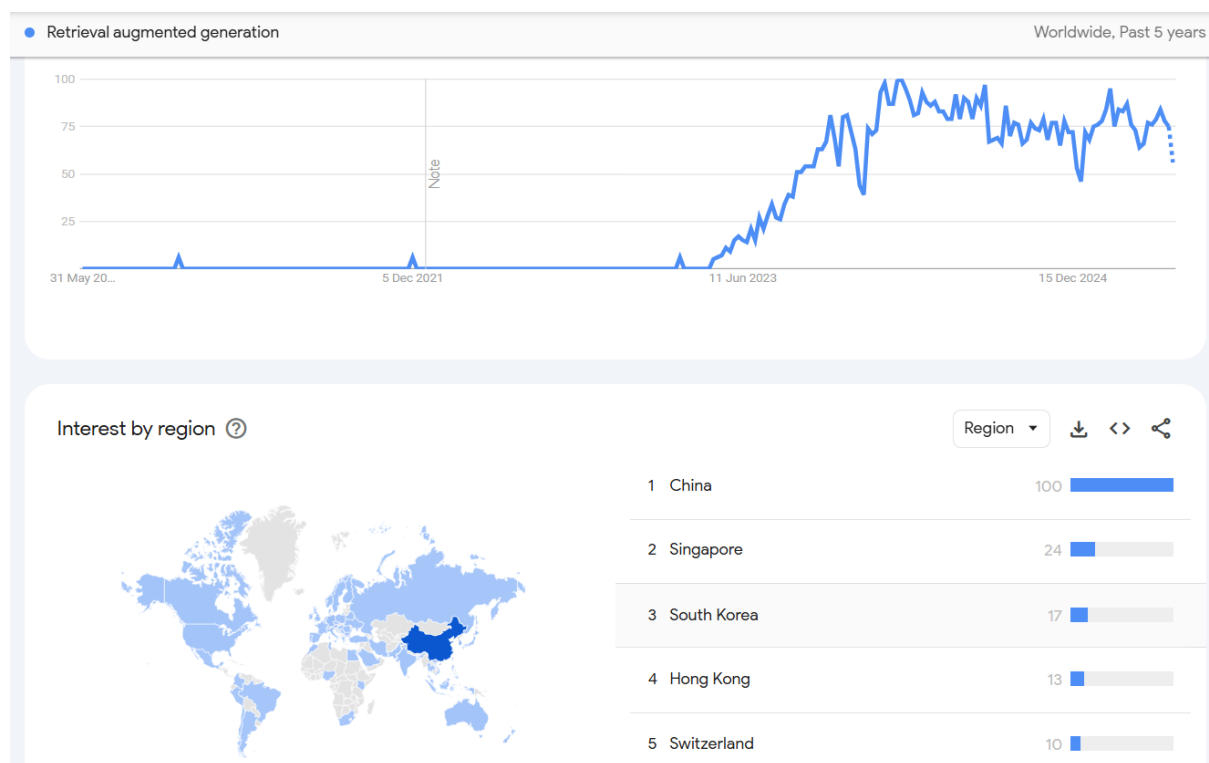
- U članku "Advanced Embedding Techniques in Multimodal Retrieval Augmented Generation" istražene su metode spajanja različitih modaliteta podataka, poput teksta i slika, u jedinstveni retrieval sustav. Predložene embedding tehnike omogućuju efikasno pretraživanje i povezivanje informacija kroz više vrsta podataka, čime se proširuju mogućnosti RAG aplikacija [28].

- **Prilagođeno učenje i petlje povratnih informacija** - prilagodba prema korisnicima je ostvarena kroz razumijevanje njihovih povratnih informacija u stvarnom vremenu.

- U studiji "Enhancing Retrieval-Augmented Generation: A Study of Best Practices" donose se nova saznanja o korištenju kontrastivnog učenja za poboljšanje kvalitete retrievala u RAG sustavima. Osim toga, uvode metode retrieval-augmented fine-tuninga kojima se sustavi mogu kontinuirano prilagođavati na temelju povratnih informacija korisnika što povećava relevantnost i točnost generiranih odgovora [29].

Kroz povijest RAG-a pokazuje se kako su se temeljna istraživanja u odvojenim područjima, poput pretraživanja i generiranja podataka, spojila kako bi se riješili izazovi u zadacima koji

zahtijevaju puno znanja. Također, može se vidjeti važnost interdisciplinarne suradnje u području umjetne inteligencije, gdje se otkrića često javljaju kombiniranjem različitih metodologija [24].



Slika 6: Popularnost RAG-a; Izvor

Važnost RAG-a povećala se posljednjih godina zbog pojave velikog broja aplikacija temeljenih na modelima umjetne inteligencije. Na slici 6. se može vidjeti kako uzlazna putanja popularnosti RAG-a kreće od svibnja 2023. godine. U sljedećoj sekciji je pojašnjena arhitektura RAG-a, uz prikaz njezinih ključnih elemenata i načina na koji oni međusobno surađuju.

4.2. Arhitektura RAG modela

U grubo objašnjeno RAG se sastoji od faze pronalaženja i faze generiranja sadržaja [21]. U fazi pronalaženja algoritmi traže relevantne informacije za korisnikov upit dok se u fazi generiranja generira tekst kako bi se odgovorilo na korisnički upit. U okruženju otvorene domene te činjenice uglavnom dolaze iz indeksiranih dokumenata na internetu. U okruženju zatvorene domene obično se koristi uži skup izvora radi dodatne sigurnosti i pouzdanosti. Prije LLM-a, agenti su kroz razgovor pratili ručni tijek dijaloga. Pretpostavljali su namjeru klijenta, dohvaćali tražene informacije i dostavljali odgovor za određeni scenarij. Za jednostavne upite, ova ručna metoda stabla odlučivanja funkcionirala je sasvim dobro. Problem takvog pristupa je da se je gubilo vrijeme na pisanje i predviđanje svakog pitanja koje bi korisnik mogao postaviti. U takvim slučajevima, ako tražena informacija nije definirana u dostupnim podacima, AI sustav ne može pružiti relevantan odgovor [23].

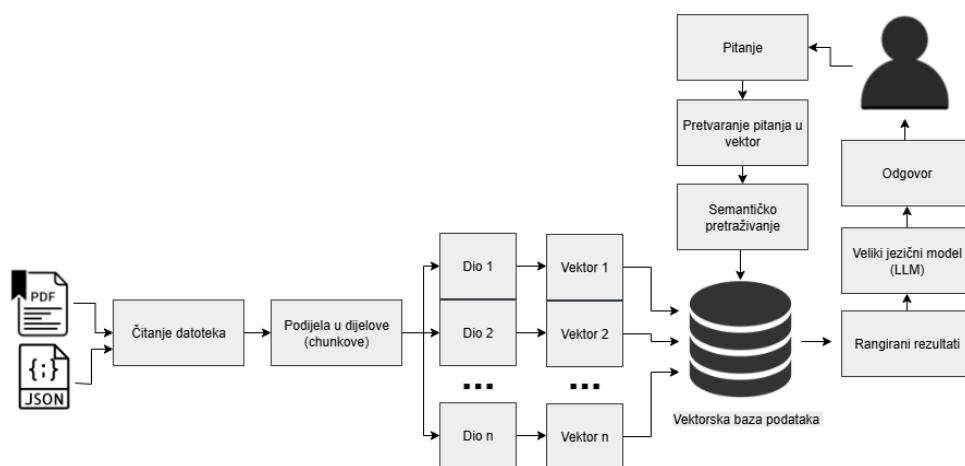
Generalno RAG arhitektura se može objasniti kroz sljedeće karakteristike [20]:

1. **Stvaranje vanjskih podataka** - Novi podaci izvan originalnog skupa podataka koji su se koristili za treniranje LLM-a se zovu vanjski podaci (eng. external data). Oni dolaze iz više izvora podataka kao što su: API-ji ili baze podataka. Podaci mogu postojati u različitim formatima. Jedna od tehnika (eng. embedding language models) pretvara podatke u numerički oblik i sprema ih u vektorsku bazu podataka (kako bi generativni modeli umjetne inteligencije to mogli razumjeti).
2. **Preuzimanje relevantnih informacija** - Korisnički upit pretvara se u vektorski prikaz i uspoređuje se s vektorskim bazama podataka. Na primjer, ako korisnik postavi pitanje chatbotu "Koliko dana godišnjeg odmora imam?", AI sustav će morati preuzeti dokumente o politici godišnjeg odmora uz evidenciju prošlih odmora za pojedinog zaposlenika. Takvi dokumenti će biti vraćeni korisniku jer su relevantni za njegov upit.
3. **Proširivanje LLM upita** - Zatim, RAG model proširuje korisnički upit dodavanjem relevantnih dohvaćenih podataka u kontekstu. Prošireni upit omogućuje velikim jezičnim modelima generiranje točnijih odgovora.
4. **Ažuriranje vanjskih podataka** - Kako bi odgovori AI sustava ostali točni i relevantni, važno je redovito ažurirati vanjske podatke koji se koriste u radu modela. To se može provoditi automatski ili ručno, primjerice periodičnim dohvaćanjem informacija iz API-ja ili brisanjem zastarjelih podataka. Na taj način sustav ostaje usklađen s najnovijim informacijama i smanjuje rizik pružanja netočnih ili zastarjelih odgovora.

Današnji AI sustavi bazirani na LLM-ovima mogu dati korisnicima odgovore bez da ljudi pišu definirane skripte. Samim nadovezivanjem RAG-a na LLM-ove smanjuje se potreba za ponovnim obučavanjem modela na novim primjerima [23]. Upiti korisnika nisu uvijek jednostavni, mogu biti dvosmisleno formulirani, složeni ili mogu zahtijevati znanje koje model nema.

U takvim uvjetima LLM-ovi su skloni izmišljanju stvari. Rod Lastra opisuje velike jezični model kao “Zamislite model kao pretjerano nestrpljivog mlađeg zaposlenika koji izbrblja odgovor prije nego što provjeri činjenice” [23]. Lastra dodatno ističe važnost svjesnog upravljanja nezvjesnošću u odgovaranju, navodeći: “Iskustvo nas uči da zastanemo i kažemo kada nešto ne znamo. LLM-ovi moraju biti eksplicitno obučeni da prepoznaju pitanja na koja ne mogu odgovoriti” [23].

Slika 7. objašnjava kako RAG funkcionira na konkretnom primjeru. Ulaz u RAG sustav mogu biti različite vrste dokumenta, poput PDF-ova ili JSON datoteka. Nakon toga čita se sadržaj dokumenta i dijeli se u manje dijelove (eng. chunks) koji se mogu lakše obraditi. Svaki od tih dijelova pretvara se u vektor koristeći model ugrađivanja (eng. embedding model) koji pretvara tekstualne podatke u numerički oblik. Ti vektori se zatim pohranjuju u vektorsku bazu podataka koja omogućuje brzo pretraživanje informacija. Nakon toga dolazi do korisničkog upita koji se također pretvara u vektor. Taj vektor se uspoređuje s vektorima u bazi podataka kako bi se pronašle relevantne informacije. Nadalje se rangiraju rezultati i odabiru oni najrelevantniji. Na kraju, LLM koristi te informacije za generiranje odgovora na korisnički upit.



Slika 7: Konceptualni tok korištenja RAG-a s LLM-ovima [20]

Jedna od zanimljivih stvari ovdje su vektorske baze podataka koje se mogu učinkovito indeksirati, pretraživati te je u njih jednostavno pohraniti veliku količinu informacija o čemu će se reći nešto detaljnije u jednom od sljedećih poglavlja.

4.2.1. Faza dohvaćanja (Retriever)

Retriever je komponenta koja je odgovorna za pronalaženje relevantnih dokumenata na temelju ulaznog upita. Retriever bilježi složenije semantičke odnose između upita i dokumenta, što dovodi do točnijih rezultata dohvaćanja. Retriever može ugraditi dokumente i upite u visokodimenzionalni vektorski prostor, gdje udaljenost između vektora odgovara relevantnosti dokumenta za upit. Dokumenti koji imaju veću "udaljenost" su manje relevantni za ulazni upit i time se mogu protumačiti kao nebitni. Retriever uzima ulazni upit, pretvara ga u vektor pomoću koder upita, a zatim pronalazi najsličnije vektore dokumenta. Dokumenti povezani s tim vektorima se naknadno proslijeđuju generatoru. Broj dokumenta dohvaćenih u bilo kojem tre-

nutku može se prilagoditi na temelju specifičnih zahtjeva zadatka i obično za zadatke koji su kompleksniji može se pozvati opsežniji skup dokumenata. Bitan aspekt kod Retrievera je da se obrati pažnja na točnost dohvaćenih dokumenta tijekom pronalaženja. Postavljanje praga za udaljenost nije isključivo tehnička odluka, već i poslovna odluka. Prag bi trebao biti usklađen s općim ciljevima, bilo da se radi o pružanju sveobuhvatnog pregleda ili navođenju konkretnih detalja [30].

Korisnik postavlja pitanje ili šalje upit nakon čega Retriever traži relevantne dokumente u unaprijed definiranom izvoru znanja (npr. Wikipedia, interni podaci tvrtke ili online članci). Nakon toga se rangiraju dokumenti na temelju njihove relevantnosti za upit i za kraj se odabrani dokumenti prosljeđuju sljedećoj komponenti - generatoru - kako bi se pomoglo mu u generiranju odgovora. Retriever je važan jer za razliku od tradicionalnih AI modela, koji se oslanjaju samo na unaprijed obučene podatke, može dohvatiti informacije u stvarnom vremenu. Također, korištenje vanjskih izvora pomaže u smanjenju AI halucinacija i omogućuje modelu da se referira na pozadinsko znanje, što dovodi do smislenijih odgovora [31].

4.2.1.1. BM25

BM25 (eng. Best Matching 25) je popularni algoritam koji koristi pojam frekvencijsko-inverzna dokumentna frekvencija (eng. term frequency-inverse document frequency - TF-IDF) za rangiranje dokumenata na temelju relevantnosti. BM25 izračunava ocjenu relevantnosti dokumenta na temelju učestalosti pojavljivanja upita u dokumentu. Iako je učinkovit za podudaranja između ključnih riječi, BM25 može imati ograničenja u razumijevanju semantičkih odnosa. Unatoč tome BM25 se široko koristi zbog svoje jednostavnosti i učinkovitosti. Često se koristi za jednostavnije upite temeljene na ključnim riječima, gdje je važno brzo dohvaćanje relevantnih informacija [32].

4.2.1.2. Dense Passage Retrieval

Algoritam DPR (eng. Dense Passage Retrieval) je predstavljen 2020. godine i predstavlja moderniji pristup pretraživanja informacija. Koristi gusti vektorski prostor u kojem su upiti i dokumenti kodirani u višedimenzionalne vektore. DPR koristi arhitekturu bi-enkodera, gdje su upit i dokument kodirani odvojeno, što omogućuje učinkovito pretraživanje najbližeg susjeda. Za razliku od BM25, DPR se ističe u pretraživanju semantičkih odnosa između upita i dokumenta, što ga čini pogodnim za složenije upite koji zahtijevaju dublje razumijevanje konteksta. DPR može pronaći dokumente koji su kontekstualno povezani s upitom, čak i ako ne sadrže točne ključne riječi [32].

4.2.1.3. REALM

REALM (eng. Retrieval-Augmented Language Model) je model koji integrira pronalaženje informacija i generiranje teksta u jednom sustavu. Ključna inovacija REALM-a je da uči pronalaziti dokumente koji poboljšavaju performanse modela na određenim zadacima kao što je npr. sažimanje teksta. Tijekom treninga, REALM ažurira retriever i generativni model za-

jedno, osiguravajući da je proces pronalaženja optimiziran za zadatak generiranja. REALM je treniran kako bi prepoznao dokumente koji nisu relevantni za upit, ali su korisni za generiranje odgovora. U literaturi se često spominje da REALM nadmašuje BM25 i DPR u zadacima koji zahtijevaju kombinaciju pronalaženja i generiranja informacija [32].

4.2.2. Faza generiranja (Generator)

Generator je komponenta koja koristi dohvaćene dokumente i izvorni upit kako bi generirala odgovor. To je obično veliki model transformatora kao što je GPT3.5, GPT4, Llama2, Falcon, PaLM i BERT. Dohvaćeni dokumenti i ulazni upit ulančavaju se i unose u generator, koji zatim koristi kombinirani ulaz za generiranje odgovora pri čemu dohvaćeni dokumenti pružaju dodatni kontekst koji pomažu generatoru da bude precizniji u svojim odgovorima [30].

Generator je važan jer pretvara neobrađene podatke u koherentan i strukturiran tekst i osigurava da odgovori zvuče prirodno i privlačno. Također, kombinira znanje stečeno tijekom prethodnog treniranja modela umjetne inteligencije s dohvaćenim informacijama u stvarnom vremenu kako bi generirao najbolji mogući odgovor [31].

4.2.2.1. T5

T5 (eng. Text-to-Text Transfer Transformer) je jedan od najčešće korištenih modela za zadatke generiranja u RAG sustavima. T5 je svestran model koji pretvara sve zadatke u tekstualni format, što ga čini pogodnim za širok spektar aplikacija. Može se koristiti za odgovaranje na pitanja, sažimanje ili generiranje dijaloga. Kroz praksu se pokazalo da RAG modeli temeljeni na T5 nadmašuju tradicionalne generativne model poput GPT-3 i BART-a u raznim zadacima kao što su odgovaranje na pitanja ili sažimanje teksta [32].

4.2.2.2. BART

BART (eng. Bidirectional and Auto-Regressive Transformers) je model koji je prikladan za zadatke koji uključuju generiranje teksta iz neuređenih podataka. Za enkoder BART koristi dvosmjerni pristup, što znači da može razumjeti kontekst cijelog ulaza, dok dekoder generira tekst na temelju tog konteksta. BART može rekonstruirati oštećene tekstove, što ga čini korisnim za zadatke poput izvlačenja informacija iz nepotpunih ili neurednih podataka [32].

4.2.2.3. GPT-4

GPT (eng. Generative Pre-trained Transformer) je veliki, multimodalni jezični model koji je sposoban prihvatiti tekstualne i slikovne ulaze te generirati tekstualne izlaze. Model je transformer arhitekture, treniran je da predviđa sljedeći token u tekstu, a naknadna prilagodba poboljšava željeno ponašanje. U usporedbi s prethodnikom GPT-3.5, GPT-4 postiže bolje rezultate na zajedničkim NLP testovima. Model je treniran korištenjem RLHF (eng. Reinforcement Learning from Human Feedback) metode kako bi se poboljšala sigurnost i preciznost njegovih odgovora te time smanjile halucinacije [33].

4.3. Evaluacija i primjena RAG metode

U ovoj sekciji je opisana kratka usporedba RAG pristupa i finog podešavanja modela, s naglaskom na njihove prednosti i izazove. Opisuju se ključne prednosti RAG sustava, tipični slučajevi njegove primjene u praksi te izazovi koji se pritom javljaju, poput halucinacija i upravljanja višestrukim izvorima podataka.

4.3.1. Usporedba: RAG vs. Fine-tuning

Fino podešavanje je metoda kod koje se uzima prethodno obučeni jezični model i dodatno se obučava na određenom, ciljanom skupu podataka kako bi se prilagodile njegove težine za određenu domenu. Tijekom finog podešavanja, model uči iz novih primjera kako bi mogao proizvesti specijaliziranije i točnije rezultate za određenu domenu. Fino podešavanje kreće odabirom modela opće namjene koji već široko razumije jezik. Nadalje se priprema skup podataka koji je relevantan za određenu domenu (npr. modeli opće namjene mogu imati poteškoća s medicinskom terminologijom.) Kvaliteta i relevantnost skupa podataka izravno određuje koliko će dobro fino podešeni model funkcionirati. Kroz npr. nadzirano učenje, model ponovno uči na novim podacima, prilagođavajući svoje interne parametre kako bi se bolje uklopili uzorci u skupu podataka za fino podešavanje. Proces učenja više puta propušta te primjere kroz neuronsku mrežu, izračunavajući pogreške između predviđanja modela i željenih izlaza, a zatim prilagođavajući težine kako bi se smanjile pogreške [34].

Kada se uspoređuje RAG i fino podešavanje, postoji nekoliko ključnih čimbenika koji mogu pomoći pri odabiru prikladnije metode za konkretni problem.

Kriterij	RAG	Fino podešavanje
Razmatranje troškova	Zahtijeva niža početna ulaganja; kasniji troškovi povezani su s hostingom baze i infrastrukturom za pretraživanje.	Zahtijeva značajne početne računalne resurse za obuku, ali nudi niže troškove po upitu.
Privatnost podataka	Podaci se čuvaju odvojeno u kontroliranim bazama, što omogućuje njihovo brisanje po potrebi.	Podaci su ugrađeni izravno u model, što može predstavljati sigurnosni rizik za neke organizacije.
Inženjerski resursi	Zahtijeva znanje o bazama podataka i pretraživačkim sustavima.	Zahtijeva znanje iz područja strojnog učenja, rad s GPU-om i iskustvo u treniranju modela.
Razina kontrole nad modelom	Nudi manje izravne kontrole, ali omogućuje lakša ažuriranja i ispravke.	Omogućuje potpunu kontrolu nad ponašanjem modela i stilom odgovora.

Tablica 2: Usporedba RAG pristupa i finog podešavanja modela [34]



Slika 8: Fino podešavanje proces (generalno); Izvor

Izbor između RAG-a i finog podešavanja je više tehnička odluka. RAG se ističe kada su potrebne ažurne informacije ili kada su ograničeni resursi za obuku. Fino podešavanje je izvrsno za specijalizirane zadatke koji zahtijevaju stručnost u određenoj domeni. Postoji opcija hibridnog pristupa gdje je moguće kombinirati ove dvije metode. Krajnja odluka bi trebala biti usklađena s potrebama i ograničenjima. Treba se uzeti u obzir proračun, učestalost promjena podataka, zahtjeve za privatnost, dostupnost inženjerskih resursa i razinu kontrole koja je potrebna nad rezultatima modela. Uspjeh bilo kojeg pristupa ovisi ponajviše o kvaliteti podataka tijekom cijelog procesa. Loša kvaliteta podataka potkopava RAG i fino podešavanje. Nebitni ili zastarjeli dokumenti smanjuju performanse RAG-a, dok krivo označeni ili pristrani podaci za obuku stvaraju nepouzdanu fino podešenu modele. Ovakvi problemi s kvalitetom podataka narušavaju performanse umjetne inteligencije, što dovodi krajnje štete i narušavanja povjerenja korisnika [34].

4.3.2. Prednosti RAG metode

U nastavku, su prikazane ključne prednosti RAG sustava, koje objašnjavaju zašto je ova tehnologija sve češći odabir u izradi modernih AI rješenja.

- **Isplativa implementacija:** Razvoj AI sustava započinje korištenjem temeljnog modela (eng. Foundation models). Temeljni modeli su obično dostupni putem API ključeva i obučeni su na širokom spektru generaliziranih i neoznačenih podataka. Računalni i financijski troškovi za treniranje takvih modela su jako visoki.
- **Trenutne informacije:** Iako su izvorni podaci LLM-a relevantni za specifične potrebe, teško je održati relevantnost. RAG se može koristiti za izravno povezivanje LLM-a s feedovima društvenih medija, web-stranicama s vijestima i samim time pružiti korisnicima najnovije informacije.
- **Povećano povjerenje korisnika:** RAG omogućuje prikazivanje točnih informacija s navođenjem izvora ili citata. Korisnici također mogu pretraživati izvorne dokumente ako im je potrebno dodatno pojašnjenje.
- **Više kontrole za razvojne programere:** Učinkovitije testiranje i poboljšavanje aplikacija. Kontroliranje i mijenjanje izvora informacija. Rješavanje i ispravljanje problema.

4.3.3. Najčešći slučajevi primjene

Kroz sljedeću sekciju istražena je primjena RAG-a. Na početku je istaknut jedan primjer koji je podijelio Siddharth Asthana u jednom od svojih članaka na LinkedInu [35]. U članku se govori o tome kako je velika bolnička mreža integrirala RAG u svoj sustav podrške kako bi donijela bolje odluke. Sustav se povezao s elektroničkim zdravstvenim kartonima i višestrukim medicinskim bazama podataka, što je dovelo do [35]:

- Smanjenja pogrešnih dijagnoza za složene slučajeve za 30 %,
- Smanjenja vremena koje su liječnici proveli pregledavajući literaturu za 25 %,
- Povećanje ranog otkrivanja rijetkih bolesti za 40 %.

U nastavku je navedeno nekoliko domena u kojima se RAG može primjenjivati:

- **Sustavi za korisničku podršku** - kupac postavlja pitanje, chatbot dohvaća relevantne informacije iz izvora kao što su baze znanja, često postavljena pitanja ili zapisi kupaca i pritom koristi generativni model za izradu personaliziranog odgovora na temelju dohvaćenih podataka [36].

- u istraživanju iz Journal of Chemical Education, primjer je chatbot koji je integriran u **edukacijske platforme za kemiju**, gdje se koristi RAG da bi iz baze kemijskih pojmova i znanstvenih članaka izvukao i generirao precizne odgovore na studentska pitanja. Na primjer, chatbot je bio u stanju precizno odgovoriti na tehničke upite o konfiguraciji molekula i kemijskim reakcijama, koristeći specifične izvore poput službenih laboratorijskih priručnika. Time je povećana točnost odgovora za 20 % u odnosu na generativne modele bez retrievala [37].

- **Sažimanje i pretraživanje dokumenta** - korištenje naprednih tehnika dohvaćanja informacija kako bi se poboljšale mogućnosti velikih LLM-ova. Na primjer, Google-ov Vertex AI Search koristi prvo algoritme poput ANN kako bi brzo prikupio rezultate i zatim primjenjuje modele dubokog učenja za ponovo rangiranje kako bi se osiguralo da su najrelevantniji dokumenti prioritetni [36].

- u radu s arXiva, prikazan je primjer primjene RAG sustava za pravno sažimanje dokumenata gdje je model u mogućnosti izvući i sažeti ključne **pravne odredbe iz stotina stranica sudskih presuda**. Sustav je konkretno korišten za analizu višegodišnjih sudskih slučajeva i generiranje sažetaka koji olakšavaju odvjetnicima brzo donošenje odluka, eliminirajući potrebu za ručnim pregledom. Također, koristi se na financijskim tržištima za sažimanje financijskih izvještaja, kako bi se izdvojili ključni pokazatelji i uočile promjene u poslovanju [38].

- **Medicinska dijagnostika i istraživanje** - korištenje ogromnih baza podataka medicinskog znanja, elektroničkih zdravstvenih kartona, kliničkih smjernica i medicinske literature kako bi se podržalo zdravstvene djelatnike u postavljanju točnih dijagnoza

i dobro informiranih odluka o liječenju. Jedan od primjera je IBM Watson koji analizira podatke o pacijentima s opsežnom medicinskom literaturom i time pomaže liječnicima u dijagnosticiranju složenijih slučajeva. IBM Watson Health se također koristi za dijagnostiku raka i shodno tome predlaže preporuke liječenja [36].

- Prema PMC članku, primjer je IBM Watson Health, koji koristi RAG u **onkologiji za analiziranje pacijentovih genetskih podataka i povezivanje s najnovijim istraživanjima** kako bi se predložili personalizirani terapijski protokoli. Sustav je identificirao alternativne terapije za određene vrste raka koje su klinički potvrđene, ali nisu standardno korištene na svim lokacijama, što je rezultiralo boljim stopama preživljavanja pacijenata [39].

- **Personalizirani sustavi učenja i poučavanja** - jedan od primjera je RAMO (eng. Retrieval Augmented Generation for MOOCs) koji se bavi problemom hladnog starta u preporukama tečajeva korištenjem LLM-ova za generiranje personaliziranih prijedloga tečajeva. RAMO pomaže u razumijevanju preferencija i ciljeva osobe, nudeći relevantnije mogućnosti tečaja. Na primjer, sveučilišta su počela uvoditi sustave poučavanja vođene RAG-om kako bi pomogli studentima u učinkovitijem snalaženju u materijalima, potičući dublje razumijevanje i bolje akademske rezultate [36].

- PeerJ-ova studija opisuje LPITutor kao primjer RAG sustava koji je **integriran u sustav za učenje matematike**, gdje LPITutor na temelju prethodnih rješenja pruža vježbe koje ciljaju na slabije razumijevanje pojmova kao što su derivacije ili integrali. Sustav koristi retrieval da bi iz baze obrazovnih materijala obradio primjere i time generirao dodatne korake koji su prilagođeni specifičnom učeniku [40].

- **Otkrivanje prijevara i procjena rizika** - sposobnost RAG-a da pristupi podacima u stvarnom vremenu i uključi ih tijekom postupka donošenja odluka. Financijske tvrtke poput JPMorgan Chasea koriste sustave za otkrivanje prijevara koje su vođene umjetnom inteligencijom. Takvi sustavi kontinuirano dohvaćaju i analiziraju podatke u stvarnom vremenu iz različitih izvora kako bi se pratile transakcije i time otkrile potencijalne prijevare [36].

- Rad na arXiv-u navodi JPMorgan Chase kao primjer **financijske institucije koja je implementirala RAG u sustav za detekciju prevara**. Sustav kontinuirano pretražuje transakcijske podatke, regulatorne izvještaje i baze prijavljenih prijevara kako bi otkrio obrasce nepravilnosti. U stvarnom vremenu sustav je identificirao složene sheme pranja novca koje se nisu mogle otkriti u standardnim modelima, čime je uštedena velika količina novca. Model također koristi retrieval za brzo učenje o novim prijevarama tako što ažurira svoje baze što je rezultiralo smanjenjem lažno pozitivnih alarma [41].

- **Preporuke proizvoda za e-trgovinu** - za razliku od tradicionalnih sustava preporuka, RAG se dinamički prilagođava specifičnim potrebama kupca u stvarnom vremenu čime preporuke postaju relevantnije i točnije što rezultira povećanom prodajom [36].

- Amazon Science opisuje Cosmo framework kao primjer RAG primjene u **e-commerce**, gdje je model u mogućnosti pružiti automatske preporuke novim i povratnim kupcima. Konkretno, tijekom pretraživanja proizvoda, model pretražuje podatke o sličnim proizvodima, recenzijama i prometnim statistikama, te kreira personalizirane prijedloge u stvarnom vremenu. Testovi su pokazali da uvođenje Cosmo frameworka povećava broj izvršenih kupovina za 12 % preporuke koje uključuju dodatke povezane s interesima kupca [42].

- **Upravljanje znanjem u poduzeću** - posebno korisno kod velikih tvrtki s opsežnom dokumentacijom i izvorima podataka [36].

- Studija iz World Journal Of Advanced Research, opisuje Siemens kao konkretan primjer korištenja RAG u internoj bazi znanja. **Sustav povezuje tehničke priručnike, interne procedure i izvještaje s ciljem ubrzavanja tehničke podrške.** Zaposlenici postavljaju prirodne upite na platformi, a RAG sustav vraća relevantne dijelove dokumenata te sažete odgovore. Time se smanjilo vrijeme potrebno za pronalaženje rješenja za tehničke probleme za 30 %, a istovremeno se povećala točnost i sigurnost odgovora u kompleksnim proizvodnim procesima [43].

4.3.4. Izazovi RAG metode

LLM-ovi predstavljaju jednu od tehnologija umjetne inteligencije koja se često koristi za pokretanje AI sustava. Priroda LLM-ova je takva da ponekad unose dozu nepredvidljivosti u generiranje odgovora. Osim toga, podaci o LLM obuci su statični i imaju krajnji rok za znanje koje posjeduju. Negativne strane LLM-ova uključuju [20]:

- Iznošenje lažnih informacija kada na korisnikovo pitanje nema odgovora (nepoželjni efekti nazvani - halucinacijama),
- Prikazivanje zastarjelih informacija kada korisnik očekuje aktualan odgovor,
- Izrada odgovora iz neautoriziranih izvora,
- Stvaranje netočnih odgovora zbog terminološke zbrke, različiti izvori obuke koriste istu terminologiju za opisivanje različite stvari.

Veliki jezični modeli ponekad mogu davati nedosljedne odgovore, što znači da će u nekim slučajevima pružiti točan odgovor, a u drugima generirati netočne ili nasumične rezultate. To proizlazi iz činjenice da oni zapravo ne razumiju značenje riječi niti rečenica u cijelini, već se oslanjaju na statističke obrasce i vjerojatnosti. RAG smanjuje potrebu za ponovnim treniranjem modela na novim podacima i ažuriranjem njegovih parametara, čime se mogu smanjiti računalni i financijski troškovi implementacije različitih AI sustava [23].

Neki od izazova kod RAG pristupa [44]:

- **Sadržaj koji nedostaje** - jedan od glavnih izazova je kada odgovor na upit korisnika nije prisutan u indeksiranim dokumentima. U takvim slučajevima sustav

je sklon generiranju obmanjujućih odgovora ili u nekim slučajevima ne prepoznaje da mu nedostaju potrebne informacije. Na primjer, idealno ponašanje u takvim slučajevima je najbolje priznati poput: "Žao mi je, ne znam", iako to većina sustava ne uspijeva, što dovodi do smanjenog povjerenja korisnika. Ovaj izazov se može umanjiti povremenim dodavanjem podataka, implementacijom povratnih informacija kroz pogreške kako bi se ukazalo na nedostatak informacija i uvođenje prestanka generiranja odgovora kada odgovor ne zadovoljava potrebe korisnika.

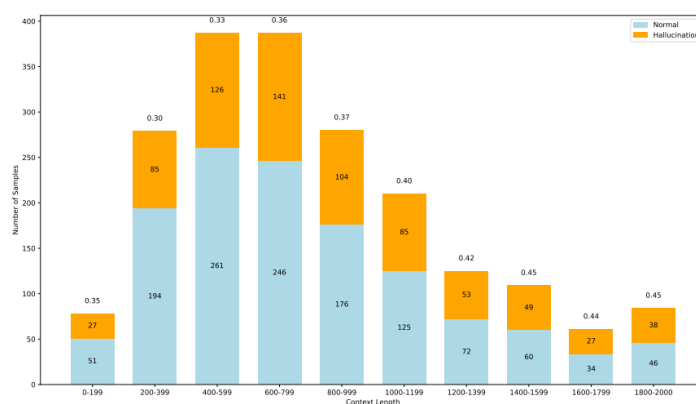
- **Neoptimalno dohvaćanje i rangiranje** - u slučaju da odgovor postoji unutar indeksiranog skupa podataka, moglo bi se desiti da neće biti dovoljno visoko rangiran za dohvaćanje npr. zbog neoptimalnih algoritama rangiranja. Algoritmi rangiranja koji se oslanjaju isključivo na rezultate sličnosti često zanemaruju kontekst i specifičnost. Ovaj izazov se može umanjiti poboljšavanjem mehanizama rangiranja uključivanjem metapodataka kao što su vrste dokumenata, autorstvo ili datum objavljivanja. Također, može se umanjiti s eksperimentiranjem s jačim modelima rangiranja kako bi se vidjelo hoće li se relevantniji dokumenti drugačije pozicionirati.
- **Proturječne informacije** - proturječne informacije mogu dovesti do toga da LLM generira zbunjujuće informacije (halucinacije). Na primjer, RAG sustav korisničke podrške dohvaća informacije zastarjele politike uz trenutne, uzrokujući da LLM generira netočne ili zbunjuće odgovore. To se može riješiti dodatnim filtriranjem nebitnih i kontradiktornih informacija prije nego što dođu do LLM-a. Dodatno trebalo bi još provesti testiranja i fino podesiti upite kako bi se usmjerila pozornost LLM-a na najrelevantnije / najnovije informacije.
- **Nepotpuni odgovori** - nepotpuni odgovori nastaju kada sustav ne uspije spojiti sve relevantne informacije. Na primjer, RAG sustav koji se bavi domenom prava i od njega se traži da sažme ključne točke tri različita slučaja, može se baviti samo jednim slučajem, izostavljajući kritične detalje iz ostala dva.
- **Izazovi s performansama i skalabilnošću u pronalaženju** - takvi izazovi nastaju kada se sustav bori s učinkovitim rukovanjem skupova podataka velikih razmjera. Kako veličina skupa raste, latencija dohvaćanja se povećava zbog računalnih troškova u pretraživanju, rangiranju i dohvaćanju relevantnih dokumenata.

4.3.4.1. Halucinacije

Veliki jezični modeli skloni su generiranju uvjerljivih, ali činjenično netočnih odgovora, koji se obično nazivaju halucinacijama. Halucinacije predstavljaju ozbiljnu prijetnju pouzdanosti i sigurnosti LLM-ova u praktičnoj primjeni [45].

Metode za otkrivanje halucinacija obično se kategoriziraju u nekoliko vrsta [45]:

- **Metode temeljene na LLM-u** - dizajniraju upute ili izravno fino podešavaju LLM-ove za otkrivanje halucinacija. Prvo se analiziraju uzorci RAG halucinacija, a zatim se primjenjuje obrezivanje konteksta kako bi se provjerila konzistentnost između unutarnjih stanja i izlaza modela,
- **Metode temeljene na nesigurnosti** - procjenjuju nesigurnosti na razini tokena,
- **Metode temeljene na konzistentnosti** - primjenjuju smetnje na LLM ulaz a zatim se procjenjuje samokonzistentnost izlaza. Slične metode potiču LLM-ove da generiraju više odgovora na isto pitanje i procjenjuju samokonzistentnost tih odgovora,
- **Metode specifične za zadatak** - fokusiraju se na specifične NLP zadatke kao što su strojno prevodenje, generiranje dijaloga i odgovaranje na pitanja. Od drugih metoda predlaže se pristup temeljen na grafovima, kombiniranje samonadgledanog i kontrastivnog učenja kako bi se otkrile i ublažile halucinacije u vizualnim modelima.



Slika 9: Utjecaj duljine konteksta na zastupljenost halucinacija u odgovoru [45]

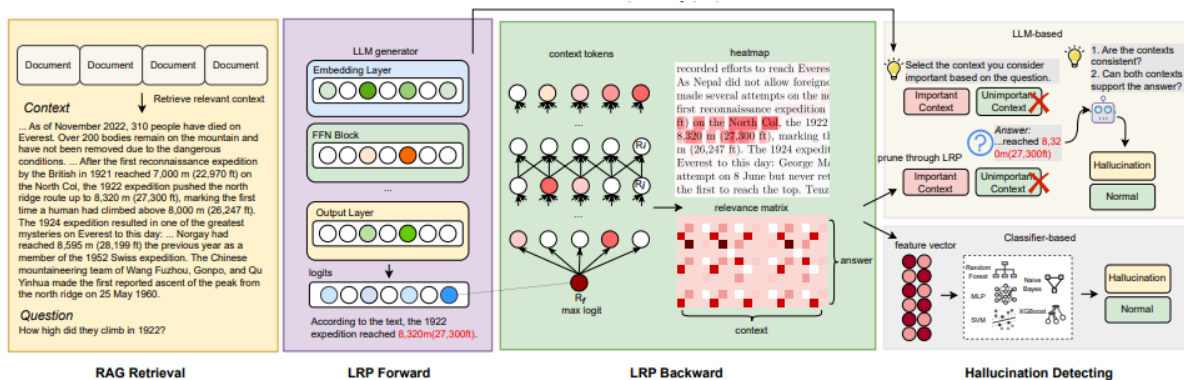
LLM-ovi često ne uspijevaju identificirati najrelevantnije informacije kada se bave dugim kontekstima. Kao što je prikazano na slici 9., stopa halucinacija općenito raste s duljinom konteksta, dosežući svoju najvišu točku od oko 45 % kada se duljina konteksta približi 2000 tokena[45].

Drugi uzrok postojanja halucinacija u RAG-u je neusklađenost između riječi i misli. Kada se zatraži da se identificiraju dijelovi konteksta koji su najrelevantniji za zadano pitanje, LLM obično može dati odgovor. Taj odgovor se naziva riječima modela, što predstavlja eksplicitni dokaz odgovora modela. Postavlja se prirodno pitanje odražavaju li te riječi doista unutarnje misli modela. Kako bi se to pratilo može se koristiti LPR (eng. Layer wise Relevance Propagation) koji prati koji dijelovi konteksta najviše doprinose odgovoru modela [45].

Complete Context: Sir John Evelyn (1591–1664) was an English politician who sat in the House of Commons at various times between 1628 and 1660. He reluctantly supported the Parliamentary side in the English Civil War. Evelyn was the son of Sir John Evelyn of Kingston, Godstone, Surrey and Marden, MP and his wife Elizabeth Stever, daughter of William Stever of Kingston upon Thames. He was baptised at Kingston upon Thames on 20 October 1591. He was admitted at Emmanuel College, Cambridge on 13 March 1606. He was a member of the Virginia Company in 1612 and of the East India Company in 1624. He was a JP for Surrey from 1624. Read the above context and answer the following question. Question: Who did John Evelyn support during the English Civil War? GroundTruth Answer: the Parliamentary side Hallucinated Answer: unanswerable Correct extracted by LLM: Evelyn reluctantly supported the Parliamentary side in the English Civil War. ✓ Context extracted by LRP : Sir John Evelyn (1591–1664) was an English politician who sat in the House of Commons at various times between 1628 and 1660. He reluctantly supported the Parliamentary side in the English Civil War. Evelyn was the son of Sir John Evelyn of Kingston, Godstone, Surrey and Marden, He was a JP for Surrey from 1624. ✗	Complete Context: The University of California, Los Angeles (UCLA) is a public land-grant research university in Los Angeles, California. UCLA's academic roots were established in 1881 as a normal school then known as the southern branch of the California State Normal School (now San José State University). This school was absorbed with the official founding of UCLA as the Southern Branch of the University of California in 1919, making it the second-oldest of the 10-campus University of California system (after UC Berkeley). Question: List the two oldest University of California campuses separated by a semicolon. GroundTruth Answer: UC Berkeley; UCLA Hallucinated Answer: unanswerable Context extracted by LLM: UCLA's academic roots were established in 1881 as a normal school then known as the southern branch of the California State Normal School (now San José State University). ✗ Context extracted by LRP : The University of California, making it the second-oldest of the 10-campus University of California system (after UC Berkeley). ✓
---	--

Slika 10: Nesklad između eksplicitnog i implicitnog dokaza [45]

Slika 10. pokazuje dva QA primjera koji naglašavaju nesklad između eksplicitnog dokaza (rečenica koju izdvoji LLM) i implicitnog dokaza (ono što model zapravo pripisuje odluku preko LRP-a). Lijevo (John Evelyn): LLM ispravno izvuče rečenicu, dok LRP izdvoji širi i nekonzistentni kontekst. Desno LLM izvuče nebitnu rečenicu o osnivanju, a LRP pogodi pravi dokaz. Poanta je da eksplicitni i implicitni tragovi ne moraju se poklapati, ponekad je jedan ispravan, a drugi promašen [45].

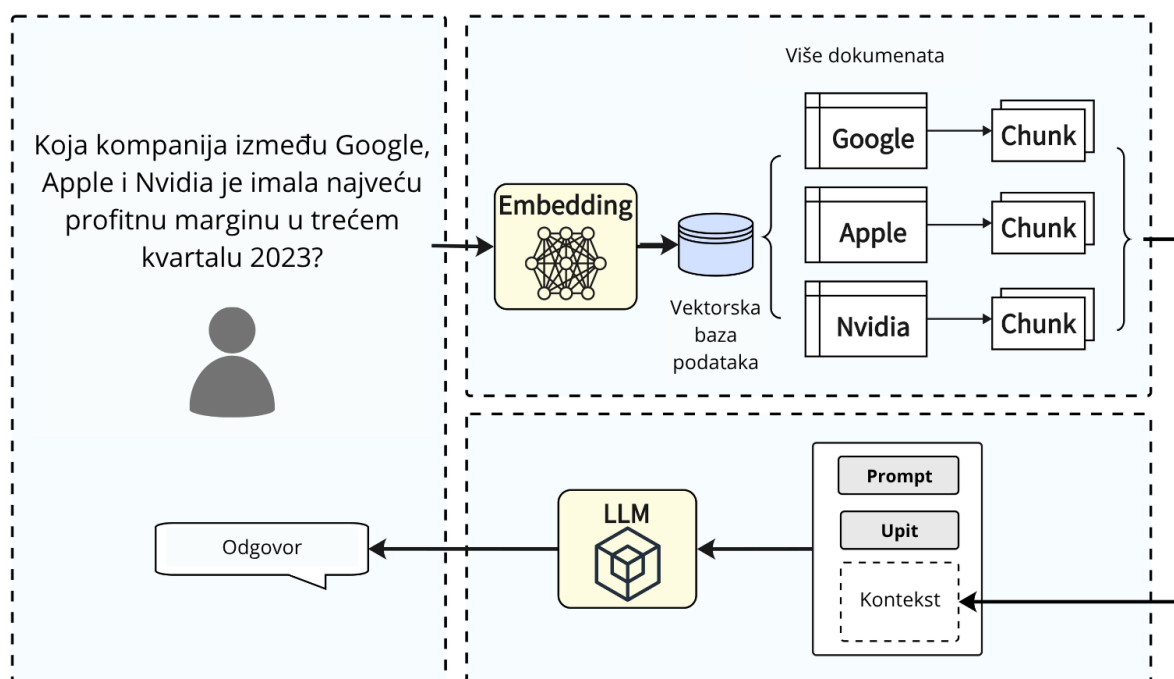


Slika 11: Arhitektura LPR4RAG metode za otkrivanje halucinacija [45]

Slika 11. ilustrira pregled metode LPR4RAG gdje se prvo uz zadani ulazni upit, dohvaća relevantan kontekst iz baze znanja, a zatim formatira prompt na temelju upita i dohvaćenog konteksta. Tijekom faze generiranja, LPR naprijed se provodi sloj po sloj kroz model generatora, s gradijentima koji se izračunavaju i pohranjuju za svaki pojedinačni neuron. Nakon završetka LPR naprijed, izvodi se LPR natrag na temelju unaprijed definiranih LPR pravila, počevši od izlaznog sloja i šireći se kroz slojeve do ulaznog sloja. LPR unatrag proizvodi matricu relevantnosti, koja kvantificira doprinos svakog kontekstnog tokena generiranom odgovoru. U fazi otkrivanja halucinacija koriste se dvije strategije. Prva strategija je temeljena na klasifikatoru, gdje se matrica relevantnosti transformira u vektore značajki i unosi binarne klasifikatore. Druga strategija temelji se na LLM-u, gdje se izdvajaju ključni segmenti konteksta identificirani putem R-a. Nakon toga LLM se upućuje da izvrši provjeru konzistentnosti između dvije verzije konteksta i odgovora te da donese konačnu odluku je li došlo do halucinacije ili ne [45].

4.3.4.2. Upravljanje višestrukim izvorima

Upravljanje višestrukim izvorima dolazi kada model istovremeno koristi podatke iz različitih vanjskih baza kako bi generirao odgovor koji je što točniji, relevantniji i potpuniji. Prije svega, različiti izvori često donose različite informacije koje nisu uvijek usklađene. To može podrazumijevati kontradiktorne podatke, različite perspektive te redundantne podatke. Model se suočava s izazovom kako inteligentno integrirati te podatke bez stvaranja zbunjujućih ili netočnih odgovora. Ako model pogrešno spoji podatke iz nespojivih izvora podataka, povećava se rizik od generiranja lažnih informacija (halucinacija) ili gubitka relevantnih detalja. Jedan od pojmova koji se spominje kroz literaturu je višeskočno zaključivanje (eng. multi hop reasoning) kod kojeg se povezuju informacije iz više izvora kako bi se odgovorilo na složenija pitanja. Posebno se ističe u situacijama kada se problem ne može riješiti u jednom dohvat podataka već zahtijeva kombiniranje više dohvaćanja iz više različitih izvora podataka. Takav pristup nudi iterativno upravljanje procesom traženja i generiranja čime se povećava efikasnost i točnost modela [46].

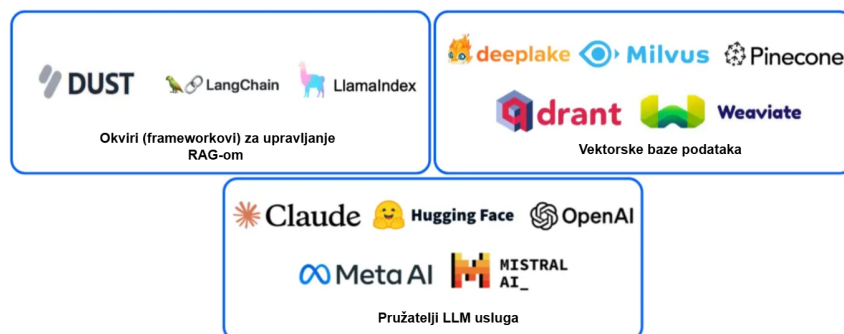


Slika 12: Višeskočno zaključivanje - primjer; Izvor

Glavna ideja primjera je da se ne oslanja samo na jedno pretraživanje ili jedan izvor, već da se kroz više iteracija i izvora spojene informacije upotrebe kako bi se dobio detaljniji odgovor.

4.4. Dostupni alati i biblioteke

U ovoj sekciji su navedeni neki od najpopularnijih alata i biblioteka koje se koriste za implementaciju RAG sustava. Izvršen je pregled i analiza svakog alata nakon čega je donesen zaključak koji od njih su korišteni u praktičnom dijelu rada. Na slici 13. može se vidjeti sažetak alata i biblioteka koje se mogu koristiti za implementaciju RAG sustava.



Slika 13: Sažetak dostupnih alata i biblioteka za razvoj RAG sustava; Izvor

4.4.1. Okviri (frameworkovi) za upravljanje RAG-om

U nastavku su navedeni neki od najpopularnijih okvira koji se koriste za jednostavniju izgradnju, testiranje i održavanje sustava temeljenih na RAG pristupu. Pružaju funkcionalnosti za povezivanje velikih jezičnih modela s vanjskim izvorima podataka kao što su baze podataka.

4.4.1.1. Haystack

Haystack je platforma otvorenog koda za obradu prirodnog jezika koja je specijalizirana za stvaranje RAG cjevovoda za sustave pretraživanja i odgovaranja na pitanja. Framework pruža komponente za dohvaćanje dokumenta, odgovaranje na pitanja i generiranje teksta, omogućujući korisnicima da brzo i jednostavno izrade RAG aplikacije. Haystack se također povezuje s modelima kao što su BERT i RoBERTa, što poboljšava njegovu sposobnost rukovanja s težim upitima. Također, Haystack ima API koji je prilagođen korisniku i korisničkom sučelju temeljenom na webu, omogućujući korisnicima jednostavnu interakciju sa sustavom [47].

Značajke koje Haystack nudi su [47]:

- Podržavanje pozadinskih sustava za pohranu kao što su Elasticsearch, FAISS, SQL i InMemory,
- Pretraživanje na temelju ključnih riječi pomoću BM25,
- GenerativePipeline - kombiniranje Retrievera i generatora (GPT - 3/4),
- TransformersReader - ekstrakcijska kontrola kvalitete koristeći Hugging Face modele,

- DensePassageRetriever - dohvaća gusto ugrađene vektore koristeći DPR (eng. Dense Passage Retrieval),
- EmbeddingRetriever - custom embeddings koristeći Hugging Face modele.

Vrlo lagano se može instalirati lokalno na Linux sustavu pomoću pip-a ili conde (pip install haystack-ai). Dokumentacija se nalazi na sljedećem linku - <https://docs.haystack.deepset.ai/docs/intro>.

4.4.1.2. LangChain

LangChain je Python paket otvorenog koda koji pruža sveobuhvatnu osnovu za izgradnju aplikacija s velikim jezičnim modelima. Kombinira modularni i fleksibilni dizajn sa sučeljem visoke razine, što ga čini idealnim za razvoj RAG sustava. Također, LangChain omogućuje jednostavnu integraciju različitih izvora podataka, uključujući dokumente, baze podataka i API-je što pomaže u procesu generiranja odgovora. LangChain nudi širok raspon funkcionalnosti i omogućuje korisnicima izmjenu i kombiniranje različitih komponenti kako bi odgovarale jedinstvenim zahtjevima aplikacije, što olakšava stvaranje dinamičkih i prilagodljivih aplikacija [47].

Značajke koje LangChain nudi su [47]:

- Integracija s vektorskim bazama podataka kao što su Pinecone, Chroma i FAISS,
- Učitavanje i dohvaćanje podataka iz baza podataka (SQL/NoSQL), API-ja i lokalnih datoteka, PDF-ova, tekstualnih datoteka, te sadržaja dobivenog web scrapingom,
- Retrieveri uključuju BM25, Chroma, FAISS i druge,
- Generiranje dinamičkih upita koji se automatski oblikuju prema unaprijed definiranom predlošku, ovisno o korisničkom unosu i dostupnim podacima.

LangChain se može instalirati lokalno na Linux sustavu koristeći pip (pip install -U langchain). Dokumentacija se nalazi na - <https://python.langchain.com/docs/introduction/>

4.4.1.3. LlamaIndex

LlamaIndex je robusna biblioteka otvorenog koda koja se usredotočuje na učinkovito indeksiranje i dohvaćanje iz ogromnih skupova podataka. LlamaIndex koristi napredne tehnike kao što su pretraživanje vektorske sličnosti i hijerarhijsko indeksiranje kako bi se omogućilo brzo i točno dohvaćanje relevantnih informacija, poboljšavajući mogućnosti generativnih jezičnih modela. Lako se povezuje s uobičajenim velikim jezičnim modelima, omogućujući umetanje primljenih podataka u proces stvaranja i čineći ga učinkovitim alatom za poboljšanje performansi RAG sustava [47].

- Optimizirano je dohvaćanje s malom latencijom,

- Moguće je učitavanje različitih vrsta podataka kao što su: TXT, PDF, DOC, CSV, API-je, baze podataka i web scraping,
- Kombiniranje modela (OpenAI ili Hugging Face) s vektorskim bazama podataka,
- Sadrži više indeksa kao što su:
 - **Indeks pohrane vektora** - koristi se za pohranu podataka kao vektora, što omogućuje brzo pretraživanje sličnosti kao što su sustavi za dohvaćanje dokumenata i sustavi preporuka,
 - **Indeks popisa** - jednostavan, sekvencijalan indeks za manje skupove podataka koji omogućuje brzo linearno pretraživanje,
 - **Indeks stabla** - koristi hijerarhijsku strukturu za izvođenje učinkovitih semantičkih pretraživanja čime je idealan za složene upite kod podataka gdje je jasna hijerarhija,
 - **Indeks tablice ključnih riječi** - tablica mapiranja se koristi kako bi olakšala pretraživanje temeljeno na ključnim riječima.

LlamaIndex je moguće instalirati pomoću naredbi `pip install llama-index` ili `pip install llama-index-core`. Dokumentacija se nalazi na sljedećem linku <https://docs.llamaindex.ai/en/stable/>.

4.4.2. Vektorske baze podataka

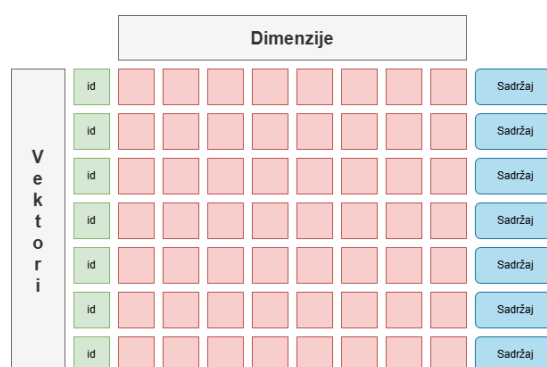
Tradicionalne baze podataka pohranjuju podatke u strukturiranom formatu (redci i stupci), dok vektorske baze podataka su optimizirane za rukovanje visokodimenzionalnim vektorskim podacima i podržavaju operacije poput pretraživanja sličnosti i pretraživanja najbližih susjeda (k-NN). Umetanje podataka se vrši kroz generiranje ugrađivanja (eng. embeddings) iz neobrađenih vrsta podataka (npr. teksta) i pohranjivanje tih ugrađivanja u vektorsku bazu podataka. Vektorske baze podataka ključne su kako bi omogućile LLM-ovima da daju točnije i skalabilnije informacije. Osim toga, vektorske baze podataka omogućuju rad s bilo kojim drugim tipovima podataka, uključujući slike, videozapise i zvuk, čime se proširuju mogućnosti RAG sustava. Većina podataka koja se generira svakodnevno su nestrukturirani podaci, bilo to tekst, slike, videozapisi ili slično. Nestrukturiranim podacima nedostaje strogi format, što otežava upravljanje konvencionalnim bazama podataka i iako se ponekad čine beznačajnim, oni sadrže vrijedne informacije koje imaju ogroman potencijal za umjetnu inteligenciju [48].

Tradicionalne baze podataka poput relacijskih odlično upravljaju strukturiranim podacima, međutim kada se pojave podaci koji se ne mogu lako kategorizirati poput sadržaja unutar PDF datoteka stvari počinju biti komplicirane. Takva baza ne može razumjeti što se nalazi u dokumentu, ne može ga kategorizirati ili pretraživati. Vektorske baze podataka dobile su na značaju u posljednjih nekoliko godina zbog porasta strojnog učenja i dubokog učenja. Vektorska ugrađivanja pretvaraju složene podatke (npr. slike ili tekst) u visokodimenzionalne vektore tako da su slične stavke bliže jedna drugoj u vektorskom prostoru [48].

Tradicionalne baze podataka su neučinkovite pri pretraživanju sličnosti u visokodimenzionalnim prostorima. Takav problem vrlo dobro rješavaju vektorske baze podataka koje vrlo dobro indeksiraju i pretražuju opsežne skupove podataka. Kako bi se proveo rad s vektorskim bazama podataka, potrebno je koristiti vektor upita koji obuhvaća kriterije pretraživanja. Sljedeći korak je korištenje metrike sličnosti za određivanje blizine između vektora upita i vektora u bazi podataka. To može uključivati metriku kao što su kosinusna sličnost, euklidska udaljenost ili Jaccardov indeks. U prošlosti su samo velike tehnološke tvrtke koje su imale resurse za njihovo stvaranje i održavanje koristile vektorske baze podataka. S obzirom na napredak tehnologije, pojavile su se mnoge vektorske baze podataka otvorenog koda koje su dostupne za korištenje i implementaciju u RAG sustavima. Tablica 6. prikazuje kada se koriste vektorske baze podataka u odnosu na tradicionalne baze podataka [48].

Karakteristika	OLTP baza podataka	OLAP baza podataka	Vektorska baza podataka
Struktura podataka	Redci i stupci	Redci i stupci	Vektori
Podaci	Strukturirani	Strukturirani / nestrukturirani	Nestrukturirani
Metoda upita	SQL (Transakcije)	SQL (agregacije i analitički upiti)	Vektorsko pretraživanje na temelju sličnosti
Fokus	Optimizirano za ažuriranje	Optimizirano za čitanje	Kontekst i semantika
Slučajevi upotrebe	CRM, obrada narudžbi	Poslovna inteligencija, skladišta	RAG, otkrivanje anomalija

Tablica 3: Usporedba OLTP, OLAP i vektorskih baza podataka [48]



Slika 14: Vektorska baza - primjer; Izvor

Slika 14. prikazuje strukturu vektorske baze podataka. Zelena polja predstavljaju jedinstveni identifikator svakog vektora, primjerice doc01. Svaki redak u crvenoj matrici odgovara jednom vektoru, odnosno nizu brojeva koji predstavlja semantičko značenje određenog sadržaja. Plava polja označena kao sadržaj sadrže dodatne informacije povezane s vektorom, poput originalnog teksta, metapodataka kao što su jezik i izvor, te putanje do dokumenta. Dimenzije vektora određuju širinu svakog vektora, odnosno broj numeričkih vrijednosti koje ga čine.

4.4.2.1. FAISS

FAISS (eng. Facebook AI Similarity Search) je robusna vektorska baza podataka koju je razvio istraživački tim Mete za umjetnu inteligenciju. Vrlo je učinkovita i brza, što je čini savršenim izborom za razvoj aplikacija u stvarnom vremenu specifičnih za industriju. Dostupno je nekoliko opcija indeksiranja, učinkovito grupira i izgrađena je na Pythonu i C++ jeziku. Jedne od poznatijih tvrtki koje koriste FAISS su Meta i Pinterest [49].

4.4.2.2. Qdrant

Qdrant je vektorska baza podataka koja je optimizirana za pretraživanje po sličnosti i omogućuje brzo i učinkovito pretraživanje velikih skupova podataka. Od dodatnih mogućnosti Qdrant nudi: mogućnost preciznog pretraživanja, pametno pohranjivanje vektora, skalabilni dizajn prilagođen oblaku. Klijenti Qdranta su: Bosch, Johnson i Johnson i Discord [49].

4.4.2.3. Pinecone

Pinecone je vektorska baza podataka u oblaku (eng. cloud-native) zatvorenog koda koja nudi upravljivost, skalabilnost i visoku učinkovitost. Jednostavnost, podrška za hibridne upite i uska integracija sa strojim učenjem čine Pinecone popularnim izborom za RAG sustave. Značajke koje Pinecone nudi su: ažuriranje u stvarnom vremenu bez zastoja, visokodimenzionalno vektorsko pretraživanje u velikoj mjeri i integracija s Pythonom, LangChainom i drugima. Neki od klijenta Pinecone su: Microsoft, Notion, Hubspot i Shopify. Pinecone je vektorska baza podataka u oblaku koja je potpuno upravljana i ne zahtijeva nikako odražavanje infrastrukture [49].

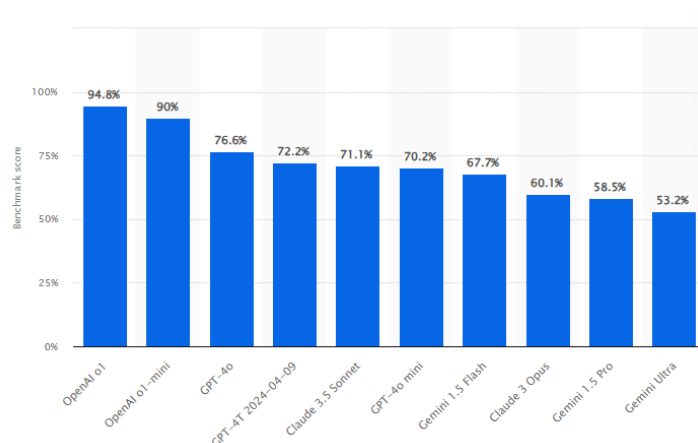
4.4.3. Veliki jezični modeli

Veliki jezični model je napredni AI model koji može razumjeti i generirati ljudski jezik. U srži LLM-ova je strojno učenje, koje je podskup umjetne inteligencije i omogućuje sustavima učenje iz podataka i poboljšanje tijekom vremena bez eksplicitnog programiranja. Također, koristi se duboko učenje kako bi se analizirala ogromna količina podataka. Neuronske mreže su jedna od opcija kod kojih se obučava model na velikim skupovima podataka kako bi se prepoznali odnosi između riječi i fraza. LLM-ovi se mogu kategorizirati na mnogo načina no najčešći način kategorizacije je prema njihovoj arhitekturi i funkcionalnosti [50]:

- Generativni modeli - osmišljeni za stvaranje visokokvalitetnog teksta nalik ljudskom u raznim formatima (GPT-4),
- Modeli enkodera-dekodera - specijalizirani za zadatke poput strojnog prevođenja i sažimanje teksta (primjer T5),
- Modeli samo s enkoderom - fokusirani na razumijevanje značenja teksta (primjer BERT),

- Multimodalni LLM-ovi - obrađivanje i generiranje više vrsta podataka, uključujući tekst, slike i zvuk. Koriste se zadatke poput opisivanja slika, analize videa i multimodalnog pretraživanja kako bi se stvorila bogatija interakcija između različitih oblika podataka.

Na sljedećoj slici se može vidjeti kako je model OpenAI o1 predstavnik u rješavanju matematičkih problema za 2024. godinu.



Slika 15: Popularnost LLM-ova u rješavanju matematičkih problema za 2024 godinu; Izvor

4.4.3.1. OpenAI

OpenAI je jedan od začetnika razvoja i najpoznatijih pružatelja usluga u području umjetne inteligencije. Poznat je po razvoju GPT modela, uključujući GPT-3, GPT-4 i, odnedavno, GPT-5, kao i Codexa za generiranje koda. Među dodatnim mogućnostima ističu se generiranje teksta, razumijevanje jezika te pomoć pri kodiranju, a modeli se mogu koristiti za razvoj AI sustava i drugih aplikacija. OpenAI je dostupan putem OpenAI API-ja i Microsoftove usluge Azure OpenAI [51].

4.4.3.2. Hugging Face

Hugging Face je platforma koja nudi širok spektar modela otvorenog koda, a posebno je poznata po svojoj biblioteci Transformers i alatu Model Hub. Nudi različite modele poput BERT-a, GPT-2 i T5-a, koji se mogu koristiti za razne zadatke. Transformers je biblioteka otvorenog koda koja omogućuje jednostavno korištenje i integraciju najsuvremenijih modela dubokog učenja u aplikacije. Model Hub je platforma na kojoj programeri i istraživači mogu prenositi, dijeliti i surađivati na razvoju modela. Također, Hugging Face potiče okruženje dijeljenja znanja i resursa, čime se ubrzava razvoj i potiču inovacije u području umjetne inteligencije [51].

Zaključak je da su svi navedeni alati i biblioteke korisni za implementaciju RAG sustava, ali za potrebe ovog rada korištena je kombinacija LangChaina, LLaMA modela putem Groq platforme i FAISS vektorske baze podataka. Groq platforma je odabrana za izvođenje LLaMA modela jer nudi besplatan limit korištenja, što je posebno pogodno s obzirom na to da lokalno

pokretanje modela zahtijeva znatne računalne resurse. LangChain i FAISS su odabrani zbog široke primjene u izradi sustava temeljenih na pretraživanju i generiranju odgovora.

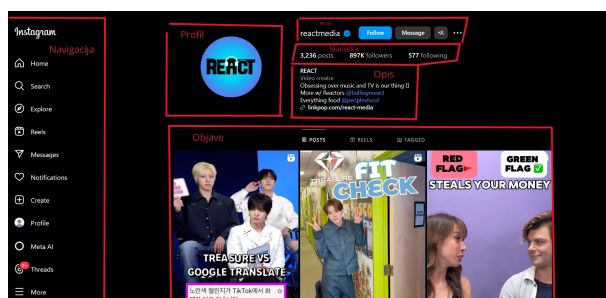
4.4.4. Ostale tehnologije

Za realizaciju praktičnog dijela ovog rada korišteno je nekoliko tehnologija koje su omogućile razvoj web-stranice i inteligentnog asistenta. Odabir tehnologija temelji se na njihovoj popularnosti, pouzdanosti i sposobnosti da zadovolje zahtjeve ovog rada. U nastavku su detaljnije opisani korišteni alati i tehnologije.

4.4.4.1. React

Svijet danas ne funkcionira bez mobilnih i web-aplikacija. Sve se orijentira prema digitalizaciji, od rezerviranja hrane do naručivanja vožnje u taksiju. Vrlo je bitno napraviti vizualno primamljivu aplikaciju a React je upravo jedna od popularnijih frontend biblioteka koja to omogućava. React je temeljen na JavaScriptu i široko se koristi u web-razvoju. U usporedbi s drugim tehnologijama, React je nova tehnologija, a osnovan je od strane Jordan Walkea 2011. godine (softverski inženjer u Facebooku) [52]. Reactova popularnost dolazi radi nekoliko razloga [52]:

- jednostavno stvaranje dinamičkih aplikacija,
- korištenje Virtual DOM-a (eng. Virtual Document Object Model), čime se aplikacije brže izrađuju (ažuriraju se samo komponente koje su promijenjene),
- podjela na komponente - jedna aplikacija se sastoji od više komponenti koje se mogu koristiti kroz cijelu aplikaciju (što smanjuje vrijeme razvoja aplikacije),
- mala krivulja učenja - React je jednostavan za učenje jer kombinira HTML i JavaScript koncepte,
- koristi se za razvoj web i mobilnih aplikacija - React Native - za mobilne aplikacije,
- jednostavno otklanjanje grešaka - Facebook je izdao Chromeovo proširenje koje se može koristiti za otklanjanje pogrešaka u React aplikacijama.



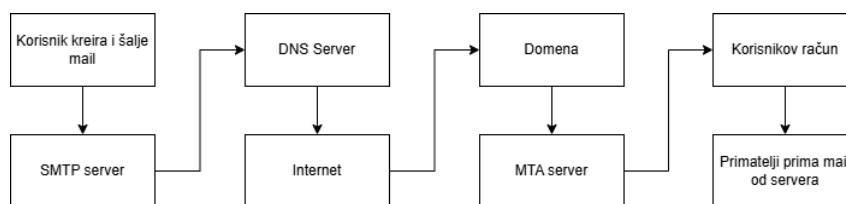
Slika 16: React komponenta

Jedna zanimljivost vezana uz React programere, promatrana na tri potpuno različite lokacije, otkriva da postoje velike razlike u plaćama s obzirom na geografsku lokaciju. To jasno odražava razliku u životnom standardu i ekonomskim potrebama pojedinih zemalja [52].

- "Prosječna plaća za početnog React developera u SAD-u iznosi oko 87000 USD godišnje" [52],
- "Prosječna plaća za početnog React developera u Indiji iznosi oko 8000 USD godišnje" [52],
- Prosječna godišnja plaća React developera u Hrvatskoj iznosi oko 14.000 EUR za početnu (junior) razinu, oko 20.000 EUR za srednju (mid-level) razinu te oko 27.000 EUR za višu (senior) razinu [53].

4.4.4.2. SMTP

"SMTP (eng. Simple Mail Transfer Protocol) je TCP/IP protokol koji se koristi za slanje i primanje e-pošte preko mreže kao što je internet" [54]. Neki od primjera klijenata koji koriste SMTP su Gmail, Microsoft Outlook, Yahoo Mail i Apple Mail [54].



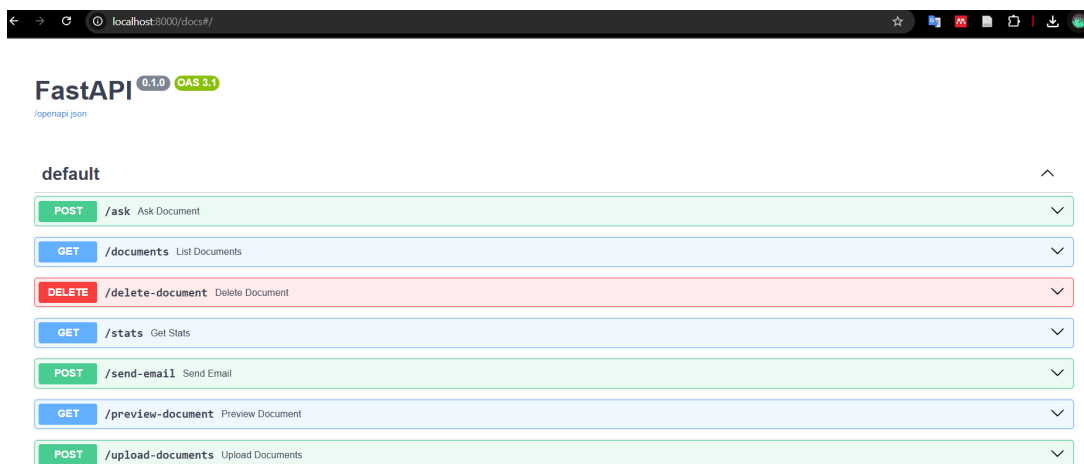
Slika 17: Proces dostavljanja emaila; Izvor

Prethodna slika prikazuje proces slanje e-pošte kroz SMTP protokol. Proces se odvija prema sljedećim koracima [54]:

1. Korisnik piše i šalje email npr. putem web-aplikacije,
2. Email odlazi na SMTP poslužitelj koji je konfiguriran za primanje poruka i njihovo prosljeđivanje,
3. Nakon toga SMTP server kontaktira DNS server kako bi pronašao IP adresu poslužitelja primatelja. DNS prevodi adresu e-mail domene u IP adresu,
4. Kada se sazna gdje poruka ide, ona putuje kroz internet prema odredišnom serveru,
5. Email stiže na domenu primatelja i njen MTA (eng. Mail Transfer Agent) poslužitelj, koji prima poruku i pohranjuje u korisnikov sandučić,
6. Email je pohranjen na serveru primatelja i čeka da ga korisnik preuzme,
7. Primatelj može dohvatiti poruku putem IMAP (ostaje na serveru, pristup s više uređaja) ili POP (skida se lokalno, briše se sa servera) načina.

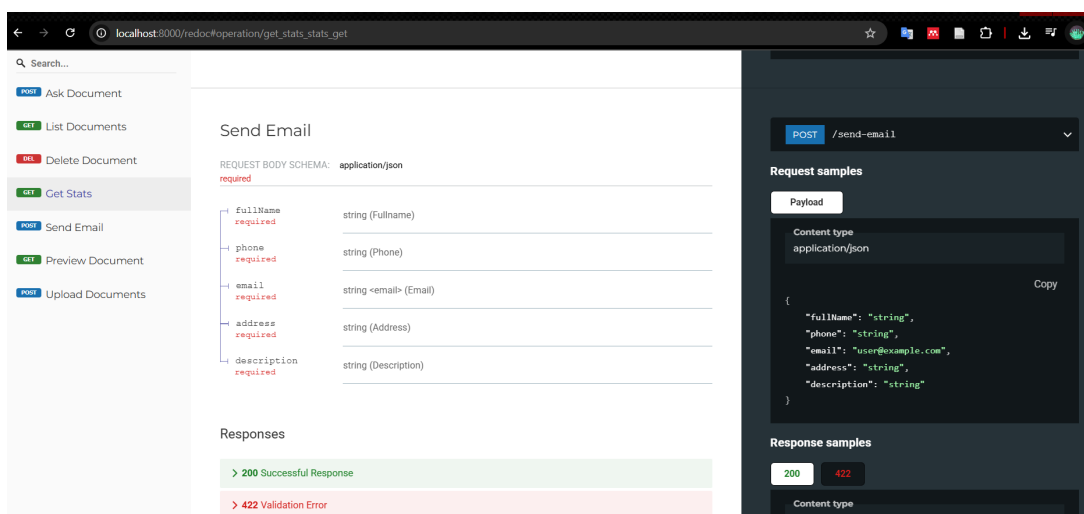
4.4.4.3. FastAPI

FastAPI je moderan, brz, visokoučinkovit web framework za izgradnju API-ja s Pythonom. Vrlo je brz (u rangu s NodeJS-om i Go-om) i omogućuje brzo kodiranje i pojavljivanje manjeg broja grešaka. Dosta je intuitivan i jednostavan za korištenje i učenje. FastAPI podržava CRUD (eng. Create, Read, Update, Delete) operacije koje je moguće koristiti kroz ključne riječi HTTP metoda (GET, POST, PUT, DELETE). Za pokretanje FastAPI-a koristi se port 8000 i može se testirati putem preglednika ili alata poput Postmana. Putanja `8000/docs` omogućuje pristup Swagger UI sučelju koje omogućuje pregled i testiranje API-ja [55].



Slika 18: Interaktivna dokumentacija (OpenAPI) u Swagger stilu

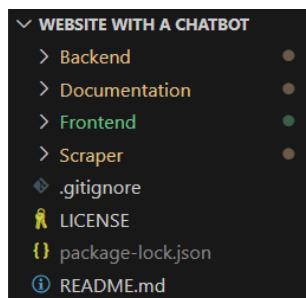
Na slici 18. je prikazano sučelje gdje je omogućeno testiranje svih dostupnih API poziva izravno kroz preglednik. Alternativno, dostupna je ReDoc dokumentacija na ruti `/redoc`, koja prikazuje API specifikaciju na čitljiviji način, ali bez mogućnosti testiranja. Također, ruta `/openapi.json` prikazuje OpenAPI specifikaciju u JSON formatu, što je korisno za automatsko generiranje klijenata ili integraciju s alatima poput Postmana.



Slika 19: ReDoc prikaz dokumentacije

5. Praktični dio rada

Praktični dio rada se sastoji od izrade web-stranice i izrade inteligentnog asistenta temeljenog na RAG metodi. Klijentski dio aplikacije je izrađen pomoću biblioteke React, pri čemu je korisničko sučelje strukturirano korištenjem JSX sintakse. Backend aplikacije razvijen je korištenjem Python web frameworka FastAPI, koji je poznat po svojoj jednostavnosti korištenja. Za izradu inteligentnog asistenta razvijen je alat za prikupljanje podataka (eng. web scraper) kojim je izrađen skup podataka korišten za rad asistenta.



Slika 20: Struktura praktičnog dijela rada

Na slici 20. se može vidjeti kako je struktura praktičnog dijela organizirana na način da postoje 4 glavna dijela: backend, frontend, scraper i dokumentacija. Ideja ovakvog pristupa je da sve što je vezano za ovaj rad bude dostupno na jednom mjestu, čime se olakšava razumijevanje ovog rada. Prije početka rada potrebno je instalirati Python, NodeJS + npm, beautifulsoup4, fastapi, uvicorn, langchain, openai i requests.

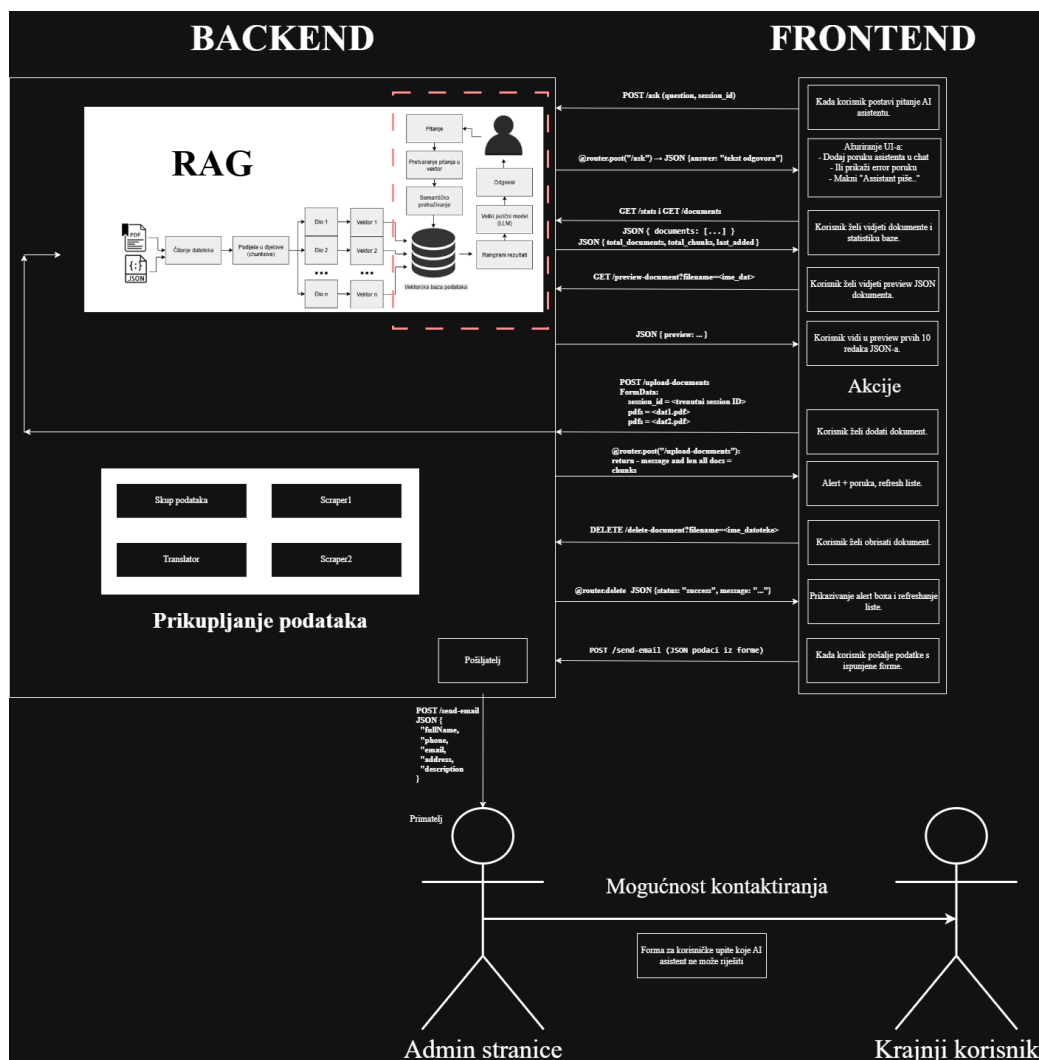
Za instalaciju i konfiguraciju LaTeX-a u lokalnom okruženju unutar VS Code-a koristan je bio sljedeći video <https://www.youtube.com/watch?v=4lyHIQ14VM8>. Za prikaz koda u dokumentu korištena je biblioteka minted, koja omogućuje sintaksno obojeni ispis izvornog koda. Budući da minted koristi vanjski program pygmentize, potrebno je omogućiti `--shell-escape` prilikom kompilacije. To se postiže izmjenom postavki u datoteci `settings.json`, gdje se dodaje odgovarajuća konfiguracija. CTRL + Shift + P (za otvaranje prozora).

```
1 {
2   "explorer.confirmDelete": false,
3   "php.validate.executablePath": "",
4   "latex-workshop.latex.tools": [{
5     "name": "pdflatex",
6     "command": "pdflatex",
7     "args": [
8       "--shell-escape",
9       "-interaction=nonstopmode",
10      "-synctex=1",
11      "%DOC%"
12    ]
13  }],
14   "latex-workshop.latex.recipes": [{
15     "name": "pdflatex",
16     "tools": ["pdflatex"]
17  }]
18 }
```

Isječak koda 1: settings.json

5.1. Arhitektura i struktura web-aplikacije

U ovoj sekciji prikazana je arhitektura sustava, kao i struktura web-aplikacije. Najprije je dan pregled toka zahtjeva i odgovora između frontenda i backenda, a zatim je prikazana struktura web-aplikacije s pripadajućim komponentama.



Slika 21: Arhitektura sustava s prikazom toka zahtjeva i odgovora

Na slici 21. prikazane su prije svega akcije koje korisnik može izvršavati na frontend strani. Izvršavanjem određene akcije aktiviraju se POST/DELETE/GET zahtjevi, na koje backend vraća odgovarajuće odgovore, a oni se potom prikazuju na frontend strani. Prilikom dodavanja dokumenta u vektorsku bazu podataka, dokument prolazi kroz proces čitanja, podjele u chunkove, ugradnje (embedding) i pohrane u bazu. Prikupljanje podataka predstavlja samostalnu i neovisnu cjelinu koja nije sastavni dio web-aplikacije, već je poslužila kao pomoćno sredstvo za prikupljanje podataka. Kod akcije u kojoj korisnik šalje podatke putem ispunjene forme, aktivira se POST zahtjev koji prosljeđuje podatke administratoru stranice kako bi bio informiran o korisničkim pitanjima. Nakon toga administrator može kontaktirati korisnika te se može nastaviti obostrana komunikacija.

Tablica 4. prikazuje pregled stranica web-aplikacije te pripadajuće komponente koje su na njima implementirane. Svaka stranica ima svoju svrhu čime se osigurava intuitivna navigacija.

Tablica 4: Struktura stranica i njihovih komponenti

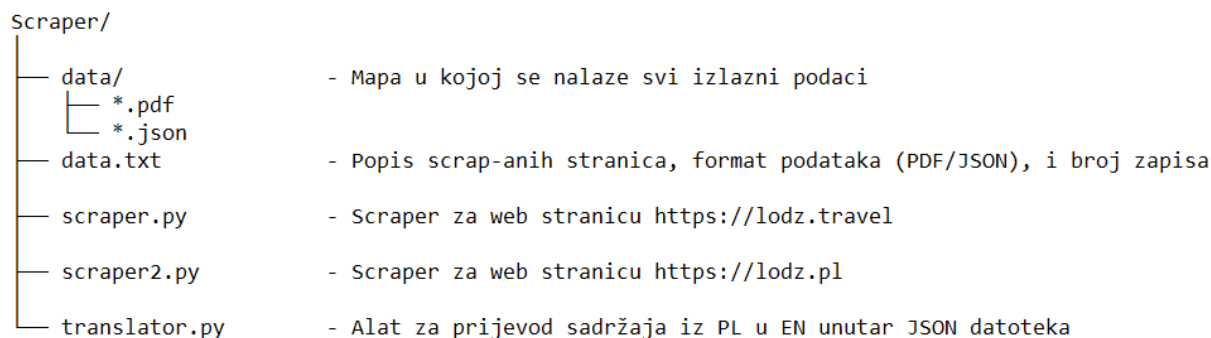
Stranica / Element	Komponente	Opis
Početna	Slider, Motivacija, Kratko o Łódžu, Drugi gradovi	Uvodna stranica s informacijama i poveznicama prema ostalim dijelovima stranice.
O Łódžu	Dodatno o gradu, Iskustva bivših studenata	Informacije o gradu i iskustvima Erasmus+ studenata.
Drugi gradovi	Pregled gradova	Pregled ostalih gradova Poljske koje je moguće posjetiti.
Pitanja	Forma za postavljanje pitanja	Korisnik postavlja pitanje ako AI asistent ne ponudi tražene informacije.
Dokumenti	Pregled, dodavanje i brisanje dokumenata za RAG	Sekcija za prikaz i pregled dokumenata koji služe kako bi se odgovorilo na pitanja korisnika.
AI asistent (pop-up)	Sučelje za razgovor	Interaktivni chat s korisnikom; pokreće se klikom na ikonu asistenta.

[Napomena] - Svaka stranica sadrži obavezne elemente kao što su podnožje (eng. footer) i zaglavlje (eng. header).

5.2. Prikupljanje podataka

Prikupljanje podataka započelo je kroz razgovor s profesoricom umjetne inteligencije koju sam upoznao tijekom Erasmus+ razmjene. Profesorica je bila izuzetno korisna osoba za ovaj rad jer je rođena i živi u gradu Łódź. Ukratko sam joj predstavio temu rada, a ona mi je preporučila nekoliko relevantnih internetskih izvora.

Za kreiranje skupa podataka korišten je Python zajedno s bibliotekom BeautifulSoup. Scraper folder je potpuno samostalna cijelina i neovisna je o backend i frontend dijelu. Unutar scraper foldera postoji data folder u kojemu su spremljeni izlazni podaci koji su u dva oblika (JSON i pdf). Data.txt predstavlja kratki sažetak koji opisuje o kakvim se podacima radi, koliko ih ima i odakle su preuzeti. Pošto su korištene dvije stranice kao izvori podataka, potrebno je bilo napraviti dva scrapera od kojih je samo za jedan u nastavku detaljnije opisana implementacija. Osim toga, pošto su stranice izvorno na poljskom jeziku bilo je potrebno napraviti skriptu koja prevodi JSONE na engleski jezik radi lakšeg razumijevanja JSON-a kako ljudima pa tako i AI modelu.



Slika 22: Struktura scraper foldera

Tablica 5. prikazuje pregled korištenih izvora, broj zapisa, naziv rezultirajuće datoteke i alate korištene za obradu podataka:

Web izvor	Veličina skupa podataka	Naziv datoteke	Korišteni alati
https://lodz.pl/	1000 zapisa	lodz_articles	scraper2.py, translator.py
https://lodz.travel/turystyka/co-zobaczyc/	185 zapisa	lodz_destinations	scraper.py, translator.py
https://polskapogodzinach.pl/lodz-atrakcje/	52 stranice	lodz_attractions	ručno konvertirano u PDF

Tablica 5: Korišteni izvori podataka i alati za obradu

Funkcija `get_soup` dohvaća HTML sadržaj stranice. Prima URL i vraća BeautifulSoup objekt za lakšu obradu HTML-a. Postavlja User-Agent kako bi se izbjeglo blokiranje od strane servera. U slučaju pogreške (npr. timeout) ispisuje se poruka i vraća None.

```

1 import requests
2 from bs4 import BeautifulSoup
3 from urllib.parse import urljoin
4 import json
5 import time
6 BASE_URL = "https://lodz.travel"
7 visited_urls = set()
8 def get_soup(url):
9     print(f"Fetching: {url}")
10    try:
11        res = requests.get(url, headers={
12            "User-Agent": "Mozilla/5.0"})
13        res.encoding = 'utf-8'
14        return BeautifulSoup(res.text, 'html.parser')
15    except Exception as e:
16        print(f"Error fetching page: {e}")
17        return None

```

Isječak koda 2: Scraper 1/6

Funkcija `parse_menu` rekurzivno prolazi kroz listu i sve li stavke u njoj. Za svaku pronađenu stranicu iz izbornika izdvaja se title i href te se stvara struktura koja sadrži title, url, content i subcategories.

```
1 def parse_menu(ul_tag, level=1):
2     data = []
3     for li in ul_tag.find_all("li", recursive=False):
4         a_tag = li.find("a", recursive=False)
5         if not a_tag:
6             continue
7         title = a_tag.text.strip()
8         href = urljoin(BASE_URL, a_tag["href"])
9         print(f'{level} * (level-1) {level} {title} -> {href}')
10        node = {
11            "title": title,
12            "url": href,
13            "content": "",
14        }
15        sub_ul = li.find("ul")
16        if sub_ul:
17            node["subcategories"] = parse_menu(sub_ul, level + 1)
18        data.append(node)
19    return data
```

Isječak koda 3: Scraper 2/6

Funkcija `fill_content_recursively` dohvaća sadržaj za svaku stranicu. Ulazi u svaku stranicu u hijerarhiji i izdvaja tekst iz diva s klasom `ce-bodytext`. Sprema pročišćeni tekst u `item["content"]` te traži dodatne poveznice unutar sadržaja i dodaje ih kao subcategories i rekurzivno ih obrađuje.

```
1 def fill_content_recursively(data):
2     for item in data:
3         if item["url"] in visited_urls:
4             continue
5         visited_urls.add(item["url"])
6
7         print(f'\nRetrieving content for: {item["title"]}')
8         soup = get_soup(item["url"])
9         if not soup:
10            continue
11
12        content_div = soup.select_one("div.ce-bodytext")
13        if content_div:
14            item["content"] = content_div.get_text(separator="\n", strip=True)
15            print(f"Content retrieved successfully ({len(item['content'])} characters)")
16
17        sub_links = []
18        for a in content_div.find_all("a", href=True):
19            href = urljoin(BASE_URL, a["href"])
20            if BASE_URL in href and href not in visited_urls:
21                title = a.get_text(strip=True)
22                if title:
23                    sub_links.append({
24                        "title": title,
25                        "url": href,
26                        "content": ""
27                    })
```

```

25         if sub_links:
26             print(f"Found {len(sub_links)} internal subpages inside:
                ↳ {item['title']}")
27             item["subcategories"] = sub_links
28             fill_content_recursively(sub_links)
29
30         if "subcategories" in item:
            fill_content_recursively(item["subcategories"])

```

Isječak koda 4: Scraper 3/6

Funkcija `count_total_items` broji sve stavke (stranice i podstranice) u hijerarhiji podataka. Koristi se za statistiku na kraju izvršavanja.

```

1 def count_total_items(data):
2     count = 0
3     for item in data:
4         count += 1
5         if "subcategories" in item:
6             count += count_total_items(item["subcategories"])
7     return count

```

Isječak koda 5: Scraper 4/6

Funkcija `main` pokreće cijeli proces. Kreće od glavne stranice navigacije gdje se dohvaća struktura poveznica pomoću `parse_menu` funkcije. Zatim se rekurzivno dohvaća sadržaj sa svih stranica pomoću `fill_content_recursively` funkcije. Na kraju svi podaci se spremaju u JSON datoteku i ispisuje se koliko je ukupno stranica pronađeno i spremljeno.

```

1 def main():
2     start_url = "https://lodz.travel/turystyka/"
3     soup = get_soup(start_url)
4     aside_nav = soup.select_one("ul.aside-nav")
5     if not aside_nav:
6         print("Navigation not found! Check the URL or HTML structure.")
7         return
8     print("Parsing main navigation...")
9     structured_data = parse_menu(aside_nav)
10    print("\nFetching content from all pages...")
11    fill_content_recursively(structured_data)
12    with open("lodz_content.json", "w", encoding="utf-8") as f:
13        json.dump(structured_data, f, indent=2, ensure_ascii=False)
14    total = count_total_items(structured_data)
15    print(f"\nTotal records extracted: {total}")
16    print("Done! Data saved to: lodz_content.json")
17 if __name__ == "__main__":
18     main()

```

Isječak koda 6: Scraper 5/6

Isječak koda 7. učitava JSON s člancima na poljskom jeziku i prevodi ih na engleski koristeći biblioteku `deep-translator`. Funkcija rekurzivno prolazi kroz sve stavke i podkategorije u JSON strukturi. Napredak prevođenja ispisuje se u konzoli, a prevedeni podaci spremaju se u novu `.json` datoteku.

```
1 import json
2 from deep_translator import GoogleTranslator

3 INPUT_FILE = "lodz_articles_pl.json"
4 OUTPUT_FILE = "lodz_articles_en.json"

5 def translate_text(text, src="pl", target="en"):
6     try:
7         return GoogleTranslator(source=src, target=target).translate(text)
8     except Exception as e:
9         return f"[Translation Error] {text}"

10 def translate_node(node, counter, total):
11     if "title" in node:
12         node["title"] = translate_text(node["title"])
13     if "content" in node and node["content"]:
14         node["content"] = translate_text(node["content"])

15     counter[0] += 1
16     print(f"[{counter[0]}/{total}] translated")

17     if "subcategories" in node:
18         for sub in node["subcategories"]:
19             translate_node(sub, counter, total)

20 def count_nodes(data):
21     count = 0
22     for item in data:
23         count += 1
24         if "subcategories" in item:
25             count += count_nodes(item["subcategories"])
26     return count

27 def main():
28     print("Loading JSON...")
29     with open(INPUT_FILE, "r", encoding="utf-8") as f:
30         data = json.load(f)

31     total = count_nodes(data)
32     print(f"Translating {total} entries...")

33     counter = [0]
34     for entry in data:
35         translate_node(entry, counter, total)

36     print("Saving translated JSON...")
37     with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
38         json.dump(data, f, ensure_ascii=False, indent=2)

39     print(f"Done! Translated file saved as: {OUTPUT_FILE}")

40 if __name__ == "__main__":
41     main()
```

Isječak koda 7: Scraper 6/6

5.3. Korisničko sučelje (Frontend)

Na slici 23. se može vidjeti struktura frontend dijela. U okviru frontend dijela prikazani su samo odabrani dijelovi koda i komponente zbog njihove opsežnosti. U slučaju interesa za konkretnim dijelom, izvorni kod je dostupan na Github repozitoriju koji je naveden u zaključku rada.

Frontend/	
node_modules/	- paketi i biblioteke potrebne za rad projekta
src/	- izvorni kod aplikacije
api/	- komunikacija s backendom (npr. slanje korisnikovog pitanja)
assets/	- slike, video materijali i druge statičke datoteke
Components/	- višekratne komponente koje se koriste unutar stranica (npr. Navigacija)
contexts/	- globalna stanja dostupna kroz cijelu aplikaciju, npr. promjena jezika
locales/	- JSON datoteke s prijevodima zbog podržavanja više jezika
pages/	- glavne stranice aplikacije
About.jsx	- informacije o gradu i iskustvima Erasmus+ studenata te video o Erasmus iskustvu
Cities.jsx	- pregled ostalih gradova Poljske koje je moguće posjetiti
Documents.jsx	- sekcija za prikaz i pregled dokumenata koji služe kako bi se odgovorilo na pitanja korisnika
Home.jsx	- uvodna stranica s informacijama i poveznicama prema ostalim dijelovima stranice
Question.jsx	- korisnik postavlja pitanje ako AI asistent ne ponudi tražene informacije
App.jsx	- definira rute pomoću BrowserRouter i omotava aplikaciju LanguageProviderom radi podrške za više jezika
main.jsx	- ulazna točka aplikacije, ovdje se React aplikacija renderira unutar HTML-a

Slika 23: Struktura frontend dijela

Funkcija `askAssistant` šalje korisničko pitanje AI asistentu putem POST zahtjeva na backend. Automatski generira i koristi jedinstveni ID sesije, koji se sprema u `localStorage`. Na kraju vraća JSON odgovor s asistentskim odgovorom ili javlja grešku ako zahtjev ne uspije.

```
1 import { v4 as uuidv4 } from "uuid";
2
3 let sessionId = localStorage.getItem("chat_session_id");
4 if (!sessionId) {
5   sessionId = uuidv4();
6   localStorage.setItem("chat_session_id", sessionId);
7 }
8
9 export async function askAssistant(question, files = []) {
10   const formData = new FormData();
11   formData.append("question", question);
12   formData.append("session_id", sessionId);
13
14   if (files && files.length > 0) {
15     files.forEach(file => formData.append("pdfs", file));
16   }
17
18   const response = await fetch("http://localhost:8000/ask", {
19     method: "POST",
20     body: formData,
21   });
22
23   if (!response.ok) {
24     throw new Error(`Request failed: ${response.statusText}`);
25   }
26
27   const data = await response.json();
28   return data;
29 }
```

Isječak koda 8: Chat.jsx

Funkcija `sendContactForm` šalje podatke iz kontakt forme na backend rutu `/send-email` koristeći POST zahtjev s JSON tijelom. Koristi se za slanje emailova s korisničkim podacima (ime, email, poruka itd.) na server putem FastAPI endpointa.

```
1 export async function sendContactForm(formData) {
2   const response = await fetch("http://localhost:8000/send-email", {
3     method: "POST",
4     headers: {
5       "Content-Type": "application/json",
6     },
7     body: JSON.stringify(formData),
8   });
9   if (!response.ok) {
10    const data = await response.json();
11    throw new Error(data.error || "Greška prilikom slanja e-maila.");
12  }
13  return response.json();
14 }
```

Isječak koda 9: Contact.jsx

Funkcija `fetchDocuments` dohvaća popis trenutno pohranjenih dokumenata sa servera. Koristi GET zahtjev prema `/documents`.

```
1 export async function fetchDocuments() {
2   const res = await fetch("http://localhost:8000/documents");
3   if (!res.ok) throw new Error("Failed to fetch documents");
4   return await res.json();
}
```

Isječak koda 10: Document.jsx 1/5

Funkcija `fetchStats` dohvaća statistiku o dokumentima s rute `/stats`. Vraća podatke poput broja dokumenta, broja ulomka i vremena zadnjeg dodavanja. Također, koristi GET i vraća grešku ako ne uspije.

```
1 export async function fetchStats() {
2   const res = await fetch("http://localhost:8000/stats");
3   if (!res.ok) throw new Error("Failed to fetch stats");
4   return await res.json();
}
```

Isječak koda 11: Document.jsx 2/5

Funkcija `deleteDocument` briše dokument prema nazivu koristeći DELETE zahtjev. Naziv datoteke se enkodira kako bi bio siguran za URL.

```
1 export async function deleteDocument(filename) {
2   const res = await fetch(`http://localhost:8000/delete-document?filename=${encodeURIComponent(filename)}`, {
3     method: "DELETE",
4   });
5   if (!res.ok) throw new Error("Failed to delete document");
6   return await res.json();
7 }
```

Isječak koda 12: Document.jsx 3/5

Funkcija `uploadDocuments` se koristi za slanje jednog ili više dokumenata (PDF/J-SON). Kreira `FormData` i uključuje ID sesije. Ako server ne prihvati datoteke, funkcija vraća grešku.

```
1 export async function uploadDocuments(files, sessionId) {
2   const formData = new FormData();
3   formData.append("session_id", sessionId);
4   files.forEach(file => formData.append("pdfs", file));
5
6   const res = await fetch("http://localhost:8000/upload-documents", {
7     method: "POST",
8     body: formData,
9   });
10
11   if (!res.ok) throw new Error("Failed to upload documents");
12   return await res.json();
13 }
```

Isječak koda 13: Document.jsx 4/5

Funkcija `previewDocument` dohvaća sadržaj dokumenta (npr. JSON) za pregled. Koristi GET zahtjev prema `/preview_document`. Vraća sadržaj u čitljivom formatu ili javlja grešku.

```
1 export async function previewDocument(filename) {
2   const res = await fetch(`http://localhost:8000/preview-document?filename=${encodeURIComponent(filename)}`);
3   if (!res.ok) throw new Error("Failed to preview document");
4   return await res.json();
5 }
```

Isječak koda 14: Document.jsx 5/5

Isječak koda 15. predstavlja About komponentu koja prikazuje sekciju s videozapisom i informacijama o gradu Łódźu. Koristi `useContext` i `LanguageContext` čime se dohvaćaju prijevodi teksta na temelju trenutno odabranog jezika (`translations.aboutPage`). Nadalje ugrađeni `iframe` iz Google Drive-a prikazuje video o Łódźu. Postavljen je na širinu 100% i visinu 400px. Za taj video bilo je potrebno odvojiti jedan dan kako bi se napisali i složili engleski titlovi kako bi i on mogao biti popraćen od strane nekoga ko ne zna hrvatski jezik. Nastavno se prikazuje naslov i dva odlomka teksta preuzeta iz konteksta jezika (`paragraph1` i `paragraph2`). Na kraju su prikazane tri ikone s pripadajućim opisima.

```
1 import React, { useContext } from 'react';
2 import { LanguageContext } from '../contexts/LanguageContext';
3 import './About.css';
4
5 const About = () => {
6   const { translations } = useContext(LanguageContext);
7
8   return (
9     <>
10      <section className='about'>
11        <div className='video'>
12          <iframe
```



```

11     src="https://drive.google.com/file/d/18k77Ntbqf1-yN5F-VsQ9CnGWgznZe6gm/_
    ↪ preview"
12     width="100%"
13     height="400"
14     allow="autoplay"
15     allowFullScreen
16     title="About Łódź Video"
17     style={{ border: "none", borderRadius: "12px" }}
18   ></iframe>
19 </div>

20   <div className='content'>
21     <h3 id='vise-o-lodzu'>{translations.aboutPage.subTitle}</h3>

22     <p>{translations.aboutPage.paragraph1}</p>
23     <p>{translations.aboutPage.paragraph2}</p>

24     <div className="icons-container">
25       <div className='icons'>
26         <i className='fas fa-shield-alt'></i>
27         <span>{translations.aboutPage.safe}</span>
28       </div>
29       <div className='icons'>
30         <i className='fas fa-leaf'></i>
31         <span>{translations.aboutPage.clean}</span>
32       </div>
33       <div className='icons'>
34         <i className='fas fa-building'></i>
35         <span>{translations.aboutPage.modern}</span>
36       </div>
37     </div>
38   </div>
39 </section>
40 </>
41 )
42 }
43 export default About;

```

Isječak koda 15: About.jsx komponenta

Isječak koda 16. postavlja kontekst za jezik aplikacije koristeći React Context API. Omogućuje da cijela aplikacija zna koji jezik je trenutno aktivan i da prema tome prikazuje prijevode. Za početak se učitavaju dva JSON-a (hr.json i en.json). State za jezik postavlja engleski kao početni jezik (`useState('en')`), dok se `setLanguage` koristi za promjenu jezika. Nadalje ako je odabrani jezik hr, koristi se hr.json, u suprotnom en.json. Funkcija `toggleLanguage` omogućuje promjenu jezika pozivom funkcije `toggleLanguage('hr')`. `LanguageContext.Provider` omogućuje da djeca komponente (`children`) pristupe vrijednostima `language`, `toggleLanguage` i `translations`.

```

1 import React, { createContext, useState } from 'react';
2 import hr from '../locales/hr.json';
3 import en from '../locales/en.json';

4 export const LanguageContext = createContext();

5 export const LanguageProvider = ({ children }) => {
6   const [language, setLanguage] = useState('en');

7   const translations = language === 'hr' ? hr : en;

```

```

8   const toggleLanguage = (lang) => {
9     setLanguage(lang);
10  };

11  return (
12    <LanguageContext.Provider value={{ language, toggleLanguage, translations }}>
13      {children}
14    </LanguageContext.Provider>
15  );
16 };

```

Isječak koda 16: LanguageContext.jsx

Isječak koda 17. predstavlja glavnu početnu stranicu aplikacije i prikazuje sve ključne sekcije. Koristi višekratne komponente poput Navbar, Slider, Motivation, Cities, Help i Footer za izgradnju cjelovitog prikaza. Stranica uključuje i pomoćne elemente poput gumba za povratak na vrh (ScrollToTopButton).

```

1  import React from 'react'
2  import Navbar from '../Components/Navbar'
3  import Footer from '../Components/Footer'
4  import Motivation from '../Components/Motivation'
5  import Slider from '../Components/Slider'
6  import AboutLodz from '../Components/AboutLodz'
7  import Cities from '../Components/Cities'
8  import Help from '../Components/Help'
9  import Icon from '../Components/AIAssistant/Icon'
10 import ScrollToTopButton from '../Components/ScrollToTopButton'

11 export default function() {
12   return(
13     <>
14       <Navbar/>
15       <Slider/>
16       <Motivation/>
17       <AboutLodz/>
18       <Cities/>
19       <Help/>
20       <ScrollToTopButton/>
21       <Footer/>
22     </>
23   )
24 }

```

Isječak koda 17: Stranica Home

Isječak koda 18. postavlja strukturu aplikacije koristeći `BrowserRouter` za upravljanje rutama. Komponenta je omotana s `LanguageProviderom` kako bi sve podstranice imale pristup jeziku i prijevodima. Unutar `Routes` definirane su rute prema komponentama Home, About, Cities, Question i Documents, dok je komponenta `Icon` prikazana globalno - neovisno o stranici (kako bi popup prozor asistenta bio i dalje otvoren iako se prelazi na druge stranice).

```

1  import React from 'react'
2  import {BrowserRouter, Routes, Route,} from 'react-router-dom';
3  import { LanguageProvider } from '../contexts/LanguageContext';
4  import Home from './pages/Home'
5  import About from './pages/About'

```

```

6 import Cities from './pages/Cities'
7 import Question from './pages/Question'
8 import Documents from './pages/Documents';
9 import Icon from './Components/AIAssistant/Icon';

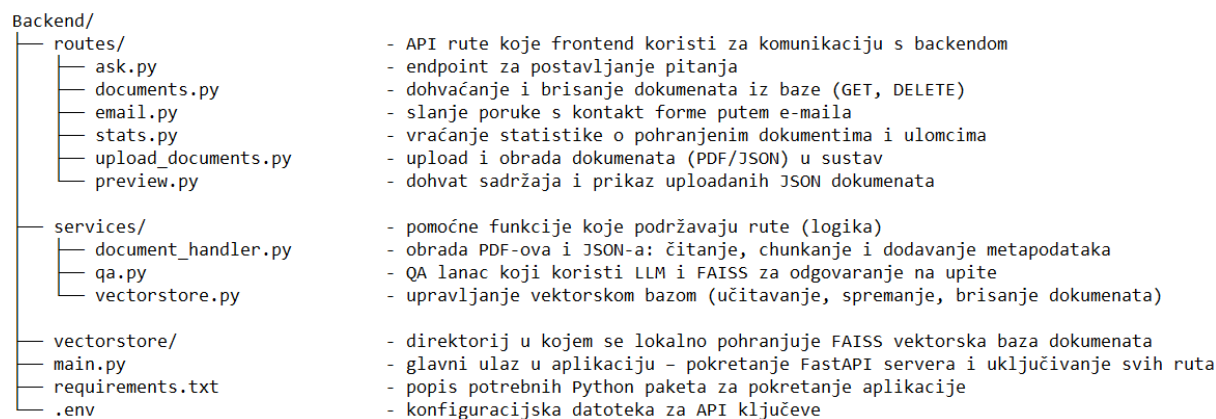
10 const App = () => {
11   return (
12     <LanguageProvider>
13       <BrowserRouter>
14         <Icon />
15         <Routes>
16           <Route index element={<Home />} />
17           <Route path='/home' element={<Home />} />
18           <Route path='/about' element={<About />} />
19           <Route path='/cities' element={<Cities />} />
20           <Route path='/question' element={<Question />} />
21           <Route path='/documents' element={<Documents />} />
22         </Routes>
23       </BrowserRouter>
24     </LanguageProvider>
25   );
26 };
27 export default App

```

Isječak koda 18: App.jsx

5.4. Poslužiteljski dio (Backend)

Na slici 24. može se vidjeti struktura backend dijela. Prikazani su svi dijelovi koji su od značaja za backend dio aplikacije.



Slika 24: Struktura backend dijela

Env datoteka sadrži konfiguracijske varijable koje se koriste u aplikaciji. Promjenjive vrijednosti uključuju API ključeve za OpenAI i Groq modele (OPENAI API KEY, GROQ API KEY) kao i HF API KEY za Hugging Face modele. Za slanje poruka putem kontakt forme koriste se Gmail podaci: Gmail user je adresa s koje se poruke šalju, Gmail password je aplikacijska lozinka za autentifikaciju i Gmail receiver je adresa primatelja. Model koji koristi Hugging Face definiran je kroz varijablu HUGGINGFACEHUB EMBEDDINGS MODEL NAME.

```
1 OPENAI_API_KEY=  
2 GROQ_API_KEY=  
3  
4 GMAIL_USER=  
5 GMAIL_PASSWORD=  
6 GMAIL_RECEIVER=  
7  
8 HF_API_KEY=  
9 HUGGINGFACEHUB_EMBEDDINGS_MODEL_NAME=
```

Slika 25: Sadržaj env datoteke

5.4.1. Poslužitelj

Isječak koda 19. predstavlja glavnu datoteku koja pokreće FastAPI aplikaciju. Učitava se konfiguracija iz .env datoteke, omogućuje se CORS pristup kako bi frontend i backend mogli komunicirati bez ograničenja, te se u aplikaciju uključuju sve definirane rute za pitanja, dokumente, statistiku i email.

```
1 from fastapi import FastAPI  
2 from fastapi.middleware.cors import CORSMiddleware  
3 from dotenv import load_dotenv  
  
4 from routes import ask, documents, stats, email  
  
5 load_dotenv()  
  
6 app = FastAPI()  
  
7 app.add_middleware(  
8     CORSMiddleware,  
9     allow_origins=["*"],  
10    allow_credentials=True,  
11    allow_methods=["*"],  
12    allow_headers=["*"],  
13 )  
  
14 app.include_router(ask.router)  
15 app.include_router(documents.router)  
16 app.include_router(stats.router)  
17 app.include_router(email.router)
```

Isječak koda 19: Main.py

5.4.2. Servisi

Isječak koda 20. upravlja vektorskom bazom podataka koristeći FAISS i HuggingFaceEndpointEmbeddings. Omogućuje učitavanje vektorske baze podataka, dodavanje novih dokumenata te brisanje dokumenata na temelju imena datoteke. Vektorska baza podataka se lokalno pohranjuje u direktorij vectorstore. Za početak se dohvaća API ključ iz .env datoteke (HF_API_KEY), ako ključ ne postoji vraća se greška. Nakon toga se vraća instanca HuggingFaceEndpointEmbeddings za model sentence-transformers/all-MiniLM-L6-v2 (koristi se u svim ostalim funkcijama gdje je potrebno raditi s vektorima).

```

1 import os
2 from langchain_community.vectorstores import FAISS
3 from langchain_huggingface import HuggingFaceEndpointEmbeddings
4 from dotenv import load_dotenv

5 load_dotenv()

6 VECTORSTORE_DIR = "vectorstore"

7 def get_embeddings():
8     api_key = os.getenv("HF_API_KEY")

9     if not api_key:
10         raise ValueError("HF_API_KEY is not set in the environment.")

11     return HuggingFaceEndpointEmbeddings(
12         model="sentence-transformers/all-MiniLM-L6-v2",
13         huggingfacehub_api_token=api_key
14     )

```

Isječak koda 20: Vectorstore.py 1/4

Nadalje provjerava se postoji li direktorij vectorstore, ako ne postoji, vraća se `FileNotFoundError`. Učitava se FAISS baza pomoću prethodno definiranih embeddinga (koristi se kada se želi čitati postojeće dokumente)

```

1 def load_vectorstore():
2     if not os.path.exists(VECTORSTORE_DIR):
3         raise FileNotFoundError("Vectorstore folder not found.")

4     embeddings = get_embeddings()
5     db = FAISS.load_local(VECTORSTORE_DIR, embeddings,
6         ↪ allow_dangerous_deserialization=True)
7     return db

```

Isječak koda 21: Vectorstore.py 2/4

Isječak koda 22. dohvaća embeddinge i ako direktorij vectorstore postoji: učitava bazu, dodaje dokumente ako ih ima, i sprema ažuriranu bazu. Ako baza ne postoji a ako ima dokumenata, kreira se nova baza i sprema se (koristi se kod prvog unosa dokumenta).

```

1 def get_or_create_vectorstore(all_docs):
2     embeddings = get_embeddings()

3     if os.path.exists(VECTORSTORE_DIR):
4         db = FAISS.load_local(VECTORSTORE_DIR, embeddings,
5             ↪ allow_dangerous_deserialization=True)
6         if all_docs:
7             print(f"Adding {len(all_docs)} documents...")
8             db.add_documents(all_docs)
9             db.save_local(VECTORSTORE_DIR)
10            print(f"Updated vectorstore. Total docs: {len(db.docstore._dict)}")
11        elif all_docs:
12            print(f"Creating vectorstore with {len(all_docs)} documents...")
13            db = FAISS.from_documents(all_docs, embeddings)
14            db.save_local(VECTORSTORE_DIR)
15            print("Vectorstore created and saved.")
16        else:

```

```

16         raise ValueError("No documents to index and no vectorstore exists.")
17     return db

```

Isječak koda 22: Vectorstore.py 3/4

Isječak koda 23. učitava vektorsku bazu podataka nakon čega filtrira sve dokumente čiji source nije jednak filename. Ako se broj dokumenata nije promijenio to znači da taj dokument nije postojao. Inače se kreira nova baza samo s preostalim dokumentima, sprema se i vraća True (koristi se kada korisnik želi ukloniti određeni dokument iz baze).

```

1 def delete_file_from_vectorstore(filename):
2     try:
3         db = load_vectorstore()
4     except FileNotFoundError:
5         print("No vectorstore to delete from.")
6         return False
7
8     original_count = len(db.docstore._dict)
9     remaining_docs = [
10         doc for doc in db.docstore._dict.values()
11         if doc.metadata.get("source") != filename
12     ]
13
14     if len(remaining_docs) == original_count:
15         print(f"No document found with source: {filename}")
16         return False
17
18     print(f"Deleting document: {filename}")
19     embeddings = get_embeddings()
20     new_db = FAISS.from_documents(remaining_docs, embeddings)
21     new_db.save_local(VECTORSTORE_DIR)
22     print("Vectorstore updated after deletion.")
23     return True

```

Isječak koda 23: Vectorstore.py 4/4

Memory store predstavlja globalni riječnik u kojem se za svaki korisnički session id pamti prethodna komunikacija (memorija razgovora). Samim time se omogućuje dati kontekst odgovorima jer se zna što je prethodno rečeno. U funkciji get conversation chain inicijalizira se LLM iz GROQ API-ja, model koristi llama-3.1-8b-instant za brze odgovore i ključ se učitava iz .env datoteke. Nadalje se inicijalizira memorija po korisniku i ako memorija za dani session id još ne postoji, kreira se nova. Samim time je omogućeno vođenje dijaloga s kontinuitetom odnosno ne počinje iznova pri svakom pitanju. Nadalje ako postoje dokumenti u FAISS bazi, pretvaraju se u Retriever koji dohvaća najrelevantnije dijelove. Spaja se jezični model, pretraga i memorija u jedan lanac (ConversationalRetrievalChain). Za kraj, ako vektorska baza podataka nije dostupna, koristi se obični lanac samo s memorijom i modelom (ConversationChain).

```

1 from langchain.memory import ConversationBufferMemory
2 from langchain.chains import ConversationalRetrievalChain, ConversationChain
3 from langchain_groq import ChatGroq
4 from langchain_community.vectorstores import FAISS
5 import os

```

```

6 memory_store = {}

7 def get_conversation_chain(db: FAISS, session_id: str):
8     llm = ChatGroq(
9         model="llama-3.1-8b-instant",
10        api_key=os.getenv("GROQ_API_KEY")
11    )

12    if session_id not in memory_store:
13        memory_store[session_id] = ConversationBufferMemory(
14            memory_key="chat_history",
15            return_messages=True
16        )
17    memory = memory_store[session_id]

18    if db and db.docstore._dict:
19        retriever = db.as_retriever(search_kwargs={"k": 3})
20        return ConversationalRetrievalChain.from_llm(
21            llm=llm,
22            retriever=retriever,
23            memory=memory,
24            verbose=True
25        )
26    else:
27        return ConversationChain(
28            llm=llm,
29            memory=memory,
30            verbose=True
31        )

```

Isječak koda 24: qa.py

Kroz isječak koda 25. prvo se provjeravaju jesu li datoteke poslane nakon čega se kreiraju privremene mape za pohranu dokumenata gdje će se dokumenti privremeno spremili lokalno prije obrade. Nakon toga iterira se kroz sve datoteke i gdje se svaki dokument privremeno zapisuje na disk (sadržaj se čita asinkrono). Zatim se provjerava tip dokumenta i ako je pdf, koristi se `PyPDFLoader` iz `LangChain`-a za učitavanje sadržaja kao `Document`. Ako je .json, parsira se kao JSON objekt i pretvara u tekst koji se spremi kao jedan `Document`. Nastavno se svakom dokumentu dodaje source i točan UTC datum i vrijeme slanja (metapodaci). Za kraj, sadržaj se dijeli u dijelove od 1000 znakova, s preklapanjem od 200 znakova. Time se čuva kontekst između dijelova.

```

1 import os
2 import tempfile
3 import json
4 from datetime import datetime
5 from fastapi import UploadFile
6 from langchain_community.document_loaders import PyPDFLoader
7 from langchain.docstore.document import Document
8 from langchain.text_splitter import RecursiveCharacterTextSplitter

9 async def process_documents(files: list[UploadFile]):
10     if not files:
11         return []

12     temp_dir = tempfile.TemporaryDirectory()
13     all_docs = []

14     for file in files:

```

```

15     temp_path = os.path.join(temp_dir.name, file.filename)
16     with open(temp_path, "wb") as f:
17         f.write(await file.read())

18     if file.filename.lower().endswith(".pdf"):
19         loader = PyPDFLoader(temp_path)
20         docs = loader.load()

21     elif file.filename.lower().endswith(".json"):
22         try:
23             with open(temp_path, "r", encoding="utf-8") as f:
24                 json_data = json.load(f)

25                 text = json.dumps(json_data, indent=2, ensure_ascii=False)
26                 docs = [Document(page_content=text, metadata={"source":
27                     ↪ file.filename})]
28             except Exception as e:
29                 print(f"Greška prilikom parsiranja JSON-a: {file.filename} -- {e}")
30                 continue

31     else:
32         print(f"Preskačem nepodržani format: {file.filename}")
33         continue

34     for doc in docs:
35         doc.metadata["source"] = file.filename
36         doc.metadata["uploaded_at"] = datetime.utcnow().isoformat()

37     splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
38         ↪ chunk_overlap=200)
39     split_docs = splitter.split_documents(docs)
40     all_docs.extend(split_docs)

41     return all_docs

```

Isječak koda 25: Document_handler.py

5.4.3. Ruta

API ruta `/ask` omogućuje korisniku da pošalje pitanje i jedinstveni ID sesije, a zatim dohvaća odgovor koristeći vektorsku bazu podataka i LLM model. Prvo se pokušava učitati lokalna vektorska baza, ako ne postoji, vraća se obavijest da dokumenti još nisu učitani. Ako je sve u redu, poziva se `ConversationalRetrievalChain` koji koristi kontekst prethodnog razgovora i vraća najrelevantniji odgovor. U slučaju greške, vraća se poruka s odgovarajućim statusom.

```

1  from fastapi import APIRouter, Form
2  from fastapi.responses import JSONResponse
3  from services.vectorstore import load_vectorstore
4  from services.qa import get_conversation_chain
5  import traceback

6  router = APIRouter()

7  @router.post("/ask")
8  async def ask_document(
9      question: str = Form(...),
10     session_id: str = Form(...)
11 ):

```



```

12     try:
13         db = load_vectorstore()
14     except FileNotFoundError:
15         return {"answer": "No documents are loaded yet. Please upload documents"}
16     except Exception as e:
17         print("Error loading vectorstore:", str(e))
18         traceback.print_exc()
19         return JSONResponse(status_code=500, content={"answer": "Error loading vectorstore."})

20     try:
21         qa_chain = get_conversation_chain(db, session_id)
22         response = qa_chain.invoke({"question": question})

23         print("== CHAT MEMORY ==")
24         for msg in qa_chain.memory.chat_memory.messages:
25             role = msg.type
26             print(f"{role.upper()}: {msg.content}")

27         answer = response["answer"] if isinstance(response, dict) else response

28         return {
29             "answer": answer
30         }

31     except Exception as e:
32         print("Error during QA invocation:", str(e))
33         traceback.print_exc()
34         return JSONResponse(status_code=500, content={
35             "answer": "Sorry, there was an error processing your question. Please try again later."
36         })

```

Isječak koda 26: ask.py

Nadalje su definirane dvije rute: jedna za dohvat popisa svih PDF dokumenata koji su trenutno indeksirani u vektorskoj bazi podataka (/documents), a druga za brisanje dokumenata po nazivu dokumenta (/delete-document). Ruta za brisanje vraća grešku ako dokument s tim imenom ne postoji u vektorskoj bazi podataka.

```

1  from fastapi import APIRouter, HTTPException
2  from services.vectorstore import load_vectorstore, delete_file_from_vectorstore
3  router = APIRouter()
4  @router.get("/documents")
5  def list_documents():
6      db = load_vectorstore()
7      sources = set()
8      for doc in db.docstore._dict.values():
9          if "source" in doc.metadata:
10             sources.add(doc.metadata["source"])
11      return {"documents": list(sources)}

12  @router.delete("/delete-document")
13  def delete_document(filename: str):
14      success = delete_file_from_vectorstore(filename)
15      if not success:
16          raise HTTPException(status_code=404, detail=f"No document found with source: {filename}")
17      return {"status": "success", "message": f"Deleted document: {filename}"}

```

Isječak koda 27: documents.py

API ruta `/send-email` omogućuje slanje e-mail poruke s kontakt forme. Korisnik ispunjava ime, email, broj telefona, adresu i poruku, a backend zatim šalje e-mail na zadanu adresu koristeći Gmail SMTP. Podaci se šalju putem SSL-a, a konfiguracija dolazi iz `.env` datoteke.

```
1 from fastapi import APIRouter
2 from pydantic import BaseModel, EmailStr
3 import smtplib
4 from email.mime.text import MIMEText
5 from email.utils import formataddr
6 from dotenv import load_dotenv
7 import os
8
9 load_dotenv()
10
11 router = APIRouter()
12
13 class ContactForm(BaseModel):
14     fullName: str
15     phone: str
16     email: EmailStr
17     address: str
18     description: str
19
20 @router.post("/send-email")
21 def send_email(data: ContactForm):
22     sender_email = os.getenv("GMAIL_USER")
23     sender_password = os.getenv("GMAIL_PASSWORD")
24     receiver_email = os.getenv("GMAIL_RECEIVER") or sender_email
25
26     msg_content = f"""
27     Nova poruka s kontakt forme:
28
29     Ime i prezime: {data.fullName}
30     Telefon: {data.phone}
31     E-mail: {data.email}
32     Adresa: {data.address}
33     Poruka:
34     {data.description}
35     """
36
37     msg = MIMEText(msg_content, "plain", "utf-8")
38     msg["Subject"] = "Nova poruka s web-stranice"
39     msg["From"] = formataddr(("Web kontakt forma", sender_email))
40     msg["To"] = receiver_email
41     msg["Reply-To"] = data.email
42
43     try:
44         with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
45             server.login(sender_email, sender_password)
46             server.sendmail(sender_email, receiver_email, msg.as_string())
47         return {"message": "Email je uspješno poslan."}
48     except Exception as e:
49         print("Greška prilikom slanja emaila:", e)
50         return {"error": "Neuspješno slanje emaila. Provjerite postavke."}
```

Isječak koda 28: email.py

Ruta `/preview-document` omogućuje prikaz pregleda sadržaja `.json` datoteke s diska. Prvo provjerava postoji li tražena datoteka unutar mape `Scraper/data`, zatim provjerava

je li to .json datoteka. Ako sve prođe, učitava prvih 10 stavki (ako je riječ o listi) ili prvih 10 ključeva (ako je riječ o riječniku) i vraća ih kao preview. U slučaju greške (nepostojeća datoteka ili pogrešan format) vraća se odgovarajuća HTTP greška.

```
1 from fastapi import APIRouter, Query
2 from fastapi.responses import JSONResponse
3 import os
4 import json
5
6 router = APIRouter()
7
8 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
9 DOCUMENTS_DIR = os.path.abspath(os.path.join(BASE_DIR, "../Scraper/data"))
10
11 @router.get("/preview-document")
12 async def preview_document(filename: str = Query(...)):
13     try:
14         filepath = os.path.join(DOCUMENTS_DIR, filename)
15         if not os.path.isfile(filepath):
16             return JSONResponse(status_code=404, content={"error": "File not
17                 ↳ found."})
18
19         if not filename.endswith(".json"):
20             return JSONResponse(status_code=400, content={"error": "Only JSON
21                 ↳ preview is supported."})
22
23         with open(filepath, "r", encoding="utf-8") as f:
24             data = json.load(f)
25
26         preview = data
27         if isinstance(data, list):
28             preview = data[:10]
29         elif isinstance(data, dict):
30             preview = {k: data[k] for k in list(data)[:10]}
31
32         return {"preview": preview}
33
34     except Exception as e:
35         return JSONResponse(status_code=500, content={"error": str(e)})
```

Isječak koda 29: Preview.py

API ruta `/stats` vraća statistiku o trenutno učitanim dokumentima u bazi. Prvo dohvaća sve dokumente i filtrira one koji imaju metapodatke o vremenu učitavanja i izvoru. Ako postoji barem jedan takav dokument, određuje se zadnje učitani dokument prema vremenu. Također, se izračunava ukupan broj dokumenata (po različitim source vrijednostima) i broj svih chunkova. Rezultat se vraća kao JSON s tri informacije: broj dokumenta, broj chunkova i naziv posljednje dodane datoteke.

```
1 from fastapi import APIRouter
2 from services.vectorstore import load_vectorstore
3 from datetime import datetime
4
5 router = APIRouter()
6
7 @router.get("/stats")
8 def get_stats():
9     db = load_vectorstore()
```

```

8 docs = list(db.docstore._dict.values())

9 docs_with_timestamps = [
10     doc for doc in docs if "uploaded_at" in doc.metadata and "source" in
        ↳ doc.metadata
11 ]

12 if docs_with_timestamps:
13     docs_sorted = sorted(docs_with_timestamps, key=lambda d:
        ↳ d.metadata["uploaded_at"])
14     last_added = docs_sorted[-1].metadata["source"]
15 else:
16     last_added = "--"

17 sources = {doc.metadata.get("source") for doc in docs if "source" in
        ↳ doc.metadata}

18 return {
19     "total_documents": len(sources),
20     "total_chunks": len(docs),
21     "last_added": last_added
22 }

```

Isječak koda 30: Stats.py

Zadnja API ruta `/upload-documents` omogućuje korisnicima da uploadaju PDF ili JSON dokumente putem forme. Primljeni dokumenti se obrađuju funkcije `process documents`, gdje se dijele na manje dijelove (chunkove). Ako su dokumenti uspješno obrađeni, poziva se `get or create vectorstore` kako bi se dokumenti dodali u vektorsku bazu. Na kraju se vraća poruka s brojem uspješno indeksiranih dijelova.

```

1 from fastapi import APIRouter, UploadFile, File, Form
2 from services.document_handler import process_documents
3 from services.vectorstore import get_or_create_vectorstore

4 router = APIRouter()

5 @router.post("/upload-documents")
6 async def upload_documents(
7     session_id: str = Form(...),
8     pdfs: list[UploadFile] = File(...)
9 ):
10     all_docs = await process_documents(pdfs)

11     if not all_docs:
12         return {"message": "No valid documents processed."}

13     get_or_create_vectorstore(all_docs)
14     return {"message": f"Successfully uploaded and indexed {len(all_docs)} chunks."}

```

Isječak koda 31: Upload.py

5.5. Prikaz rada aplikacije

Projekt je razvijan korištenjem Vite okoline, a pokretanje aplikacije omogućeno je korištenjem naredbe `npm run dev`.

Backend se pokreće lokalno pomoću Uvicorn ASGI servera, korištenjem naredbe `uvicorn main:app --reload` koja omogućuje automatsko ponovno učitavanje prilikom promjena u kodu.

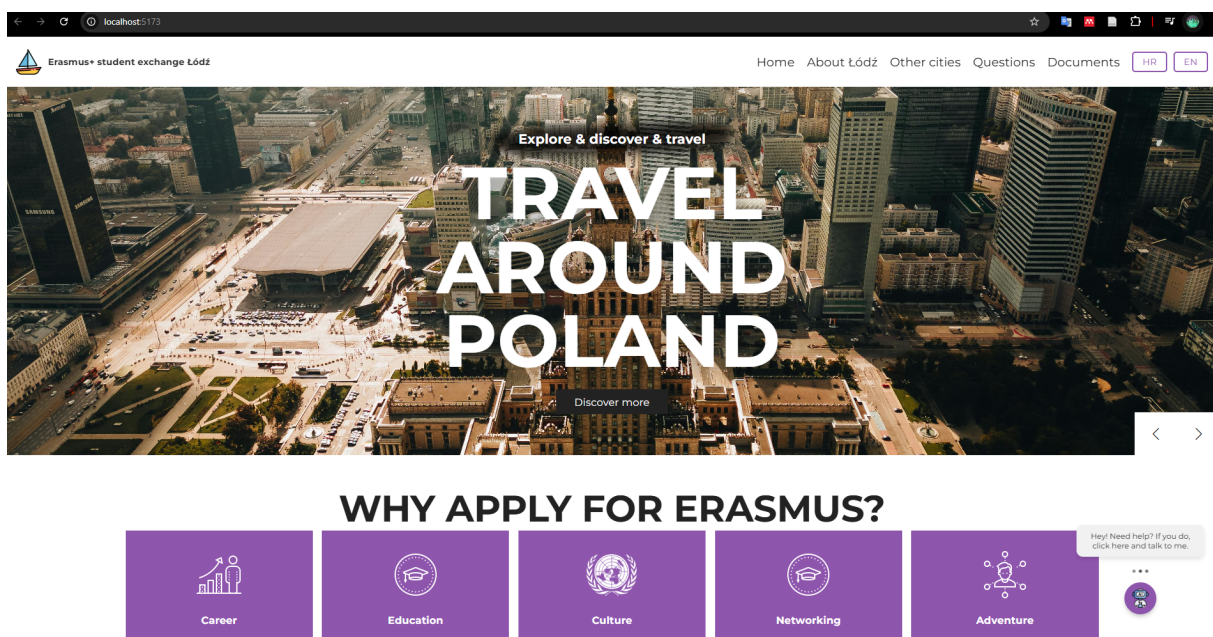
```
PS C:\Users\aleks\Radna površina\Website with a chatbot\Backend> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\aleks\\Radna površina\\Website with a chatbot\\Backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [10076] using StatReload
INFO: Started server process [17668]
INFO: Waiting for application startup.
INFO: Application startup complete.

PS C:\Users\aleks\Radna površina\Website with a chatbot\Frontend> npm run dev
> test@0.0.0 dev
> vite

VITE v6.2.2 ready in 329 ms
  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

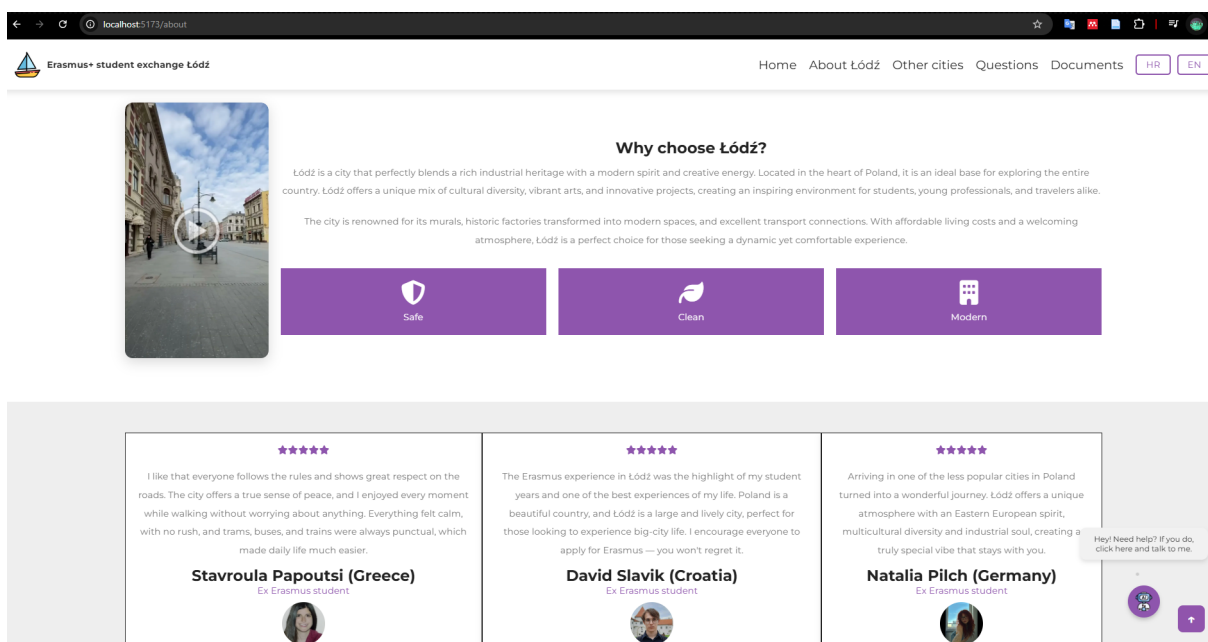
Slika 26: Pokretanje aplikacije kroz terminal

Slika 27. prikazuje početnu stranicu aplikacije na koju se dolazi nakon pokretanja aplikacije. Na početnoj stranici nalazi se nekoliko komponenta kao što su slider, motivacija, kratko o Łódźu i drugi gradovi. Ideja početne stranice je da korisniku pruži osnovne informacije i odvede korisnika na ostale stranice aplikacije.



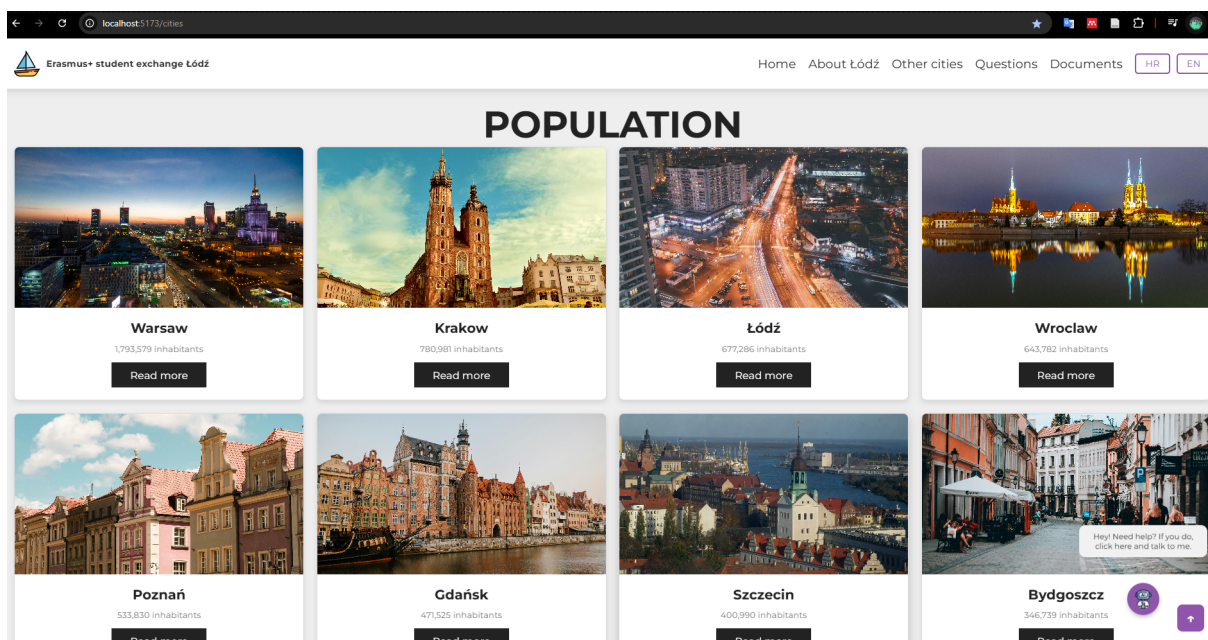
Slika 27: Početna stranica aplikacije

Na slici 28. može se vidjeti stranica O Łódźu koja prikazuje dodatne informacije o gradu, uključujući iskustva bivših studenata koji su proveli Erasmus+ razmjenu u tom gradu.



Slika 28: O Łódźu stranica

Na slici 29. može se vidjeti stranica Drugi gradovi koja je namijenjena za pregled ostalih gradova u Poljskoj koje je moguće posjetiti. Ova stranica pruža korisnicima informacije u obliku slika i podataka kao što je broj stanovnika.



Slika 29: Drugi gradovi stranica

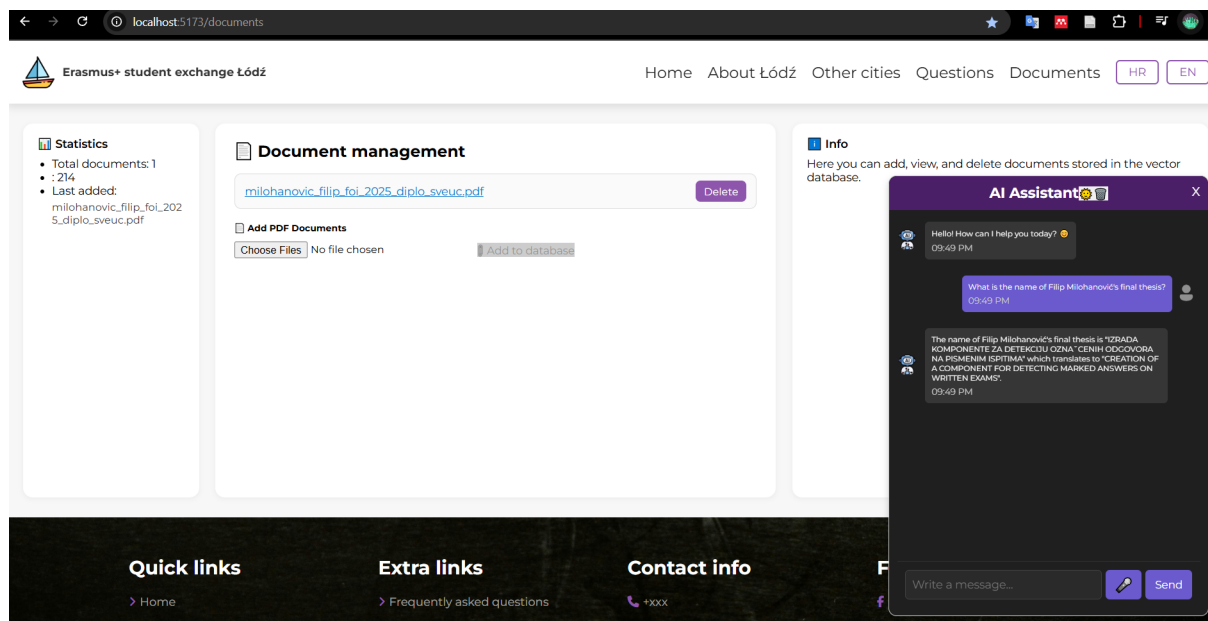
Na slici 30. može se vidjeti stranica **Pitanja** koja je namijenjena za postavljanje pitanja od strane korisnika u slučaju da AI asistent ne pruži odgovor koji korisnik traži. Nakon slanja odgovora, vlasnik stranice dobiva email s informacijama prema definiranoj formi.

Slika 30: Forma za postavljanje pitanja stranica

Na slici 31. može se vidjeti stranica **Dokumenti** koja omogućuje pregled i upravljanje dokumentima koji se koriste za rad s asistentom. Nakon što je započet rad s AI dijelom, pojavila se potreba za dodavanjem ove stranice kako bi se bolje vizualiziralo što se događa u pozadini rada asistenta.

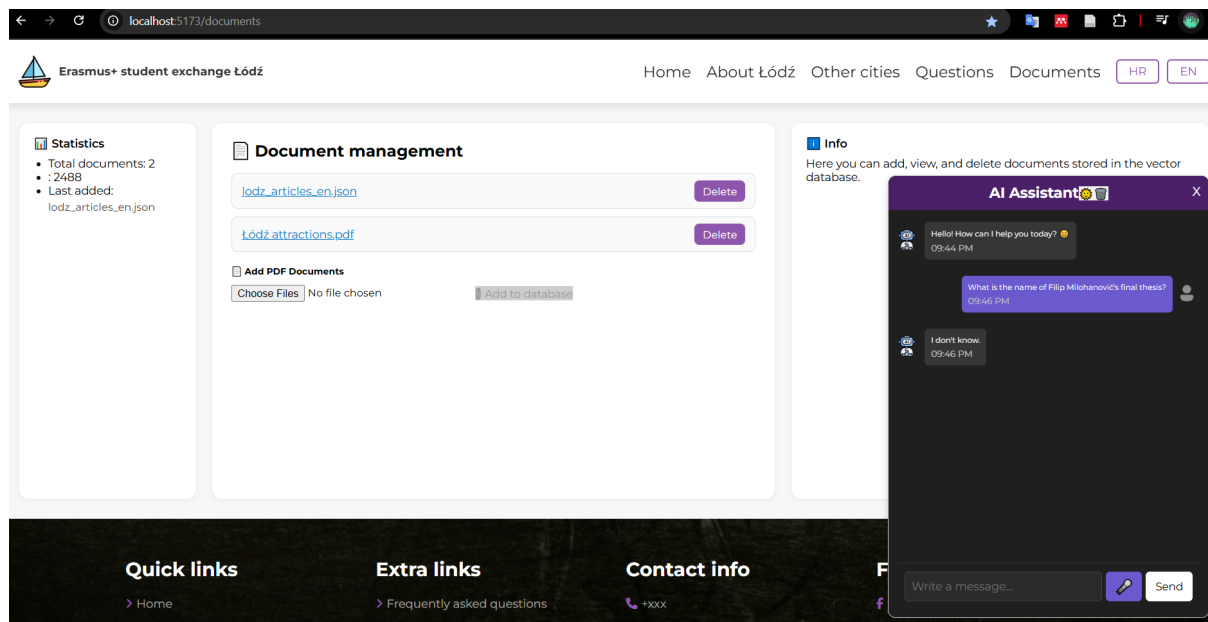
Slika 31: Stranica za pregled i upravljanje dokumentima

Na slici 32. može se vidjeti kako funkcionira AI asistent kada je dokument u bazi podataka. Asistent u tom slučaju može odgovoriti na pitanje korisnika koristeći informacije iz dokumenta.



Slika 32: Testiranje asistenta kada je dokument u bazi

Na slici 33. može se vidjeti kako AI asistent reagira kada dokument nije pronađen u bazi podataka.



Slika 33: Testiranje asistenta kad dokument nije u bazi

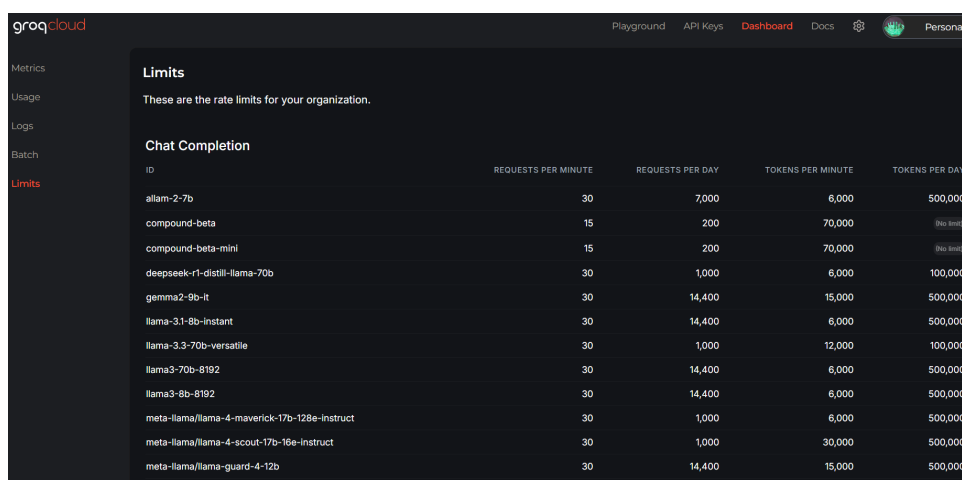
5.6. Praćenje performansi i ograničenja korištenih alata

Na slici 34. je prikazan dashboard s Groq platforme, konkretno za model llama-3.1-8b-instant. Prikazuju se grafovi s brojem zahtjeva i ukupnim brojem tokena u određenom vremenskom okviru. Ovo omogućuje korisnicima praćenje iskorištenosti modela u realnom vremenu te detekciju prekoračenja limita koji su označeni crvenim crtama. Prikazani vrhovi na grafu označavaju trenutke kada su slani upiti prema modelu. Osim toga, moguće je pregledati i dnevnu/mjesečnu potrošnju u smislu financijskih sredstava. Isto tako kroz karticu Logs omogućen je detaljniji uvid u zahtjeve koji su stigli na Groq (kada je kreiran, koji model je korišten, koji API key, status code, koliko input tokena i output je potrošeno i od kog usera).



Slika 34: Groq platforma - pregled zahtjeva i potrošenih tokena; Izvor

Slika 35. prikazuje tablicu s ograničenjima korištenja za različite modele unutar Groq platforme. Za svaki model prikazani su maksimalni brojevi upita u minuti i danu, kao i ograničenja na broj tokena. Na primjer, model llamallama-3.1-8b-instant dopušta 30 zahtjeva u minuti, 14400 dnevno, i do 500000 tokena mjesečno. Ova tablica jasno prikazuje tehnička ograničenja koja korisnici moraju imati na umu prilikom dizajniranja aplikacija koje koriste LLM modele u oblaku.



ID	REQUESTS PER MINUTE	REQUESTS PER DAY	TOKENS PER MINUTE	TOKENS PER DAY
allam-2-7b	30	7,000	6,000	500,000
compound-beta	15	200	70,000	(No limit)
compound-beta-mini	15	200	70,000	(No limit)
deepseek-r1-distill-llama-70b	30	1,000	6,000	100,000
gemma2-9b-it	30	14,400	15,000	500,000
llama-3.1-8b-instant	30	14,400	6,000	500,000
llama-3.3-70b-versatile	30	1,000	12,000	100,000
llama3-70b-8192	30	14,400	6,000	500,000
llama3-8b-8192	30	14,400	6,000	500,000
meta-llama/llama-4-maverick-17b-128e-instruct	30	1,000	6,000	500,000
meta-llama/llama-4-scout-17b-16e-instruct	30	1,000	30,000	500,000
meta-llama/llama-guard-4-12b	30	14,400	15,000	500,000

Slika 35: Groq limitacije; Izvor

6. Zaključak

Rezultat ovog rada je web-aplikacija u koju je integriran AI asistent temeljen na RAG tehnologiji. U uvodnom dijelu rada naglašena je važnost studiranja, kao i brojni benefiti koje ono donosi. Poseban naglasak stavljen je na Erasmus+ studentsku razmjenu te izazove i strahove s kojima se studenti često suočavaju pri odluci o odlasku u inozemstvo. Među najčešćim izazovima ističu se: odlazak iz doma i obitelji, financijska neodgovornost, jezična barijera, prilagodba akademskom životu te organizacija i pakiranje. Glavna motivacija za izradu praktičnog dijela ovog rada bila je upravo vlastito iskustvo sudjelovanja u Erasmus+ programu, s ciljem motiviranja budućih studenata da se odvaži na isti korak.

Nadalje, rad se dotiče pojma inteligentnih agenata, koji danas predstavljaju aktualan i sve prisutniji oblik pružanja informacija. Napravljena je klasifikacija AI sustava s obzirom na način korištenja i primjenu: kopiloti, asistenti, chatbot platforme i autonomni agenti. Također je kratko opisana njihova primjena u obrazovnom sustavu te izazovi s kojima se suočavaju u tom kontekstu. Glavno teorijsko poglavlje posvećeno je RAG metodi (Retrieval-Augmented Generation), koja je i temelj razvoja AI asistenta u praktičnom dijelu rada. RAG metoda kombinira dva ključna elementa: retriever i generator. U usporedbi s treniranjem i fine-tuningom modela, predstavlja značajno bolji izbor kada su u pitanju troškovi i zaštita privatnosti podataka. Danas se široko primjenjuje u sustavima za otkrivanje prijevара i procjene rizika, preporučiteljskim sustavima za e-trgovinu, sustavima za učenje i brojnim drugim područjima.

Jedan od izazova RAG sustava su halucinacije, tj. obmanjujući odgovori koje ova metoda nastoji smanjiti, ali ih u potpunosti ne eliminira, čak i kada se koristi vanjski izvor znanja. Također, problem predstavlja i upravljanje višestrukim izvorima podataka, gdje model mora učinkovito kombinirati informacije iz različitih baza kako bi generirao što točniji odgovor. Na kraju teorijskog dijela napravljen je pregled dostupnih biblioteka i tehnologija koje se koriste pri izradi RAG sustava.

Za praktičnu implementaciju AI asistenta odabrani su:

- LangChain i FAISS vektorska baza podataka,
- Llama model preko platforme Groq,
- Hugging Face model za embedanje teksta,
- React (Vanilla + JavaScript) za izradu korisničkog sučelja,
- SMTP protokol za slanje podataka putem kontakt forme,
- MiKTeX za izradu dokumentacije (lokalno postavljen unutar VS Code-a).

Dodatno je razvijen i alat za prikupljanje podataka s web-stranica, čiji se sadržaj koristi kao referenca za odgovore AI asistenta. Proces dodavanja dokumenata u bazu obuhvaća: čitanje dokumenta, dijeljenje teksta na manje dijelove (chunkove), pretvaranje u vektore te spremanje u FAISS bazu. Kada korisnik postavi pitanje, ono se također pretvara u vektor, a

retriever pronalazi najrelevantnije chunkove koji se proslijeđuju LLM-u za generiranje konačnog odgovora. Glavni izazov aplikacije trenutno je ograničeno korištenje AI asistenta zbog dnevnih/-mjesečnih limita besplatnih upita. Nakon što se ti limiti premaše, potrebno je plaćati dodatne upite. Kao moguće rješenje, model bi se mogao samostalno hostati te izvoditi lokalno ili na poslužitelju, neovisno o vanjskim platformama.

U slučaju nastavka rada moguće bi bilo:

- razviti drugačiji koncept stranice s admin dashboardom kroz koji bi se mogla objavljivati različita Erasmus iskustva,
- izraditi osnovni kostur stranice za daljnji razvoj,
- posvetiti posebnu pažnju sigurnosti aplikacije prije produkcijskog okruženja, s naglaskom na zaštitu kontakt forme od automatiziranih botova i spam poruka.

Ideja ove aplikacije, zajedno s AI asistentom, bila je prvenstveno pomoći budućim studentima koji će se odlučiti za Erasmus destinaciju u Łódźu. Polazim od vlastitog iskustva jer ja takav oblik podrške nisam imao, a kroz ovu aplikaciju studenti mogu na jednom mjestu pronaći korisne i karakteristične informacije vezane uz grad, druge lokacije u državi, dodatnu pomoć putem kontakt forme ili pak podršku AI asistenta koji se oslanja na provjerene i relevantne izvore.

Izrada ovog rada bila je izrazito zanimljiva i korisna jer sam imao priliku raditi s novim i modernim tehnologijama. Posebno iskustvo bilo je razviti vlastitog AI asistenta temeljenog na RAG tehnologiji te povezati osobno Erasmus iskustvo s praktičnom implementacijom u obliku web-aplikacije. Za kraj, Erasmus se ne treba bojati jer je to prekasno iskustvo koje se pamti do kraja života. Svaki problem s kojim se suoči osoba, čini osobu jačom i neovisnijom. Boravak na jednom mjestu dulje vrijeme postat će čudan način života. Jednom Erasmus+, uvijek Erasmus+!

Poveznica na GitHub repozitorij ovog diplomskog rada: **Poveznica**

Popis literature

- [1] Digital 360. „Are studies important in life? If yes, then why?” Pristupljeno: 10. svibanj 2025. adresa: <https://digital360india.medium.com/are-studies-important-in-life-if-yes-then-why-4583029b95e3>.
- [2] K. Bisio. „15 Reasons Why College is Important for Your Career.” Pristupljeno: 11. svibanj 2025. adresa: <https://www.msudenver.edu/15-reasons-why-college-is-important-for-your-career/>.
- [3] „What is Erasmus+? - Erasmus+.” Pristupljeno: 27. svibanj 2025. adresa: <https://erasmus-plus.ec.europa.eu/about-erasmus/what-is-erasmus>.
- [4] „Data on Erasmus+ in Croatia in 2023 - Erasmus+.” Pristupljeno: 10. svibanj 2025. adresa: <https://erasmus-plus.ec.europa.eu/factsheets/2023/croatia>.
- [5] „Studying abroad - Erasmus+.” Pristupljeno: 10. svibanj 2025. adresa: <https://erasmus-plus.ec.europa.eu/opportunities/opportunities-for-individuals/students/studying-abroad>.
- [6] Erasmus Generation Blog. „Six common Erasmus fears and how to deal with them.” Pristupljeno: 22. svibanj 2025. adresa: <https://blog.erasmusgeneration.org/six-common-erasmus-fears-and-how-deal-them>.
- [7] Erasmus Student Network AISBL. „Erasmus Careers - Feeling homesick.” Video. Objavljeno: 15. listopada 2024. Pristupljeno: 22. svibanj 2025. adresa: https://www.youtube.com/watch?v=TT__3yOkAdk.
- [8] J. Buchan. „Why you should add an AI assistant to your app or website.” Pristupljeno: 06. svibanj 2025., Zudu. adresa: <https://zudu.co.uk/blog/why-you-should-add-an-ai-assistant-to-your-app-or-website/>.
- [9] S. J. Russell i P. Norvig, *Artificial Intelligence: A Modern Approach*, 4. izdanje. Pearson Education Limited, 2022.
- [10] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2. izdanje. John Wiley & Sons, 2009.
- [11] c2061348. „The difference between ChatBot, AI Assistant, Copilot, and AI Agent.” Pristupljeno: 10. svibanj 2025. adresa: <https://exomindset.co/the-difference-between-chatbot-ai-assistant-copilot-and-ai-agent/>.
- [12] E. Mixon. „What is Siri? | Definition from TechTarget.” Pristupljeno: 20. lipanj 2025. adresa: <https://www.techtarget.com/searchmobilecomputing/definition/Siri>.

- [13] „Create Virtual Agents with Copilot | Microsoft Copilot Studio.” Pristupljeno: 27. svibanj 2025. adresa: <https://www.microsoft.com/en-us/copilot/microsoft-copilot-studio>.
- [14] iaanw. „Quotas and limits - Microsoft Copilot Studio.” Pristupljeno: 27. svibanj 2025. adresa: <https://learn.microsoft.com/en-us/microsoft-copilot-studio/requirements-quotas>.
- [15] „Introducing Operator.” Pristupljeno: 10. svibanj 2025. adresa: <https://openai.com/index/introducing-operator/>.
- [16] Riley Brown. „TikTok video objavljen 24. siječnja 2025.” [Online video]. Pristupljeno: 4. lipanj 2025., TikTok. adresa: <https://www.tiktok.com/@rileybrown.ai/video/7463264903327976734>.
- [17] A. Connell. „50 Critical Chatbot Statistics You Need To Know In 2025.” Accessed online, pogledano 12. srpnja 2025. adresa: <https://adamconnell.me/chatbot-statistics/>.
- [18] „Gradescope | Save time grading.” Pristupljeno: 26. svibanj 2025. adresa: <https://www.gradescope.com/>.
- [19] P. Weber. „SmythOS - Intelligent Agents in Education.” Pristupljeno: 26. svibanj 2025. adresa: <https://smythos.com/ai-industry-solutions/education/intelligent-agents-in-education/>.
- [20] Amazon Web Services, Inc. „What is RAG? - Retrieval-Augmented Generation AI Explained - AWS.” Pristupljeno: 06. svibanj 2025. adresa: <https://aws.amazon.com/what-is/retrieval-augmented-generation/>.
- [21] Marina Danilevsky, *What is Retrieval-Augmented Generation (RAG)?* [Online video]. Pristupljeno: 6. svibanj 2025., IBM Technology, 8. 2023.
- [22] A. R. J. „Planets With The Most Moons.” Accessed: 2025-08-15.
- [23] IBM Research. „What is retrieval-augmented generation (RAG)?” Pristupljeno: 06. svibanj 2025. adresa: <https://research.ibm.com/blog/retrieval-augmented-generation-RAG>.
- [24] R. W. G. Studio) MD (Custom AI. „This history of Retrieval-Augmented Generation in 3 minutes...!” Medium, pogledano 29. svibnja 2025. adresa: https://medium.com/@custom_aistudio/this-history-of-retrieval-augmented-generation-in-3-minutes-f7f07073599a.
- [25] O. Khattab i M. Zaharia, „ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, serija SIGIR '20, New York, NY, USA: Association for Computing Machinery, 7. 2020., str. 39–48. DOI: 10.1145/3397271.3401075.
- [26] W. Zhao, S. Tan i P. Li, „SONG: Approximate Nearest Neighbor Search on GPU,” *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, IEEE, 4. 2020., str. 1033–1044. DOI: 10.1109/ICDE48307.2020.00094.

- [27] L. M. Amugongo, P. Mascheroni, S. Brooks, S. Doering i J. Seidel, „Retrieval augmented generation for large language models in healthcare: A systematic review,” *PLOS Digital Health*, sv. 4, br. 6, e0000877, 6. 2025. DOI: 10.1371/journal.pdig.0000877.
- [28] R. Zhou, „Advanced Embedding Techniques in Multimodal Retrieval Augmented Generation: A Comprehensive Study on Cross Modal AI Applications,” *Journal of Computing and Electronic Information Management*, sv. 13, br. 3, str. 16–22, 7. 2024. DOI: 10.54097/h8wf8vah.
- [29] S. Li, L. Stenzel, C. Eickhoff i S. A. Bahrainian, „Enhancing Retrieval-Augmented Generation: A Study of Best Practices,” 1. 2025. DOI: 10.48550/arXiv.2501.07391. arXiv: 2501.07391 [cs.CL].
- [30] Redis. „Introduction to Retrieval Augmented Generation (RAG).” Redis, pogledano 29. svibnja 2025. adresa: <https://redis.io/glossary/retrieval-augmented-generation/>.
- [31] Shaheryar. „Understanding the Key Components of RAG: Retriever and Generator.” Pristupljeno: 29. svibanj 2025. adresa: <https://dev.to/shaheryaryousaf/understanding-the-key-components-of-rag-retriever-and-generator-1a1j>.
- [32] S. Gupta, R. Ranjan i S. N. Singh, „A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions,” *arXiv preprint arXiv:2410.12837*, 10. 2024., Preuzeto: 3. listopada 2024. DOI: 10.48550/arXiv.2410.12837.
- [33] OpenAI. „GPT-4 Technical Report,” pogledano 26. srpnja 2025. adresa: <https://cdn.openai.com/papers/gpt-4.pdf>.
- [34] Monte Carlo Data. „RAG Vs Fine Tuning: How To Choose The Right Method,” pogledano 26. srpnja 2025. adresa: <https://www.montecarlodata.com/blog-rag-vs-fine-tuning/>.
- [35] A. Siddharth. „A practical guide to RAG implementation: Part I | LinkedIn.” Pristupljeno: 04. lipanj 2025. adresa: <https://www.linkedin.com/pulse/practical-guide-rag-implementation-part-i-siddharth-asthana-bppbe/>.
- [36] Nishtha. „Top 7 RAG Use Cases and Applications to Explore in 2025.” Pristupljeno: 20. lipanj 2025., ProjectPro. adresa: <https://www.projectpro.io/article/rag-use-cases-and-applications/1059>.
- [37] M. M. Cooper i M. W. Klymkowsky, „Let Us Not Squander the Affordances of LLMs for the Sake of Expedience: Using Retrieval Augmented Generative AI Chatbots to Support and Evaluate Student Reasoning,” *Journal of Chemical Education*, sv. 101, br. 11, str. 4847–4856, 11. 2024. DOI: 10.1021/acs.jchemed.4c00765.
- [38] L. Shengjie, „Towards a Robust Retrieval-Based Summarization System,” 2024., Accessed: 2025-08-18. arXiv: 2403.19889 [cs.CL].
- [39] O. K. Gargari i G. Habibi, „Enhancing medical AI with retrieval-augmented generation: A mini narrative review,” *Digital Health*, sv. 11, str. 20552076251337177, 4. 2025. DOI: 10.1177/20552076251337177.

- [40] Z. Liu, P. Agrawal, S. Singhal, V. Madaan, M. Kumar i P. K. Verma, „LPITutor: an LLM based personalized intelligent tutoring system using RAG and prompt engineering,” *PeerJ Computer Science*, sv. 11, e2991, 8. 2025. DOI: 10.7717/peerj-cs.2991.
- [41] G. Singh, P. Singh i P. Singh, „Advanced Real-Time Fraud Detection Using RAG-Based LLMs,” 2025., Accessed: 2025-08-18. arXiv: 2501.15290 [cs.LG].
- [42] A. Sun i dr., „A product-aware query auto-completion framework for e-commerce search via retrieval-augmented generation method,” 2025., Amazon Science, Accessed: 2025-08-18.
- [43] A. Gadde, „AI-enhanced knowledge management systems in enterprises: Transforming organizational intelligence,” *World Journal of Advanced Research and Reviews*, sv. 26, br. 2, str. 1913, 8. 2025. DOI: 10.30574/wjarr.2025.26.2.1913.
- [44] Aim. Labs i P. Anand. „Top Problems with RAG systems and ways to mitigate them - AIMon Labs.” Pristupljeno: 2025-06-04. adresa: https://www.aimon.ai/posts/top%5C_problems%5C_with%5C_rag%5C_systems%5C_and%5C_ways%5C_to%5C_mitigate%5C_them.
- [45] H. Hu, C. He, X. Xie i Q. Zhang, „LRP4RAG: Detecting Hallucinations in Retrieval-Augmented Generation via Layer-wise Relevance Propagation,” 6. 2025. DOI: 10.48550/arXiv.2408.15533. arXiv: 2408.15533.
- [46] C. Sharma, „Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers,” *arXiv preprint arXiv:2506.00054*, 2025., Pristupljeno: 26. srpanj 2025. Preprint pod recenzijom u ACM TOIS.
- [47] I. Novogroder. „Top 9 RAG Tools to Boost Your LLM Workflows.” Pristupljeno: 2025-06-02. adresa: <https://lakefs.io/blog/rag-tools/>.
- [48] S. Aquino. „An Introduction to Vector Databases - Qdrant.” Pristupljeno: 2. srpnja 2025. adresa: <https://qdrant.tech/articles/what-is-a-vector-database/>.
- [49] V. Baghel. „13 Best Vector Databases for AI-Powered Solutions.” Openxcell. Pristupljeno: 04. lipanj 2025. adresa: <https://www.openxcell.com/blog/best-vector-databases/>.
- [50] M. Vermeir. „What are large language models (LLMs)?” Accessed: Jul. 11, 2025, ABBYY. adresa: <https://www.abbyy.com/blog/large-language-models/>.
- [51] M. Ozkaya. „LLM Providers: OpenAI, Meta AI, Anthropic, Hugging Face, Microsoft, Google, and Mistral AI.” Medium. Pristupljeno: 04. lipanj 2025. adresa: <https://mehmetozkaya.medium.com/llm-providers-openai-meta-ai-anthropic-hugging-face-microsoft-google-and-mistral-ai-46ad8c027f6b>.
- [52] „What Is React? [Easily Explained].” Pristupljeno: 11. svibanj 2025. adresa: <https://www.simplilearn.com/tutorials/reactjs-tutorial/what-is-reactjs>.
- [53] M. Biberović. „U Hrvatskoj su najbolje plaćeni Go, Kotlin i Ruby programeri. Što je s ostalima?” [Na internetu]. Pristupljeno: 15. kolovoz 2025. adresa: <https://www.netokracija.com/programeri-hrvatska-place-tabu-227799>.

- [54] A. Gillis i R. Awati. „What is SMTP (Simple Mail Transfer Protocol)?” Pristupljeno: 08. srpanj 2025. adresa: <https://www.techtarget.com/whatis/definition/SMTP-Simple-Mail-Transfer-Protocol>.
- [55] Obafemi. „Building a Website with Python, FastAPI, and Streamlit.” Pristupljeno: 7. srpnja 2025., pogledano 7. srpnja 2025. adresa: <https://medium.com/@obaff/building-a-website-with-python-fastapi-and-streamlit-418f48c41af2>.

Popis slika

1.	Opći učeći agent - objašnjenje [[9], p. 74]	8
2.	Primjer kopilota kreiranog u Microsoft Copilot Studiu	13
3.	Adaptive cards - primjer	13
4.	Globalno tržište chatbotova; Izvor	15
5.	Ocjenjivanje zadatka - Gradescope [18]	17
6.	Popularnost RAG-a; Izvor	21
7.	Konceptualni tok korištenja RAG-a s LLM-ovima [20]	23
8.	Fino podešavanje proces (generalno); Izvor	27
9.	Utjecaj duljine konteksta na zastupljenost halucinacija u odgovoru [45]	32
10.	Nesklad između eksplicitnog i implicitnog dokaza [45]	33
11.	Arhitektura LPR4RAG metode za otkrivanje halucinacija [45]	33
12.	Višeskočno zaključivanje - primjer; Izvor	34
13.	Sažetak dostupnih alata i biblioteka za razvoj RAG sustava; Izvor	35
14.	Vektorska baza - primjer; Izvor	38
15.	Popularnost LLM-ova u rješavanju matematičkih problema za 2024 godinu; Izvor	40
16.	React komponenta	41
17.	Proces dostavljanja emaila; Izvor	42
18.	Interaktivna dokumentacija (OpenAPI) u Swagger stilu	43
19.	ReDoc prikaz dokumentacije	43
20.	Struktura praktičnog dijela rada	44
21.	Arhitektura sustava s prikazom toka zahtjeva i odgovora	45
22.	Struktura scraper foldera	47
23.	Struktura frontend dijela	51

24.	Struktura backend dijela	56
25.	Sadržaj env datoteke	57
26.	Pokretanje aplikacije kroz terminal	66
27.	Početna stranica aplikacije	66
28.	O Łódźu stranica	67
29.	Drugi gradovi stranica	67
30.	Forma za postavljanje pitanja stranica	68
31.	Stranica za pregled i upravljanje dokumentima	68
32.	Testiranje asistenta kada je dokument u bazi	69
33.	Testiranje asistenta kad dokument nije u bazi	69
34.	Groq platforma - pregled zahtjeva i potrošenih tokena; Izvor	70
35.	Groq limitacije; Izvor	70

Popis tablica

1.	Razlika između pojmova [11]	10
2.	Usporedba RAG pristupa i finog podešavanja modela [34]	26
3.	Usporedba OLTP, OLAP i vektorskih baza podataka [48]	38
4.	Struktura stranica i njihovih komponenti	46
5.	Korišteni izvori podataka i alati za obradu	47

Popis isječaka koda

1.	settings.json	44
2.	Scraper 1/6	47
3.	Scraper 2/6	48
4.	Scraper 3/6	49
5.	Scraper 4/6	49
6.	Scraper 5/6	49
7.	Scraper 6/6	50
8.	Chat.jsx	51
9.	Contact.jsx	52
10.	Document.jsx 1/5	52
11.	Document.jsx 2/5	52
12.	Document.jsx 3/5	52
13.	Document.jsx 4/5	53
14.	Document.jsx 5/5	53
15.	About.jsx komponenta	54
16.	LanguageContext.jsx	55
17.	Stranica Home	55
18.	App.jsx	56
19.	Main.py	57
20.	Vectorstore.py 1/4	58
21.	Vectorstore.py 2/4	58
22.	Vectorstore.py 3/4	59
23.	Vectorstore.py 4/4	59
24.	qa.py	60

25.	<code>Document_handler.py</code>	61
26.	<code>ask.py</code>	62
27.	<code>documents.py</code>	63
28.	<code>email.py</code>	63
29.	<code>Preview.py</code>	64
30.	<code>Stats.py</code>	65
31.	<code>Upload.py</code>	65